AXLERO

INTERNSHIP PROJECT REPORT

Advanced MERN Stack Engineering Programme — Project 1

SyncSpace

A Real-Time Collaborative Whiteboard and Code Editor

Conflict-free Replicated Data Types, Event-Sourced Session Replay and Sandboxed Multi-Language Execution over the MERN Stack

PROJECT ASSIGNED AND CARRIED OUT AT

Axlero Solutions

Innovating Solutions

SUBMITTED BY

Serah Joyce Rasquinha

TEAM MEMBERS

Antara • Anmol • Manasvi • Priyanshu Kumar • Shivani

PROJECT DURATION

13 July 2026 – 15 August 2026

ABSTRACT

The conventional web application is built on a request–response contract: a client asks, a server answers, and the last write wins. That contract is adequate for transactional systems and wholly inadequate for collaboration. When two people edit the same artefact at the same instant, "the last write wins" is not a synchronisation strategy — it is a data-loss policy.

SyncSpace is a real-time collaborative workspace that replaces that contract with a convergent one. It presents two panes that a distributed team shares simultaneously: an infinite whiteboard and a multi-language code editor. Every object either pane contains — freehand strokes, sixteen shape primitives, flowchart connectors, text blocks, uploaded images, the code buffer itself and the shared execution console — lives inside a single *Conflict-free Replicated Data Type* document implemented with Yjs. Edits are exchanged as opaque binary deltas over a Socket.IO relay, and convergence is a mathematical property of the data structure rather than a behaviour the server must arbitrate. Field-level merge semantics mean that one collaborator recolouring a rectangle while another drags it produces a document in which both intentions survive.

Three subsystems extend that core well beyond a drawing tool. First, an *event-sourced session replay* subsystem persists every relayed update to an append-only log; because a Yjs update is a causally ordered, self-contained delta, replaying a prefix of that log into an empty document reconstructs exactly the state that existed at that moment, which makes the entire history of a session scrubbable. Second, a *workspace permission system* built on two distinct classes of signed token places a genuine access boundary in front of the document: a socket holding only a lobby ticket has no synchronisation handler registered at all, so exclusion is structural rather than enforced by checks. Third, a *provider-abstracted execution service* compiles and runs JavaScript, Python, C, C++ and Java in a remote sandbox and publishes every result — stdout, stderr, compiler diagnostics, exit code, duration and attribution — into the same shared document, so the console is collaborative in the same sense the canvas is.

The report documents the architecture in depth, including a class of failure that only

appears at scale and is instructive precisely because no amount of functional testing at small sizes would surface it: the Socket.IO parser rejects any packet carrying more than ten binary attachments, which silently capped session replay at approximately ten updates and produced a diagnostic message that blamed the wrong component entirely. The resolution — a chunked streaming protocol in which each message carries exactly one packed buffer — is described alongside the update-coalescing scheme that reduced a sixty-frame drag from sixty log entries to a handful.

The implementation is verified by 442 automated assertions distributed across ten headless suites covering CRDT convergence, connector geometry, brush and eraser mathematics, replay reconstruction fidelity, the permission system over live sockets, execution across five languages and three providers, and a stress suite that drives a room past five thousand updates. Measured performance shows a 5,000-update history streaming back in approximately 43 ms across seventeen bounded chunks and reconstructing byte-for-byte in approximately 175 ms.

Keywords: Conflict-free Replicated Data Types; Operational Transformation; Yjs; Real-Time Collaboration; WebSockets; Socket.IO; Event Sourcing; Session Replay; MERN Stack; MongoDB; Sandboxed Code Execution; JSON Web Tokens; Konva; Monaco Editor; Awareness Protocol.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Expansion
ACL	Access Control List
API	Application Programming Interface
AST	Abstract Syntax Tree
BSON	Binary JSON (MongoDB's on-disk document encoding)
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CQRS	Command Query Responsibility Segregation
CRDT	Conflict-free Replicated Data Type
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DOM	Document Object Model
DoS	Denial of Service
DPI	Dots Per Inch
ER	Entity–Relationship
FIFO	First In, First Out
GCC	GNU Compiler Collection
GFM	GitHub Flavoured Markdown
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
JVM	Java Virtual Machine

JWT	JSON Web Token
LLM	Large Language Model
LRU	Least Recently Used xvii
MERN	MongoDB, Express, React, Node.js
MIME	Multipurpose Internet Mail Extensions
MVC	Model–View–Controller
NoSQL	Not Only SQL
OT	Operational Transformation
PNG	Portable Network Graphics
POSIX	Portable Operating System Interface
PWA	Progressive Web Application
RDP	Ramer–Douglas–Peucker (line simplification algorithm)
REST	Representational State Transfer
RGBA	Red, Green, Blue, Alpha
SDK	Software Development Kit
SPA	Single Page Application
SSE	Server-Sent Events
SVG	Scalable Vector Graphics
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TTFT	Time To First Token
UI	User Interface
UML	Unified Modeling Language
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
UX	User Experience
VM	Virtual Machine
WS	WebSocket
XSS	Cross-Site Scripting
Yjs	A CRDT framework for shared, concurrently edited data

CHAPTER 1

Introduction

1.1 Overview

Software is written by teams that are increasingly not in the same room. The technical interview, the architecture review, the pair-programming session and the classroom laboratory have all migrated to the browser, and each of them requires something the browser was not originally built to provide: two or more people acting on the same artefact at the same instant, each seeing the other's effect without perceptible delay, and neither destroying the other's work.

SyncSpace is a web application built to satisfy exactly that requirement. It presents a single *workspace* — a named, access-controlled room — containing two panes that all participants share simultaneously. The left pane is an infinite whiteboard on which any participant can draw freehand strokes, place any of sixteen geometric and flowchart primitives, wire those primitives together with intelligent connectors, add formatted text, and upload images. The right pane is a full code editor built on Monaco, the engine behind Visual Studio Code, in which all participants type into the same buffer and can compile and execute the result in five languages. Neither pane is a screen-share; both are genuinely shared documents, and every participant's client holds a complete, independently authoritative copy of the state.

The distinguishing engineering claim of this project is that convergence is not arbitrated. There is no server-side referee deciding whose edit survives when two clients modify the same object in the same millisecond. Instead the shared state is a *Conflict-free Replicated Data Type* (CRDT), a family of data structures whose merge operation is commutative, associative and idempotent. Given the same set of updates in any order, every replica provably arrives at the same state. The server's role is reduced to relaying opaque binary deltas and enforcing who is allowed to receive them — a considerably smaller and more defensible responsibility than adjudicating intent.

Around that core, SyncSpace layers three subsystems that are individually substantial: an event-sourced session replay facility that makes the entire history of a workspace scrubbable; a two-tier authentication and authorisation model with an administrator-controlled waiting room; and a provider-abstracted remote execution service that compiles and runs code in a sandbox and publishes the result into the shared document so that the console itself is collaborative.

1.2 Motivation

The project originated in the Axlero Solutions Advanced MERN Stack Engineering programme, whose stated premise is that a portfolio built from create–read– update–delete applications demonstrates familiarity with a stack but not competence in system design. The programme brief explicitly directs candidates away from to-do lists and storefronts and towards three architectural themes: real-time collaboration, complex event sourcing, and AI-augmented workflows. SyncSpace is the first of those — the real-time collaboration project.

Three specific frustrations shaped the design.

First, most "collaborative" tutorials are not collaborative. A very large fraction of published real-time examples broadcast the full document on every keystroke and let the last message received overwrite everything. This appears to work in a demonstration with one user and two browser tabs operated by the same hand, and fails the first time two people type at once. Building on a genuine CRDT was a deliberate rejection of that shortcut.

Second, whiteboards and editors are usually separate products. A technical interview typically requires both: a candidate sketches an architecture while an interviewer writes code against it. Splitting those across two applications splits the session's history across two systems and makes the correlation between "the diagram we drew" and "the code we wrote" impossible to recover afterwards. In SyncSpace both live in one CRDT document, which means a single snapshot captures both and a single replay reproduces both in step.

Third, session history is almost always discarded. Collaboration tools routinely persist the current state and throw away the path taken to reach it. Yet in an educational or interview context the path is often more valuable than the destination — it shows how a solution was reasoned towards. Because a Yjs update is a self-contained, causally ordered delta, retaining the updates costs little and buys the ability to reconstruct any intermediate state exactly.

1.3 Problem Statement

Problem Statement

Standard web applications operate on a request/response model. Building a system in which multiple users can draw on a shared canvas or type into a shared code buffer simultaneously — without race conditions, perceptible lag or data overwriting — requires state-synchronisation algorithms (Operational Transformation or Conflict-free

Replicated Data Types), an access-control model that is enforced at the transport layer rather than in the user interface, and a persistence strategy that distinguishes between *what the document is now* and *how it came to be that way*. None of these are covered by conventional full-stack development practice, and each of them fails in ways that are invisible during single-user testing.

Decomposed into its constituent technical problems, the statement above comprises the following:

1. **The concurrent-edit problem.** Two clients modify the same object within one network round-trip. A naive implementation loses one of the two modifications. A correct implementation must merge them at a granularity fine enough that a colour change and a position change to the same rectangle both survive.
2. **The late-joiner problem.** A third participant connects to a workspace in which substantial work already exists. They must receive the complete current state, not merely the updates issued after their arrival.
3. **The isolation problem.** Two workspaces running on the same server process must be mutually invisible. Isolation that depends on the client sending the correct room identifier is not isolation, because a client can send any identifier it chooses.
4. **The durability problem.** A server restart must not destroy a session. The shared state is binary and structurally complex, so persisting it as decoded JSON is both lossy and expensive.
5. **The history problem.** Reconstructing intermediate states requires an append-only record of changes, which is a fundamentally different storage shape from a periodically overwritten snapshot, and which introduces its own capacity, ordering and transport problems at scale.
6. **The execution problem.** Running arbitrary user-submitted code is a security liability. It must be isolated, resource-bounded, time-bounded, output-bounded, and it must fail informatively rather than by taking the collaboration server down with it.

1.4 Existing Systems

Several mature products occupy adjacent parts of this space. Understanding what each does well — and where each stops — is what justifies building a new system rather than composing existing ones.

1.4.1 Diagramming and Whiteboarding Tools

Miro and **FigJam** are commercial infinite-canvas whiteboards with excellent multi-user support, presence indicators and a large template library. **Excalidraw** is an open-source,

hand-drawn-aesthetic whiteboard with an end-to-end encrypted collaboration mode. **draw.io** (now diagrams.net) is a mature diagramming tool whose connector system — shapes that remain attached as boxes are moved, with automatic re-routing — is the specific behaviour SyncSpace's connector subsystem was designed to match.

All four are drawing tools. None embeds a compiler-backed code editor, and none exposes a scrubable history of the drawing session at update granularity.

1.4.2 Collaborative Code Editors

Google Docs pioneered mainstream operational transformation for text but is not a code editor. **Visual Studio Live Share** provides genuinely excellent shared editing with language services and shared debugging, but it is an extension to a desktop IDE rather than a browser application, and it requires every participant to install tooling. **Replit** and **CodeSandbox** offer collaborative browser IDEs with execution, but no drawing surface. **CoderPad** and **HackerRank CodePair** target technical interviews specifically and do provide a rudimentary whiteboard alongside the editor, though the whiteboard is typically a thin drawing layer rather than a structured object model.

1.4.3 Comparative Position

Table 1.1 places SyncSpace against these systems on the dimensions this project treats as load-bearing.

Table 1.1: Feature comparison of SyncSpace against representative existing systems

Capability	**Miro**	**Excali-draw**	**draw.io**	**VS Live Share**	**Coder-Pad**	**Sync-Space**
Real-time shared canvas	Yes	Yes	Partial	No	Partial	Yes
Structured shape object model	Yes	Yes	Yes	—	No	Yes
Attached auto-routing connectors	Yes	Partial	Yes	—	No	Yes
True partial vector eraser	Partial	No	No	—	No	Yes
Shared code buffer (CRDT)	No	No	No	Yes	Yes	Yes
In-browser multi-language run	No	No	No	Partial	Yes	Yes
Shared, attributed console	No	No	No	No	No	Yes
Update-level session replay	No	No	No	No	No	Yes
Administrator approval lobby	Partial	No	No	Partial	No	Yes
Runtime-switchable access policy	No	No	No	No	No	Yes
Self-hostable / open stack	No	Yes	Yes	No	No	Yes

The row that matters most is *update-level session replay*. Several products offer version history at the granularity of a named checkpoint or a periodic autosave. None of the surveyed systems allows a reviewer to move a slider and watch the document reconstruct itself one edit at a time, with each edit attributed to the participant who made it. That capability follows directly from the choice of a CRDT whose updates are individually meaningful, and it is the clearest illustration in this project of an architectural decision paying a dividend that was not its original justification.

1.5 Limitations of Existing Systems

Consolidating the survey, six limitations motivated the present work.

1. **Domain separation.** Drawing tools and coding tools are separate products. A session that legitimately needs both is fragmented across two applications with two histories, two access models and no shared timeline.
2. **Opaque or absent history.** Where history exists it is coarse — hourly snapshots, named versions — and cannot answer "what did the board look like immediately before that shape was deleted?"
3. **Coarse access control.** Access is typically a link: possession of the URL is authorisation. There is rarely an intermediate state in which a prospective participant is authenticated but not yet admitted, and rarer still an ability to change the admission policy while a session is running without disturbing the participants already inside.
4. **Closed, hosted architecture.** The most capable products are proprietary and hosted. An institution that must keep interview recordings or student work on its own infrastructure cannot use them.
5. **Execution coupled to the host.** Systems that do execute code frequently require a specific toolchain to be installed on the serving machine, which makes the language list a property of the deployment rather than of the product, and which turns a missing compiler into an opaque runtime failure.
6. **Object eraser rather than vector eraser.** Nearly every browser-based whiteboard implements "erase" as "delete the whole stroke you touched". Rubbing out the middle of a line and leaving both ends intact — the behaviour of a physical eraser — requires splitting the underlying point list, and is uncommon.

1.6 Proposed System

SyncSpace addresses each limitation above with a specific architectural commitment. The system is a MERN-stack single-page application composed of a React front end, an Express and Socket.IO back end, and MongoDB persistence, with Yjs providing the replicated data structure that binds them.

1.6.1 Core Architectural Commitments

1. **One document, two panes.** The whiteboard's shapes, the code buffer, the shared language selection, the shared run console and the workspace chat are five named top-level types inside a single `Y.Doc`. Because they share a document, they share a synchronisation channel, a persistence snapshot and a replay timeline. Replaying the

whiteboard therefore replays the editor for free — not because that was implemented, but because there was never a second thing to implement.

2. **Convergence by construction.** Every shape is a `Y.Map`, which makes merging occur at the level of the individual field. Concurrent edits to *different* fields of the same shape both survive; concurrent edits to the *same* field resolve deterministically and identically on every replica.
3. **Isolation by token, not by request.** A socket is placed into the room named by the workspace identifier embedded in its own signed token. There is no client-supplied room parameter anywhere in the protocol, so there is no message a malicious client can send to reach another workspace's document.
4. **Two storage shapes for two questions.** The `docstates` collection holds one debounced binary snapshot per room and answers "what does this board look like now?". The `updatelogs` collection holds one append-only row per update and answers "how did it get here?". The report refers to these throughout as the photograph and the film reel respectively.
5. **Execution as a replaceable adapter.** The execution service knows nothing about any specific provider. It walks a configured chain of adapters, each of which must return one canonical result shape. Switching from a hosted sandbox to a self-hosted one is a single environment-variable change; adding a new provider is one new file.
6. **Additive milestones.** Every subsystem added after the initial synchronisation core — connectors, brushes, the eraser, the executable editor, session replay, chat — was implemented without altering the synchronisation protocol. Each is an ordinary record in an existing shared type, and therefore inherits selection, undo, persistence, replication and replay without new code.

1.6.2 System at a Glance

Figure 1.1 shows the application's overall workflow from first contact through to an active collaborative session with replay available.

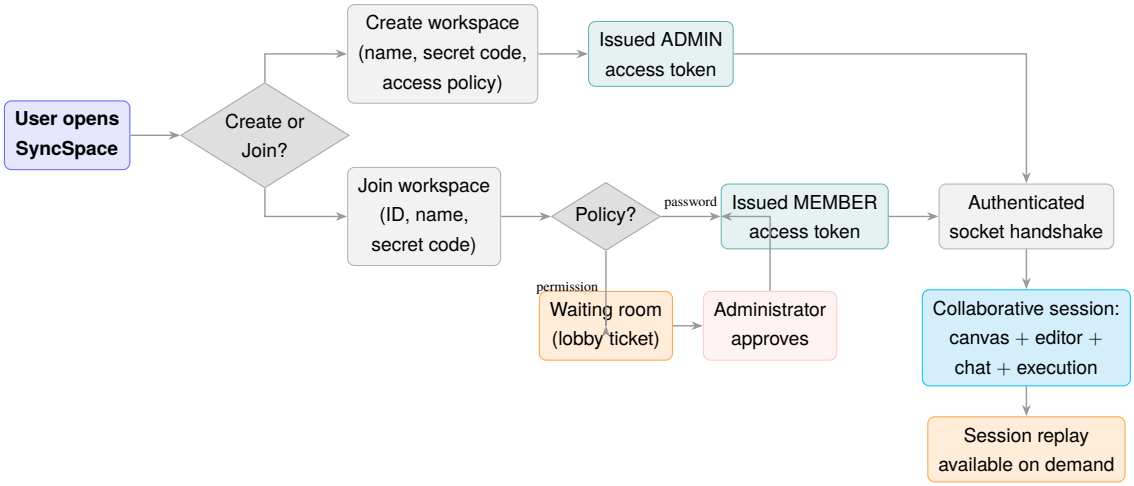


Figure 1.1: Overall application workflow, from landing page to an active collaborative session

1.7 Objectives

The project set out to achieve the following objectives, each of which is revisited and evidenced in the chapters indicated.

1. **To implement a low-latency, bidirectional synchronisation engine** over WebSockets that relays opaque binary deltas between clients in an isolated room, and that delivers the complete current state to any client joining late. *(Chapters 6, 8)*
2. **To achieve provable conflict-free convergence** by modelling the shared state as a CRDT with field-level merge granularity, and to demonstrate through automated tests that concurrent edits to the same object are merged rather than lost. *(Chapters 8, 10)*
3. **To build a structured whiteboard object model** supporting freehand strokes, sixteen shape primitives, attached auto-routing connectors, formatted text and images, all as records in one shared array so that each new type inherits selection, styling, undo, persistence and replication without bespoke code. *(Chapter 7)*
4. **To integrate a professional code editor** bound directly to a shared text type, with remote cursors and selections, and to extend it into an executable development environment with a shared language selection and a shared, attributed console. *(Chapter 7)*
5. **To design a persistence layer with two distinct shapes** — a periodically overwritten binary snapshot for current state and an append-only log for history — each of which degrades gracefully to an in-memory equivalent when no database is configured. *(Chapter 6)*
6. **To implement session replay** such that any prefix of the update log reconstructs the exact document state at that point, drawn by the same renderer as the live board, and read-only by construction rather than by defensive programming. *(Chapter 8)*

7. **To enforce authentication and authorisation at the transport layer**, using two distinct classes of signed token so that a participant awaiting approval is structurally incapable of reading the document. *(Chapter 9)*
8. **To execute user-submitted code safely** in five languages behind the same authorisation boundary as every other operation, with resource, time and output bounds, provider fallback and informative failure. *(Chapters 6, 9)*
9. **To validate the implementation empirically** through headless automated suites covering geometry, CRDT semantics, reconstruction fidelity, the permission system over live sockets, execution across languages and providers, and behaviour under stress. *(Chapter 10)*

1.8 Scope of the Project

1.8.1 In Scope

The delivered system includes the complete collaborative workspace: workspace creation and joining with two access policies, an administrator control panel with a live join-request queue and member management, an authenticated socket transport, the CRDT synchronisation engine with presence and awareness, the full whiteboard object model with sixteen shapes and a connector system, a seven-style brush engine with a true partial eraser, undo and redo across every operation, zoom and pan, PNG export, a collaborative Monaco editor with five-language execution and a shared console, workspace chat, binary snapshot persistence, append-only update logging, the streamed replay protocol and the replay scrubber, together with the automated test suites that verify all of the above.

1.8.2 Out of Scope

The following were consciously excluded and are discussed as future work in Section 12.3:

- **Global user identity.** SyncSpace has no cross-workspace account system. Identity is workspace-scoped: a participant is a member of one workspace, not a user of the platform. This is a deliberate design decision defended in Section 5.3, not an omission.
- **Horizontal scaling.** One server process owns a workspace's authoritative document and its sequence counter. Multi-process deployment would require both to move into shared storage.
- **Audio and video conferencing.** Out of scope; text chat is provided instead.
- **Container-level execution isolation.** The default execution path delegates to a remote sandboxed provider; the optional local runner is process-level rather than container-level and is disabled by default.

- **Offline-first operation.** Yjs would merge offline edits on reconnection, but no service worker or offline shell was implemented.
- **Mobile-optimised layout.** The interface targets desktop viewports; the two-pane layout is not responsive below tablet width.

1.9 Organisation of the Report

The remainder of this report is organised as follows. **Chapter 2** surveys the literature on operational transformation, CRDTs and event sourcing that underpins the design. **Chapter 3** specifies the technology stack, the software and hardware requirements and the development environment. **Chapter 4** presents the high-level and low-level architecture, the component and deployment views, and the data flow diagrams. **Chapter 5** covers system design: use cases, class structure, the entity–relationship and database schema design, sequence, activity and state-transition models. **Chapter 6** documents the server implementation module by module. **Chapter 7** documents the client implementation. **Chapter 8** isolates and analyses the significant algorithms, including CRDT merge semantics, replay reconstruction, connector routing, stroke simplification and the eraser sweep. **Chapter 9** covers the security architecture, validation, error handling and performance optimisation. **Chapter 10** describes the testing methodology and results. **Chapter 11** presents the user interface with annotated figures. **Chapter 12** states the advantages, limitations, future enhancements and conclusions. References and appendices follow.

CHAPTER 2

Literature Survey

2.1 Introduction to the Survey

The technical problem at the centre of SyncSpace — keeping several replicas of a mutable document identical while each replica accepts edits independently and without coordination — has a research history spanning more than three decades. This chapter surveys that history in four strands: the operational transformation tradition, the conflict-free replicated data type tradition, the event-sourcing tradition from which session replay is drawn, and the practical literature on real-time web transports and untrusted code execution. Each strand concludes with the specific decision it informed in this project.

2.2 The Concurrency Problem in Shared Editing

2.2.1 Why Locking Fails

The classical database answer to concurrent mutation is mutual exclusion: a writer acquires a lock, mutates, and releases. Ellis and Gibbs [1] observed early that this answer is unusable for interactive group editing. Locking imposes a round trip on every keystroke, which destroys the responsive feel that makes collaboration worth doing, and it introduces a failure mode — a participant holding a lock whose network drops — that has no graceful resolution. Their conclusion, which the field has broadly accepted since, is that interactive collaborative editing must be *optimistic*: every replica applies local edits immediately and reconciliation happens afterwards.

Optimism creates the problem this project exists to solve. If every replica applies edits immediately and in its own order, the replicas diverge. Something must guarantee that they reconverge.

2.2.2 Operational Transformation

Operational Transformation (OT), introduced in the GROVE system by Ellis and Gibbs [1] and substantially developed by Sun and Ellis [2], achieves reconvergence by transforming operations against one another. If replica A inserts a character at position 3 while replica B concurrently deletes the character at position 1, then A's insertion must be transformed

into an insertion at position 2 before B applies it. The *transformation function* encodes, for each pair of operation types, how each must be adjusted in the presence of the other.

OT was the basis of Google Wave and remains the basis of Google Docs. Its strengths are a compact wire format — an operation is small and human-legible — and a mature body of implementation experience. Its principal weakness is the difficulty of proving the transformation functions correct. Oster et al. [3] demonstrated that several published OT algorithms violated the transformation properties (commonly designated TP1 and TP2) they claimed to satisfy, and therefore did not in fact guarantee convergence in all interleavings. A further practical weakness is that OT conventionally requires a central server to impose a total order on operations, which makes it a poor fit for peer-to-peer or offline-tolerant topologies.

2.2.3 Conflict-free Replicated Data Types

Shapiro et al. [4] formalised an alternative. A CRDT is a data type whose merge operation is a join on a semilattice: commutative, associative and idempotent. Given those three properties, replicas that have received the same *set* of updates — in any order, with any duplication — are provably in the same state. Convergence therefore becomes a property of the data structure rather than a property of an algorithm applied to the data structure, which is the essential difference from OT.

The literature distinguishes state-based CRDTs (which exchange complete merged states) from operation-based CRDTs (which exchange deltas and require causal delivery). Preguica [5] surveys the trade-off: state-based designs tolerate any delivery guarantee but transmit more, while operation-based designs are compact but need the transport to preserve causal order.

The relevant CRDT for text is the sequence CRDT, of which several designs exist — WOOT [3], Logoot, Treedoc, RGA and YATA. Nicolaescu et al. [6] introduced *YATA* (Yet Another Transformation Approach), the algorithm implemented by Yjs. YATA models a sequence as a doubly linked list of items, each carrying a unique identifier and origin references naming the items it was inserted between. Concurrent insertions at the same position are ordered deterministically by comparing client identifiers, and deletions are represented as tombstones. The design achieves $O(1)$ amortised insertion in the common case of sequential typing and avoids the identifier growth that affects position-identifier schemes such as Logoot.

2.2.4 Decision Taken

SyncSpace adopts an operation-based CRDT (Yjs/YATA) rather than OT for three reasons drawn directly from the survey:

1. **Correctness is structural.** The convergence guarantee follows from the algebra of the merge, not from the exhaustiveness of a transformation matrix. Given the finding of Oster et al. [3] regarding published OT implementations, this is a materially lower-risk foundation for a project of this size.
2. **The server can be simple.** A CRDT relay does not need to understand the operations it forwards, so the server handles opaque binary buffers. This directly enabled the persistence design (store the same opaque bytes) and the replay design (re-apply the same opaque bytes), neither of which would be as simple with a transformation-aware server.
3. **Heterogeneous shared types.** Yjs provides `Y.Array`, `Y.Map` and `Y.Text` within one document. The whiteboard needs an array of maps; the editor needs text; the console needs an array. A single document containing all of them was decisive for the single-timeline replay described in Section 8.2.

2.3 Awareness and Presence

Convergent document state is necessary but not sufficient for collaboration. Participants must also see one another. Gutwin and Greenberg [7] established the concept of *workspace awareness* — the up-to-the-moment understanding of another person's interaction with the shared space — and demonstrated empirically that its absence increases coordination errors even when the shared document itself is correct.

Critically, awareness data has different requirements from document data. A cursor position from three seconds ago is worthless; a shape created three seconds ago is not. Awareness state should therefore be ephemeral, non-persistent and excluded from history. Yjs reflects this by shipping the awareness protocol as a *separate* module from the document CRDT, with its own state map keyed by client identifier and its own update encoding.

Decision taken. SyncSpace relays awareness on a distinct socket event (`awareness-update`) from document synchronisation (`sync-update`). Awareness carries cursor position, the current selection, the in-progress freehand stroke, the eraser ring position and the chat typing indicator. None of it is persisted, none of it enters the update log, and none of it appears in replay — which is correct, because replay reconstructs the artefact, not the pointers that were hovering over it.

2.4 Event Sourcing and Temporal Reconstruction

2.4.1 The Pattern

Event sourcing, as described by Fowler [8] and elaborated by Vernon [9], stores an application's state as an ordered, append-only sequence of the events that produced it, rather

than as a mutable representation of the current state. Current state is a fold over the event stream. The pattern's characteristic benefits are a complete audit trail, temporal queries ("what was the state at time t?") and the ability to derive new projections retroactively.

Its characteristic costs are storage growth proportional to activity rather than to data size, and the expense of replaying long streams — conventionally mitigated by periodic snapshots that allow replay to begin from an intermediate point rather than from the origin.

2.4.2 Relevance to CRDT Systems

The intersection of event sourcing with CRDTs is unusually favourable, and it is worth stating precisely why. In a conventional event-sourced system the events are domain-level commands (*OrderPlaced*, *ItemShipped*) and replaying them requires the domain logic that interprets them. If that logic changes, old events may replay differently — a well-documented versioning hazard.

In an operation-based CRDT the events are the CRDT updates themselves. They are self-describing, causally ordered and interpreted by the CRDT library rather than by application code. Consequently:

1. **No interpretation layer can drift.** Replay applies exactly the bytes that were originally relayed to collaborators, so a reconstructed state cannot disagree with the state that actually existed.
2. **A prefix is always valid.** Because updates form a causal chain, applying updates $0 \ldots k$ yields a complete, internally consistent document. This is the property on which the entire replay subsystem rests, and it has a sharp corollary: a *suffix* is not valid, which determines that the history cap must trim the tail and never the head (Section 6.5.3).
3. **Snapshotting is native.** `Y.encodeStateAsUpdate` produces a single update equivalent to the entire history, so the classical event-sourcing snapshot optimisation is one library call — used both for the `docstates` collection and for the replay cache's checkpoints.

Decision taken. SyncSpace maintains both projections described in the literature: a periodically overwritten snapshot for fast recovery, and an append-only log for temporal reconstruction. Section 5.3.3 argues why one cannot substitute for the other.

2.5 Real-Time Web Transports

Fette and Melnikov's WebSocket protocol [10] established a full-duplex, low-overhead channel over a single TCP connection, replacing the long-polling and streaming workarounds

that preceded it. Socket.IO layers on top of WebSocket a set of pragmatic additions: automatic transport negotiation and fallback, automatic reconnection with backoff, acknowledgement callbacks, namespaces, and a room abstraction for server-side multicast.

The literature on Socket.IO is largely practitioner documentation rather than peer-reviewed work, but one property proved decisive in this project and is documented only in the library's own source and issue tracker: the *binary attachment ceiling*. Socket.IO's packet encoder extracts every binary value nested anywhere within an emitted payload and transmits it as a separate attachment frame; `socket.io-parser` version 4.2.5 introduced a hard limit of ten attachments per packet as a denial-of-service mitigation, and rejects any packet exceeding it.

Decision taken. This ceiling directly determined the design of the replay transport protocol. A response carrying one buffer per log entry is structurally impossible for any session longer than ten updates. The chunked protocol described in Section 8.3 — in which each message carries exactly one packed buffer regardless of how many entries it contains — was designed specifically against this constraint. The episode is discussed in detail because it is an instructive example of a defect that is invisible in small-scale functional testing and that presents with a symptom ("the server did not answer") pointing at an entirely innocent component.

2.6 Vector Graphics and Canvas Rendering

Two rendering models are available in the browser: retained-mode SVG, in which each graphic is a DOM node, and immediate-mode Canvas 2D, in which the application redraws pixels. SVG offers native hit-testing and CSS integration but degrades once the node count reaches the low thousands, since each node participates in layout and style resolution. Canvas has no such ceiling but provides no built-in notion of an object.

Konva occupies a middle position: it maintains a scene graph of objects over a Canvas surface, providing object-level events, hit detection via a hidden hit canvas, transformation handles and layer caching, without incurring DOM cost per object.

Two classical geometry algorithms from the graphics literature are used directly in the drawing subsystem. The Ramer–Douglas–Peucker algorithm [**?**] recursively discards points lying within a tolerance of the chord joining the retained endpoints, and is the standard technique for compacting digitised curves — here applied so that a densely-sampled pointer path is stored, synchronised and re-rendered compactly. Chaikin's corner-cutting algorithm [**?**] replaces each vertex with two points at one quarter and three quarters along its adjacent edges, converging on a quadratic B-spline; one iteration is sufficient to remove the visible faceting from a hand-drawn stroke.

Decision taken. SyncSpace renders through Konva via `react-konva`, applies RDP simplification at stroke commit time (reducing both the persisted record and the network message), and applies one Chaikin pass at render time when the user has smoothing enabled. The ordering matters and is analysed in Section 8.6: simplifying before smoothing gives a compact record that still renders smoothly, whereas the reverse gives a large record and no additional visual benefit.

2.7 Untrusted Code Execution

Executing arbitrary user-supplied code is a well-characterised hazard. Goldberg et al. [**?**] established the system-call interposition model for confining untrusted helper applications, which remains the conceptual ancestor of modern sandboxes. Contemporary practice, surveyed in the container security literature, layers namespaces, control groups, seccomp filters and resource limits.

Judge0 [**?**] is an open-source execution service used widely in competitive-programming and educational platforms. It executes submissions inside *isolate*, a sandbox originally developed for the International Olympiad in Informatics that uses Linux namespaces and control groups to bound CPU time, wall-clock time, memory, process count and file-system access. It exposes a documented HTTP interface returning stdout, stderr, compiler output, exit code, exit signal, wall time and peak memory as distinct fields.

Decision taken. The execution subsystem was deliberately restructured during development, and the reasoning is instructive. The initial implementation forked compilers on the collaboration server and confined them with POSIX `ulimit` settings and a throwaway working directory. That approach produced a persistent stream of environment-specific defects — a missing mathematics library at link time, platform-specific executable naming, and most instructively an address-space limit that terminated V8 and the JVM at startup because managed runtimes reserve gigabytes of virtual address space up front. It also made the language list a property of whichever machine happened to be serving.

The revised design delegates execution to a remote provider behind an adapter interface, with a chain of fallbacks and one canonical result shape. This removes an entire class of deployment-coupling defects, removes untrusted execution from the collaboration host entirely, and reduces the security surface to a network call. The original local runner is retained as an explicitly opt-in, clearly-labelled development escape hatch.

2.8 Summary of Findings

Table 2.1 consolidates the survey and the design decision each strand produced.

Table 2.1: Literature survey summary and resulting design decisions

Source / Area	Key Finding	Decision Taken in SyncSpace
Ellis & Gibbs [1]	Locking is unusable for interactive group editing; optimism is required	Local edits apply immediately; reconciliation is deferred to the merge
Sun & Ellis [2]; Oster et al. [3]	OT transformation functions are hard to prove; several published ones were incorrect	CRDT chosen over OT so convergence is structural, not algorithmic
Shapiro et al. [4]	Commutative, associative, idempotent merge yields provable convergence	Shared state modelled entirely as CRDT types
Nicolaescu et al. [6]	YATA gives an efficient sequence CRDT with deterministic concurrent ordering	Yjs adopted as the CRDT implementation
Preguica [5]	Operation-based CRDTs are compact but need causal delivery	Socket.IO over TCP provides ordered delivery per connection
Gutwin & Greenberg [7]	Workspace awareness reduces coordination error	Separate ephemeral awareness channel for cursors, selections, drafts
Fowler [8]; Vernon [9]	Append-only event log enables temporal queries; snapshots bound replay cost	`updatelogs` collection plus checkpointed replay cache
Fette & Melnikov [10]; Socket.IO parser	Ten-attachment ceiling per packet	Chunked replay stream with one packed buffer per message
Douglas & Peucker [**?**]	Tolerance-based polyline simplification	RDP applied at stroke commit to compact records
Chaikin [**?**]	Corner cutting converges to a smooth spline	One Chaikin pass at render time for smoothing
Goldberg et al. [**?**]; Judge0 [**?**]	Untrusted execution requires layered confinement	Execution delegated to a remote sandboxed provider behind an adapter

2.9 Research Gap Addressed

The survey establishes that each individual technique employed here is well-founded. The gap this project addresses is not in any one technique but in their composition. Specifically,

no surveyed system combines:

- a structured, object-model whiteboard and a compiler-backed code editor inside *one* CRDT document, such that a single snapshot, a single access boundary and a single replay timeline cover both; and
- update-granularity temporal reconstruction that is read-only by construction and rendered by the same code path as the live view, so that history cannot diverge from reality through implementation drift.

Demonstrating that this composition is achievable — and documenting the non-obvious failure modes encountered while achieving it — is the contribution of this work.

CHAPTER 3

System Requirements and Development Environment

3.1 Technologies Used

SyncSpace is implemented on the MERN stack — MongoDB, Express, React and Node.js — augmented by four specialist libraries that carry the parts of the system the stack itself does not address: Yjs for replicated state, Socket.IO for the real-time transport, Konva for canvas rendering and Monaco for code editing. Table 3.1 enumerates the complete stack together with the specific responsibility each element discharges.

Table 3.1: Technology stack and the responsibility of each component

Technology	Version	Layer	Responsibility in SyncSpace
Node.js	18 LTS+	Runtime	Server process; native `fetch` used by the execution HTTP client
Express	4.19	Backend	REST routing, JSON body parsing, middleware chain, central error handler
Socket.IO	4.7	Backend	Authenticated WebSocket transport, rooms, acknowledgements, binary framing
MongoDB	6.0+ / Atlas	Database	Durable storage of workspaces, binary document snapshots and update logs
Mongoose	8.5	Backend	Schema definition, indexes, query construction
Yjs	13.6	Shared	The CRDT itself — used identically on client and server
y-protocols	1.0	Frontend	Awareness protocol (cursors, selections, presence)
y-monaco	0.1	Frontend	Binding between `Y.Text` and the Monaco text model

Continued on next page

Table 3.1 – continued from previous page

Technology	Version	Layer	Responsibility in SyncSpace
React	19.2	Frontend	Component model, hooks, declarative rendering
React Router	8.3	Frontend	Client-side routing across the five application screens
Konva	9.3	Frontend	Canvas scene graph, hit detection, transformer handles
react-konva	19.2	Frontend	React reconciler bindings for Konva nodes
Monaco Editor	0.52	Frontend	The code editor (syntax, folding, find/replace, multi-cursor)
Vite	7.3	Build	Development server with hot module replacement; production bundling
bcryptjs	2.4	Security	Hashing of workspace secret codes at cost factor 10
jsonwebtoken	9.0	Security	Signing and verification of access tokens and lobby tickets
CORS	2.8	Security	Origin restriction for both HTTP and the socket handshake
dotenv	16.4	Config	Environment configuration loading
esbuild	bundled	Testing	Bundles real ES-module frontend sources for headless test suites
jsdom	24.1	Testing	DOM environment for the component rendering suite

3.1.1 Rationale for Key Selections

Yjs over Automerge or a bespoke implementation. Automerge is the other mature JavaScript CRDT library, but its historical performance profile for long text sequences was weaker and its document model is JSON-centric rather than offering distinct shared types. Yjs's provision of `Y.Array`, `Y.Map` and `Y.Text` within one document is what allows the whiteboard, the editor, the console and the chat to share a single synchronisation channel, snapshot and replay timeline. Writing a CRDT from scratch was rejected on the basis of Section 2: the class of defect that afflicted published OT implementations applies equally to hand-rolled CRDTs, and would not be detectable within the project timeline.

Socket.IO over a raw WebSocket. The raw protocol provides no reconnection, no ac-

knowledgement mechanism and no server-side multicast primitive. Socket.IO supplies all three. Its room abstraction maps exactly onto the workspace concept and is the mechanism by which isolation is enforced.

Konva over raw Canvas or SVG. Sixteen shape types, multi-select transformation handles, rotation-aware hit testing and per-object event dispatch would each have required bespoke implementation on a raw canvas. SVG was rejected because a whiteboard is expected to hold thousands of freehand strokes, at which point per-object DOM nodes become a layout liability.

Monaco over CodeMirror. Monaco is the editor from Visual Studio Code and brings the behaviours users already expect — bracket matching, folding, multi-cursor, find and replace, go-to-line — with no additional configuration. The `y-monaco` binding connects its text model directly to a `Y.Text`, including remote selection decorations, which is precisely the Week 3 requirement of the project brief.

3.2 Software Requirements

3.2.1 Development Machine

Table 3.2: Software requirements for development

Requirement	Specification	Notes
Operating system	Windows 10/11, Linux or macOS	Development was carried out on Windows 10
Node.js runtime	18.0 LTS or later	18+ required for global `fetch` and `crypto.randomUUID`
Package manager	npm 9 or later	Ships with Node.js
Database	MongoDB 6.0+, or MongoDB Atlas	Optional — the server degrades to memory-only mode
Browser	Chrome 110+, Edge 110+, Firefox 110+	Chrome used for two-tab collaboration demonstration
Editor	Visual Studio Code (recommended)	Any editor is sufficient
Version control	Git 2.30+	

MongoDB is optional, and that is deliberate

Local MongoDB 8.x will not install on the Windows 10 development machine used for this project. Rather than allow that to block development, both the workspace store and the update-log store were written against a *dual-mode repository contract*: identical call signatures, backed either by Mongoose or by an in-memory structure, selected by a single flag set at boot from whether the connection succeeded. Every feature — including replay — is therefore fully demonstrable with no database present, and promoting the deployment to durable storage on Atlas is a one-line environment change with no code modification. This is described further in Section 6.5.1.

3.2.2 Runtime and Deployment

The production deployment requires only a Node.js host for the backend, a static host or CDN for the built frontend bundle, and a MongoDB endpoint. Because code execution is delegated to a remote provider, the server host requires *no* compilers, SDKs, language runtimes, container engine or build tools of any kind — an explicit design goal, discussed in Section 6.6.

Table 3.3: Runtime environment variables

Variable	Default	Purpose
`PORT`	5000	Backend listen port
`CLIENT_ORIGIN`	`http://localhost:5173`	CORS allow-list for HTTP and socket handshake
`MONGO_URI`	(empty)	Connection string; empty selects memory-only mode
`JWT_SECRET`	dev placeholder	Signing key for both token classes
`VITE_SERVER_URL`	`http://localhost:5000`	Backend address baked into the frontend bundle
`EXEC_PROVIDERS`	`judge0,piston,paiza`	Ordered execution provider chain
`JUDGE0_URL` / `JUDGE0_KEY`	(unset)	Primary execution provider endpoint and key
`PISTON_URL`	(unset)	Self-hosted Piston endpoint
`PAIZA_ENABLED`	true	No-signup fallback provider
`EXEC_RUN_TIMEOUT_MS`	8000	Wall-clock limit per execution

Continued on next page

Table 3.3 – continued from previous page

Variable	Default	Purpose
`EXEC_MEMORY_KB`	128000	Memory ceiling requested of the provider
`EXEC_MAX_CONCURRENT`	8	Simultaneous executions before queueing
`EXEC_RATE_MAX`	20	Executions per user per 30-second window

3.3 Hardware Requirements

Table 3.4: Hardware requirements

Component	Development (min.)	Development (rec.)	Server
Processor	Dual-core 2.0 GHz	Quad-core 2.5 GHz+	2 vCPU
Memory	4 GB RAM	8–16 GB RAM	1–2 GB RAM
Storage	2 GB free	5 GB free (SSD)	10 GB
Display	1366×768	1920×1080 or higher	—
Graphics	Integrated	Integrated with hardware canvas acceleration	—
Network	2 Mbps	10 Mbps	100 Mbps

Server memory is modest because the process holds only one `Y.Doc` per active workspace and forwards opaque buffers; it performs no rendering, no compilation and no image processing. The dominant client-side cost is canvas rasterisation, which is why a display capable of hardware-accelerated canvas compositing is recommended rather than a discrete graphics processor.

3.4 Functional Requirements

Table 3.5: Functional requirements

ID	Requirement	Description	Priority
FR-01	Workspace creation	A user creates a named workspace with a secret code and an access policy, becoming its sole administrator	High
FR-02	Workspace joining	A user joins with workspace ID, display name and secret code	High
FR-03	Access policy	Administrator selects between immediate password entry and explicit approval	High
FR-04	Waiting room	A prospective member awaiting approval is held without document access	High
FR-05	Approval workflow	Administrator sees live join requests and approves or rejects each	High
FR-06	Member removal	Administrator removes a member, whose sockets are disconnected immediately	Medium
FR-07	Real-time canvas sync	All shape operations propagate to every connected member	High
FR-08	Conflict-free merge	Concurrent edits to distinct fields of one object both survive	High
FR-09	Late-joiner sync	A client connecting to an existing workspace receives the full current state	High
FR-10	Presence and cursors	Named, coloured cursors and remote selections render for every peer	High
FR-11	Drawing tools	Seven brush styles with colour, thickness, opacity, smoothing and pressure	High
FR-12	Partial eraser	Erasing the middle of a stroke leaves the remaining segments intact	Medium
FR-13	Shape primitives	Sixteen shape types with fill, stroke, gradient, shadow and corner radius	High
FR-14	Connectors	Arrows attached to shapes that re-route automatically when shapes move	High
FR-15	Text objects	Drag-defined text regions with font, size, weight, alignment and wrapping	Medium

Continued on next page

Table 3.5 – continued from previous page

ID	Requirement	Description	Priority
FR-16	Images and stickers	Uploaded raster images sized from natural dimensions	Low
FR-17	Undo and redo	Every operation reversible; a peer's edits never reverted by local undo	High
FR-18	Zoom and pan	Per-user camera from 0.2× to 4× with constant-size handles	Medium
FR-19	PNG export	The current board exported as a raster image	Low
FR-20	Collaborative editor	One shared code buffer with remote cursors and selections	High
FR-21	Shared language	Language selection applies to every participant simultaneously	Medium
FR-22	Code execution	Five languages compiled and run with stdin, returning structured results	High
FR-23	Shared console	Every run result visible to all members, attributed and persisted	Medium
FR-24	Workspace chat	Text chat with typing indicators and unread notification	Low
FR-25	Persistence	Document state survives a server restart	High
FR-26	Update logging	Every relayed update appended to an ordered history	High
FR-27	Session replay	Scrub, play and step through the session's entire history	High
FR-28	Rate limiting	Join attempts and execution requests bounded per principal	Medium

3.5 Non-Functional Requirements

Table 3.6: Non-functional requirements

ID	Attribute	Requirement
NFR-01	Latency	A remote edit must appear within one network round trip; no server-side transformation delay is permitted
NFR-02	Correctness	Convergence must be a structural property, verified by automated tests over concurrent edit interleavings
NFR-03	Isolation	No message a client can construct may reach another workspace's document or history
NFR-04	Availability	Failure of logging, persistence or execution must degrade the affected feature only, never live collaboration
NFR-05	Scalability	Hundreds of objects per board with live connector re-routing must remain interactive
NFR-06	Security	Secrets hashed; tokens signed; no credential ever crosses to the browser
NFR-07	Usability	Every destructive or irreversible action reversible via undo, except workspace closure
NFR-08	Portability	The server must require no compilers, SDKs or container runtime
NFR-09	Testability	Logic capable of being wrong must be importable and testable without a browser
NFR-10	Observability	Every failure path must produce a specific, human-readable diagnostic, not a generic one

3.6 Development Environment

3.6.1 Project Layout

The repository is a two-package workspace. The backend and frontend have independent dependency trees and independent test scripts, joined by a root launcher.

```
ProjectFolder/
   backend/
       config/db.js
       models/      Workspace.js DocState.js UpdateLog.js
       routes/      workspaceRoutes.js executeRoutes.js
       controllers/ workspaceController.js
       middleware/  authMiddleware.js
       services/    socketService.js workspaceService.js
                    workspaceStore.js updateLogService.js
                    realtime.js execution/
       utils/       token.js validate.js ids.js
       server.js
   frontend/
       src/pages/   Landing CreateWorkspace JoinWorkspace
                    WaitingRoom Workspace
       src/components/ Canvas Editor Toolbar PropertyPanel
                    BrushPanel AdminPanel ReplaySlider ChatPanel
       src/canvas/  shapes shapeDoc connectors brushes
                    normalize replay ShapeNode ConnectorNode
       src/hooks/   useCollaboration.js useToasts.js
       src/utils/   socket.js session.js api.js
   dev.bat      one-click local launcher
   *.md         design documentation
```

Figure 3.1: Repository structure of the SyncSpace project

3.6.2 Layering Discipline

The directory structure encodes a deliberate rule that recurs throughout the implementation chapters: **code that can be numerically or logically wrong imports nothing that requires a browser**. The modules `canvas/connectors.js`, `canvas/brushes.js`, `canvas/normalize.js` and `canvas/replay.js` contain the geometry, the stroke mathematics, the record sanitisation and the history reconstruction respectively, and none of them imports React or Konva. Consequently all four are unit-tested headlessly against the genuinely shipped sources, bundled by esbuild, rather than against reimplementations. The React components consume these modules and decide only what to draw.

3.6.3 Local Launcher

A Windows batch launcher, `dev.bat`, removes the manual steps from local startup. It validates the Node and npm installations and the folder structure, installs missing dependency trees, creates a `.env` from the example when absent, checks both ports for conflicts, starts the backend and *waits until it is genuinely listening* before starting the frontend, and finally opens a browser once the development server responds. The ordering matters: starting both concurrently produces a frontend that fails its first API call and shows a connection

error the user must manually recover from.

3.6.4 Build and Test Commands

Table 3.7: Development, build and test commands

Command	Location	Effect
`npm run dev`	backend	Starts the API and socket server under nodemon
`npm run dev`	frontend	Starts the Vite development server with HMR
`npm run build`	frontend	Produces the optimised production bundle
`node test-shapes.mjs`	frontend	Shape CRDT convergence suite
`node test-connectors.mjs`	frontend	Connector geometry and synchronisation suite
`node test-brushes.mjs`	frontend	Brush, eraser and undo suite
`node test-replay.mjs`	frontend	Reconstruction, cache and chunk-unpacking suite
`npm run test:render`	frontend	Component rendering suite under jsdom
`node test-updatelog.mjs`	backend	Log coalescing, ordering and cap policy (no server)
`node test-workspace.mjs`	backend	Permission system over live sockets
`node test-replay.mjs`	backend	Replay wire protocol over live sockets
`node test-execute.mjs`	backend	Execution adapters against a wire-format mock
`node stress-replay.mjs`	backend	Long-session transport stress suite

3.7 Browser Compatibility

Table 3.8: Browser compatibility matrix

Browser	Min.	Canvas	WebSocket	Monaco	Notes
Chrome	110	Full	Full	Full	Reference platform
Edge	110	Full	Full	Full	Chromium-equivalent
Firefox	110	Full	Full	Full	Minor font-metric differences in text objects
Safari	16	Full	Full	Full	`multiply` blend for the highlighter renders slightly lighter
Opera	96	Full	Full	Full	Chromium-equivalent
Mobile browsers	—	Partial	Full	Partial	Two-pane layout is not responsive below tablet width

The only feature with a genuine compatibility caveat is the highlighter brush, which uses the `multiply` composite operation so that overlapping strokes darken as physical ink does. Composite-operation colour management differs subtly between rendering engines; the effect is present everywhere but its exact tone varies.

CHAPTER 4

Software Architecture

4.1 Architectural Style

SyncSpace is a layered client–server application with an unusual property: the authoritative state is *replicated* rather than centralised. Each connected browser holds a complete copy of the workspace document and applies its own edits without asking permission. The server holds a copy too, but its copy is authoritative only in the narrow sense of being the one handed to clients that arrive late — it never arbitrates between conflicting edits, because the CRDT makes arbitration unnecessary.

This produces a hybrid style. The management plane — creating workspaces, joining, approving, removing members, executing code — is a conventional three-tier REST architecture with routes, controllers, a service layer and a repository. The data plane — everything that happens inside a live workspace — is an event-driven replication architecture in which the server is a relay with an access check in front of it.

Table 4.1: Architectural responsibilities by plane

	Management Plane	**Data Plane**
Transport	HTTP / REST	WebSocket (Socket.IO)
Style	Request–response, three-tier	Event-driven replication
State	Server-authoritative, in MongoDB	Replicated CRDT, converged
Server role	Decides and records	Relays and gates
Failure mode	Request fails, user retries	Client keeps working, reconciles on reconnect
Examples	Create, join, approve, remove, execute	Shape edits, typing, cursors, chat

4.2 High-Level Architecture

Figure 4.1 shows the system's four tiers and the two distinct paths between the browser and the server.

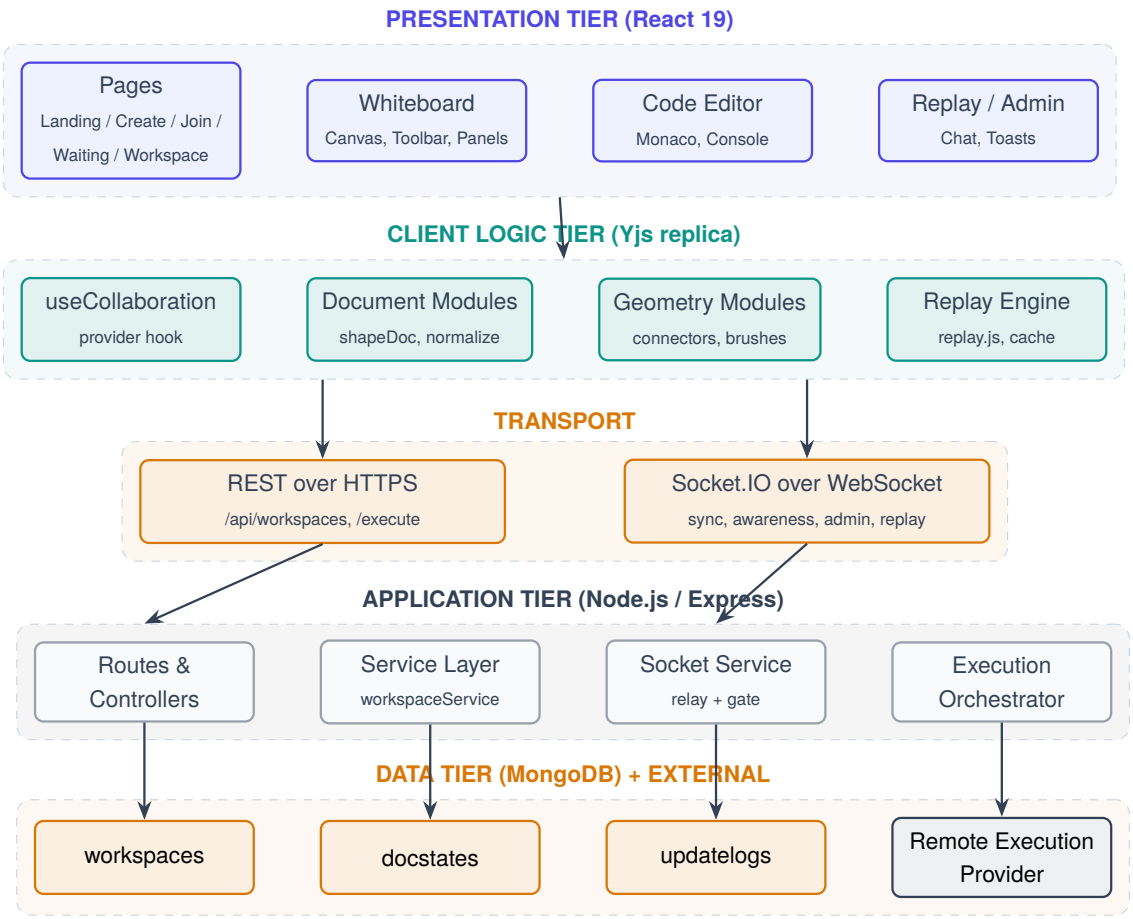


Figure 4.1: High-level four-tier architecture showing the separate REST and WebSocket paths between client and server

4.2.1 Tier Responsibilities

Table 4.2: Module responsibilities by tier

Tier	Module	Responsibility
Presentation	Pages	Routing, forms, session bootstrap, fatal-state screens
	Canvas subsystem	Tool state, pointer handling, camera, selection, rendering
	Editor subsystem	Monaco lifecycle, language selection, run trigger, console
	Panels	Property editing, brush settings, administration, replay, chat
Client logic	`useCollaboration`	Owns the `Y.Doc` and awareness; bridges them to the socket
	`shapeDoc.js`	The single write path into the shared document
	`normalize.js`	Sanitises every record read out of the document before rendering
	`connectors.js`, `brushes.js`, `replay.js`	Pure geometry, stroke mathematics and history reconstruction
Application	`server.js`	Composition root: middleware, routes, error handling, socket attachment
	`socketService.js`	Handshake authentication, room placement, relay, replay streaming
	`workspaceService.js`	All workspace business rules, called by both REST and sockets
	`workspaceStore.js`, `updateLogService.js`	Dual-mode repositories
	`execution/`	Provider-agnostic orchestration of code execution
Data	`workspaces`	Membership, pending requests, policy, hashed secret
	`docstates`	One debounced binary snapshot per workspace
	`updatelogs`	Append-only ordered update history per workspace

4.3 Low-Level Architecture

Figure 4.2 expands the server into its actual modules and shows the direction of every internal dependency. Two features of the graph are deliberate and worth noting.

First, `realtime.js` exists solely to break a circular import. The workspace service must push socket events ("a join request arrived"), and the socket service must call workspace business rules ("approve this request"). If each imported the other directly the module graph would be cyclic. Instead `realtime.js` holds a reference to the Socket.IO instance, bound once at boot, and exposes emit helpers; the service imports the helper module, not the socket service.

Second, the execution subtree imports *nothing* from the collaboration subtree. It holds no per-workspace state and knows nothing about documents, rooms or members. Isolation between users is therefore structural rather than enforced: an execution request cannot touch document synchronisation because it has no reference by which to do so.

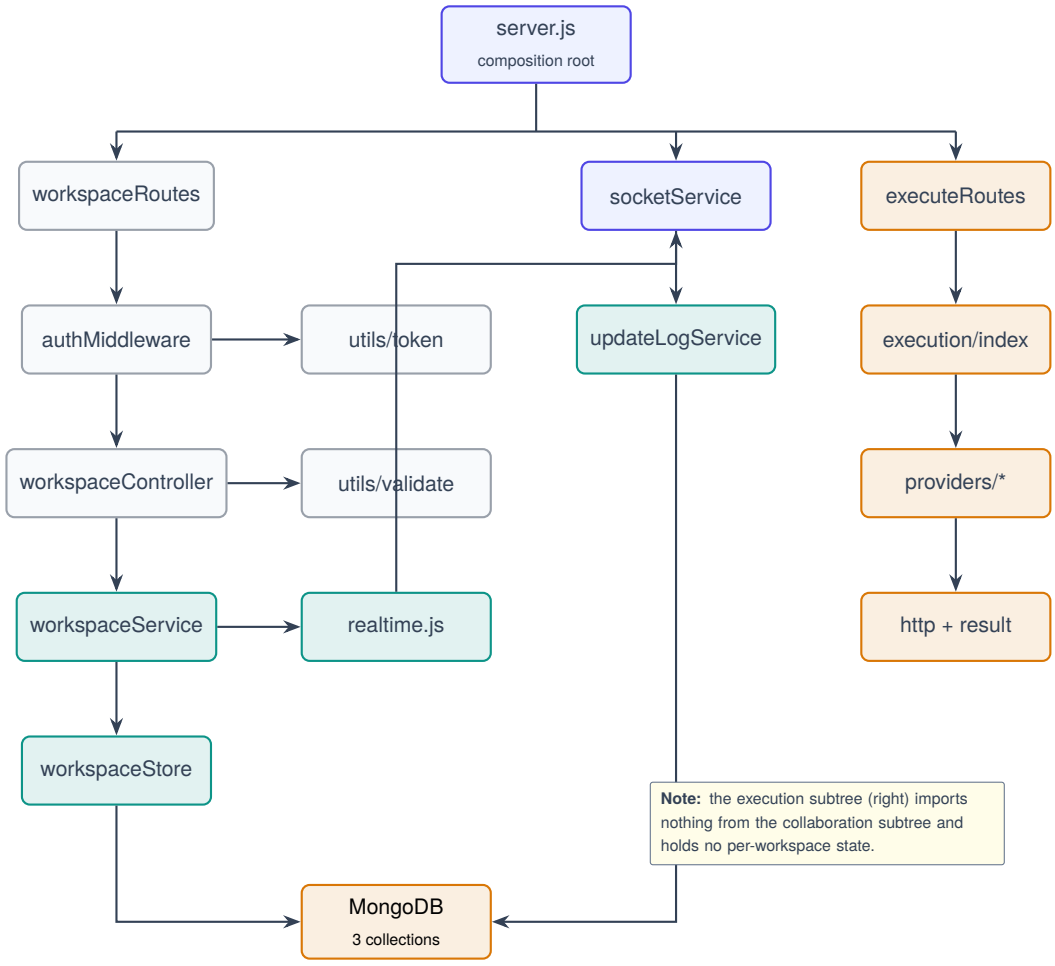


Figure 4.2: Low-level module dependency graph of the server

4.4 Component Diagram

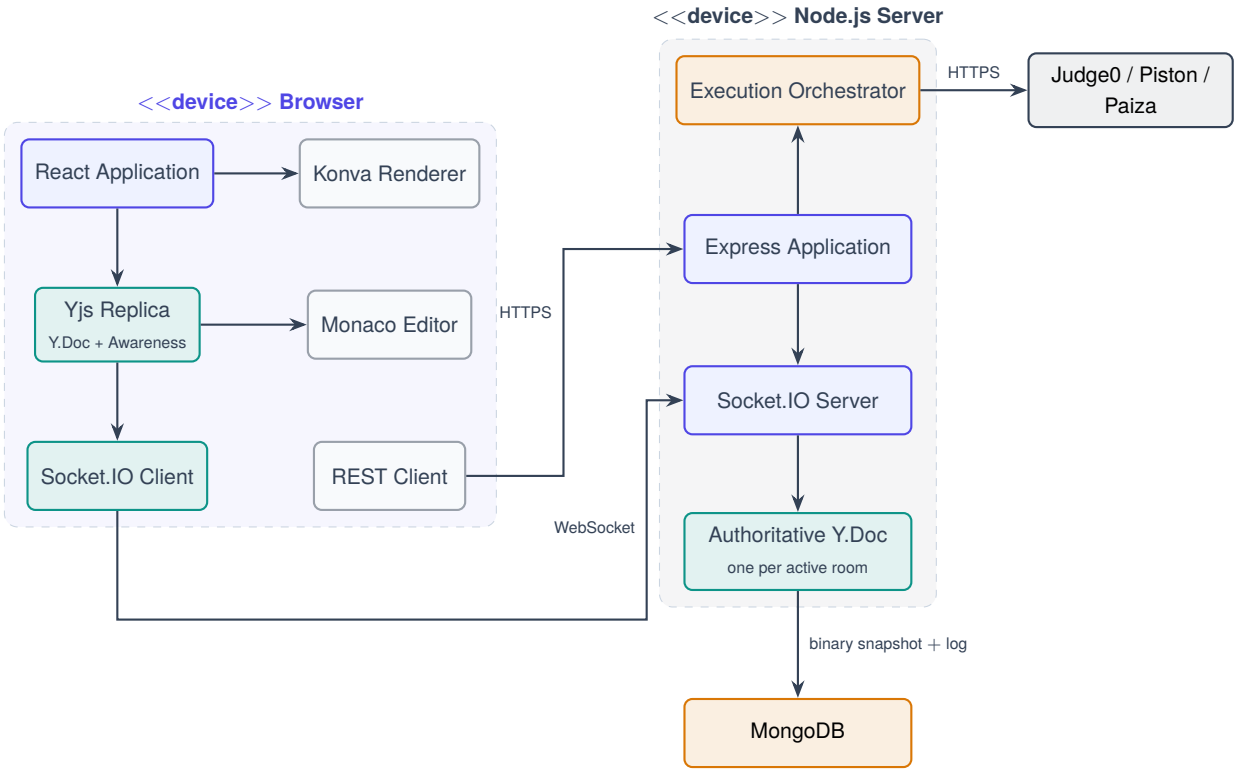
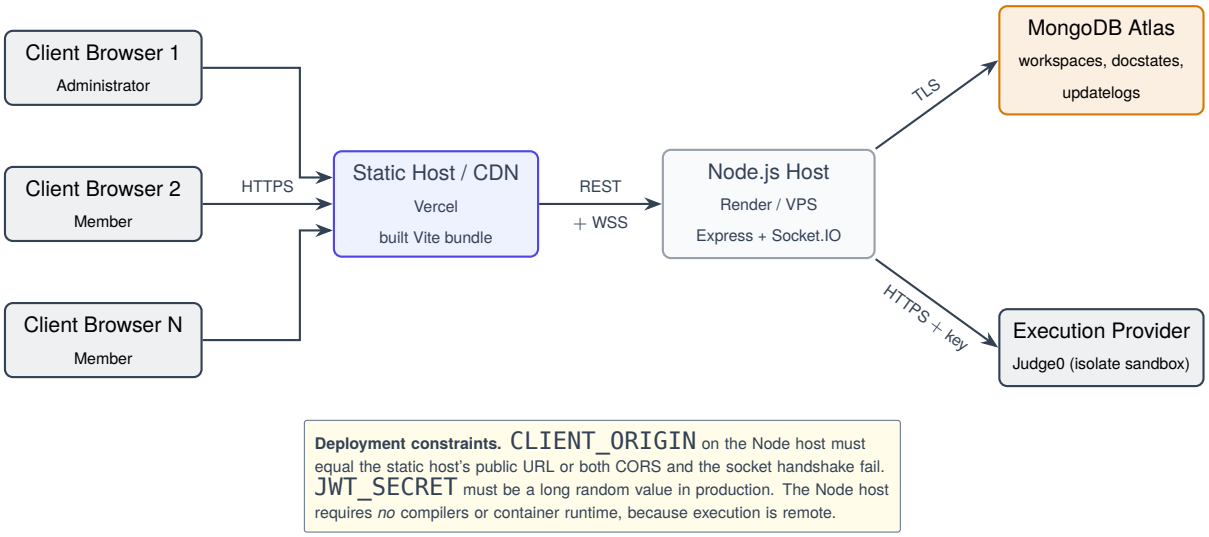


Figure 4.3: Component diagram showing the browser, server and external execution provider

4.5 Deployment Architecture



Deployment constraints. `CLIENT_ORIGIN` on the Node host must equal the static host's public URL or both CORS and the socket handshake fail. `JWT_SECRET` must be a long random value in production. The Node host requires *no* compilers or container runtime, because execution is remote.

Figure 4.4: Deployment architecture

4.6 Data Flow Diagrams

4.6.1 Level 0 — Context Diagram

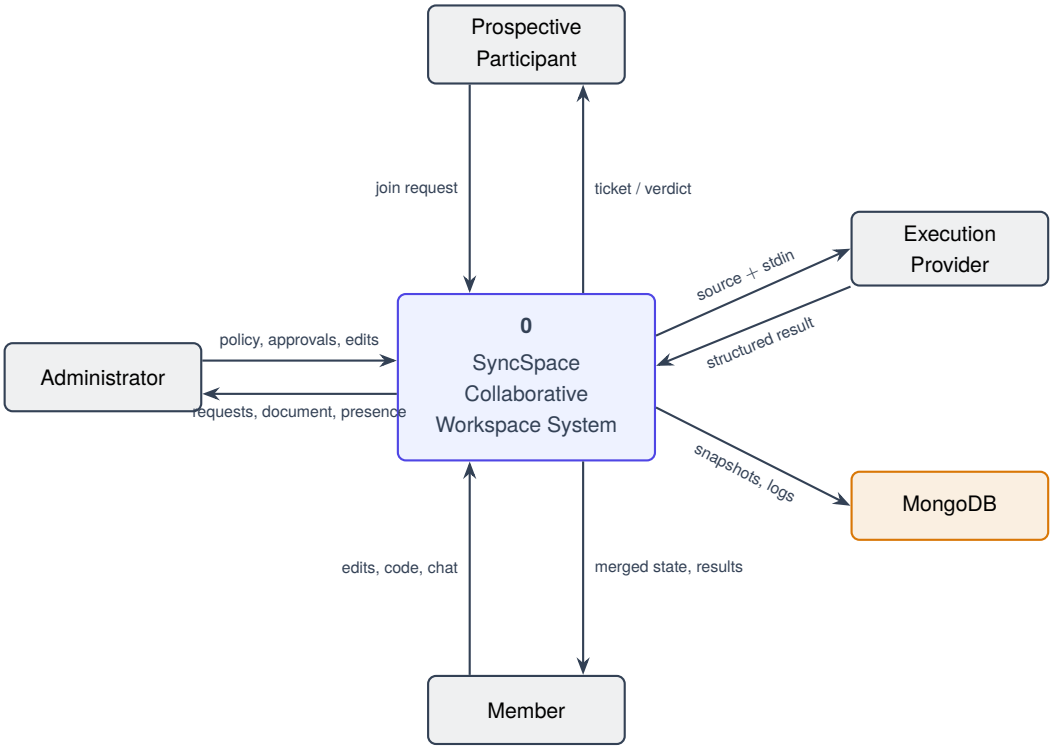


Figure 4.5: DFD Level 0 — context diagram

4.6.2 Level 1 — Major Processes

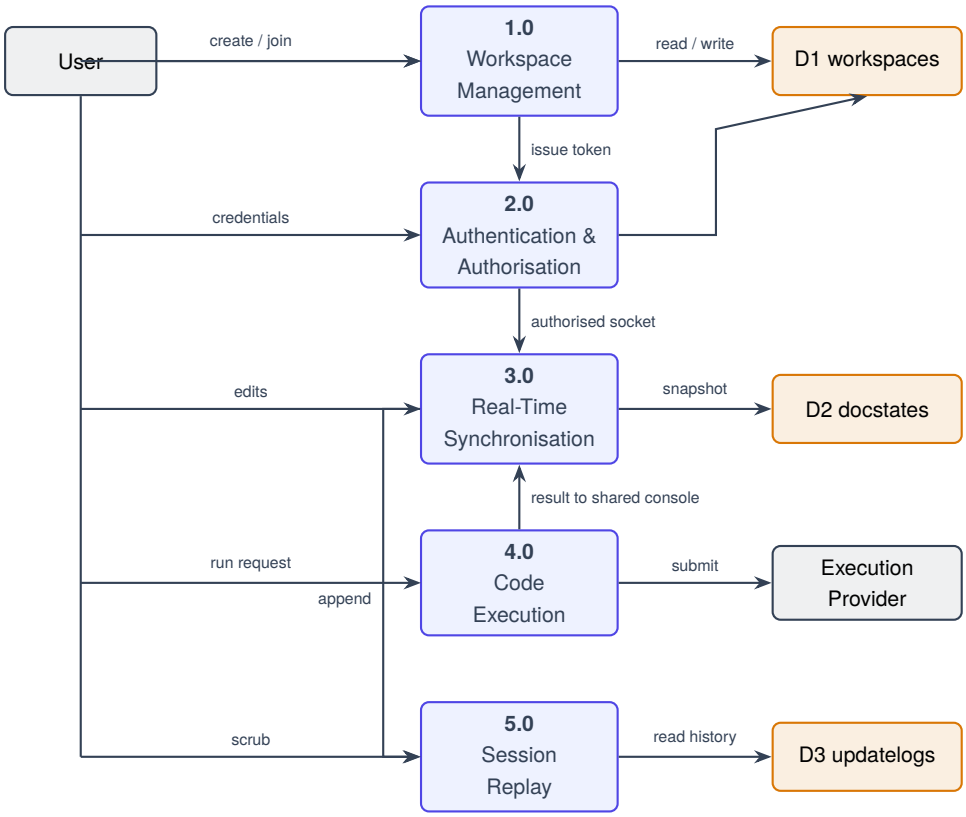


Figure 4.6: DFD Level 1 — decomposition into five major processes

4.6.3 Level 2 — Real-Time Synchronisation (Process 3.0)

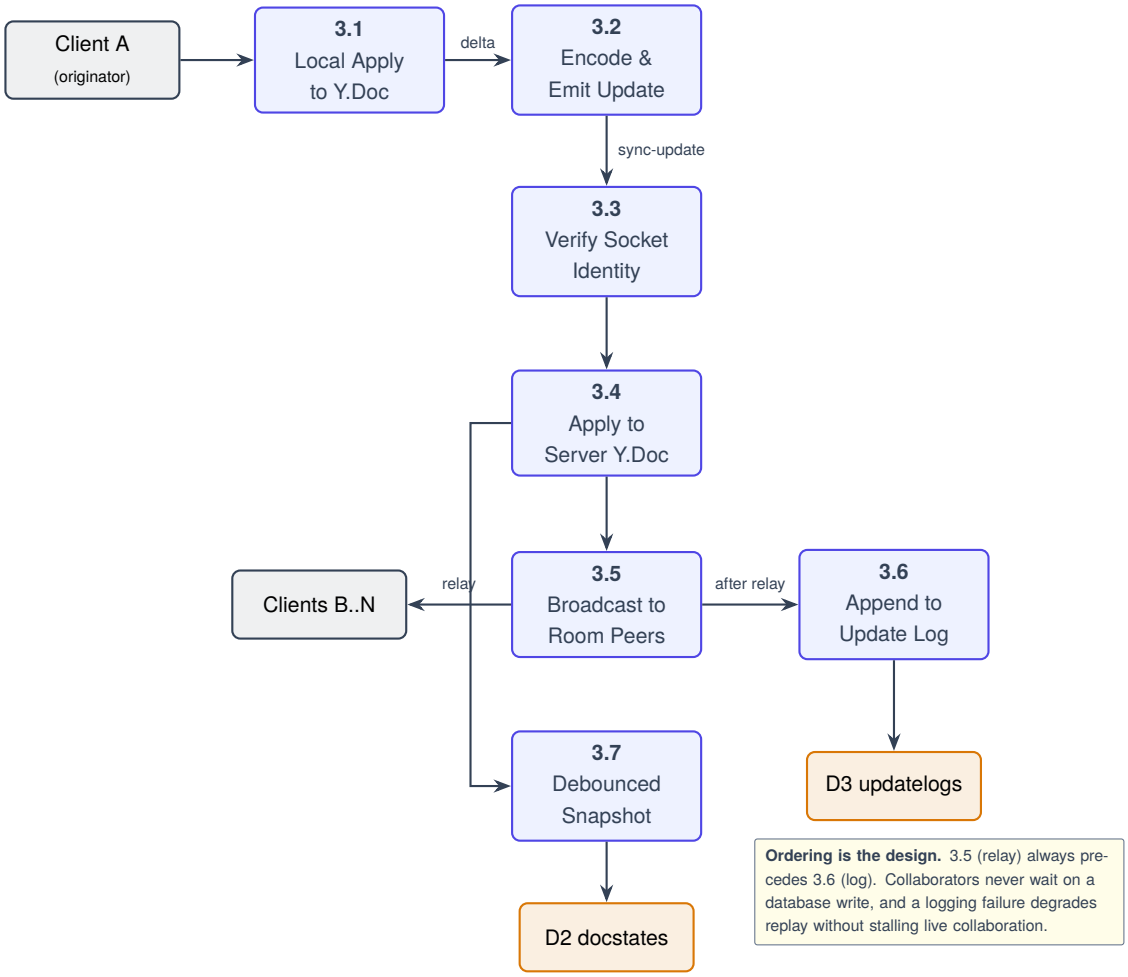


Figure 4.7: DFD Level 2 — decomposition of real-time synchronisation

4.7 API Request Lifecycle

Every authenticated REST request traverses the same eight stages. The order is significant: rate limiting precedes validation so that a flood of malformed requests is cheap to reject, and membership re-verification occurs on every request so that a removed member's token stops working immediately rather than at token expiry.

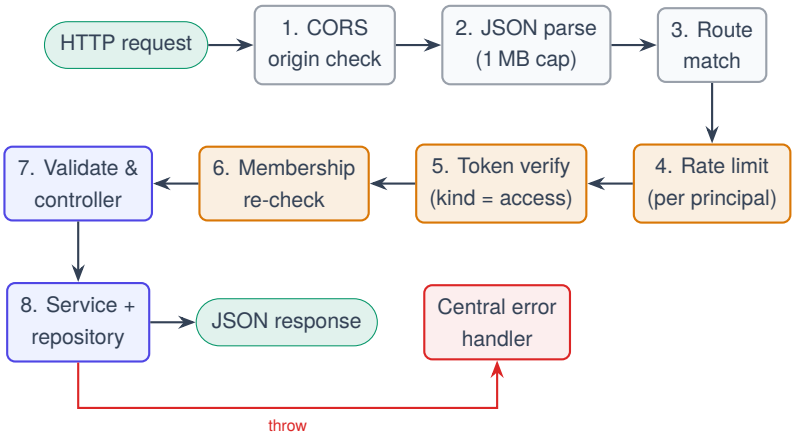


Figure 4.8: Lifecycle of an authenticated REST request

4.8 Design Patterns Employed

Table 4.3: Design patterns applied and their locations

Pattern	Location	Benefit realised
Repository	`workspaceStore`, `updateLogService`	Dual-mode persistence; every feature works with or without a database
Service layer	`workspaceService`	One implementation of each rule, called by both REST and sockets — no drift
Adapter	`execution/providers/*`	New execution backend is one file; switching is one environment variable
Strategy	Brush registry, routing modes, head styles	New brush or routing mode is a data entry, not a new code path
Observer	Yjs `observeDeep`, awareness `on('change')`	Rendering reacts to state without polling
Facade	`realtime.js`	Breaks the service/socket import cycle with a minimal emit surface
Memento	Replay checkpoints, `Y.UndoManager`	Encoded snapshots allow restoration without full replay
Composite	`shapes` array holding every object type	Connectors, strokes and shapes share one selection, undo and sync path
Guard clause	`requireMember`, `requireAdmin`, `asAdmin`	Authorisation expressed once, applied uniformly
Null object	Memory-mode stores	Absent database is a configuration, not an error path

CHAPTER 5

System Design

5.1 Use Case Model

The system recognises three actors. A **Prospective Participant** has an identity but no membership; a **Member** holds an access token and may operate on the document; an **Administrator** is a member with additional authority over admission and membership. The **Execution Provider** is a supporting external actor.

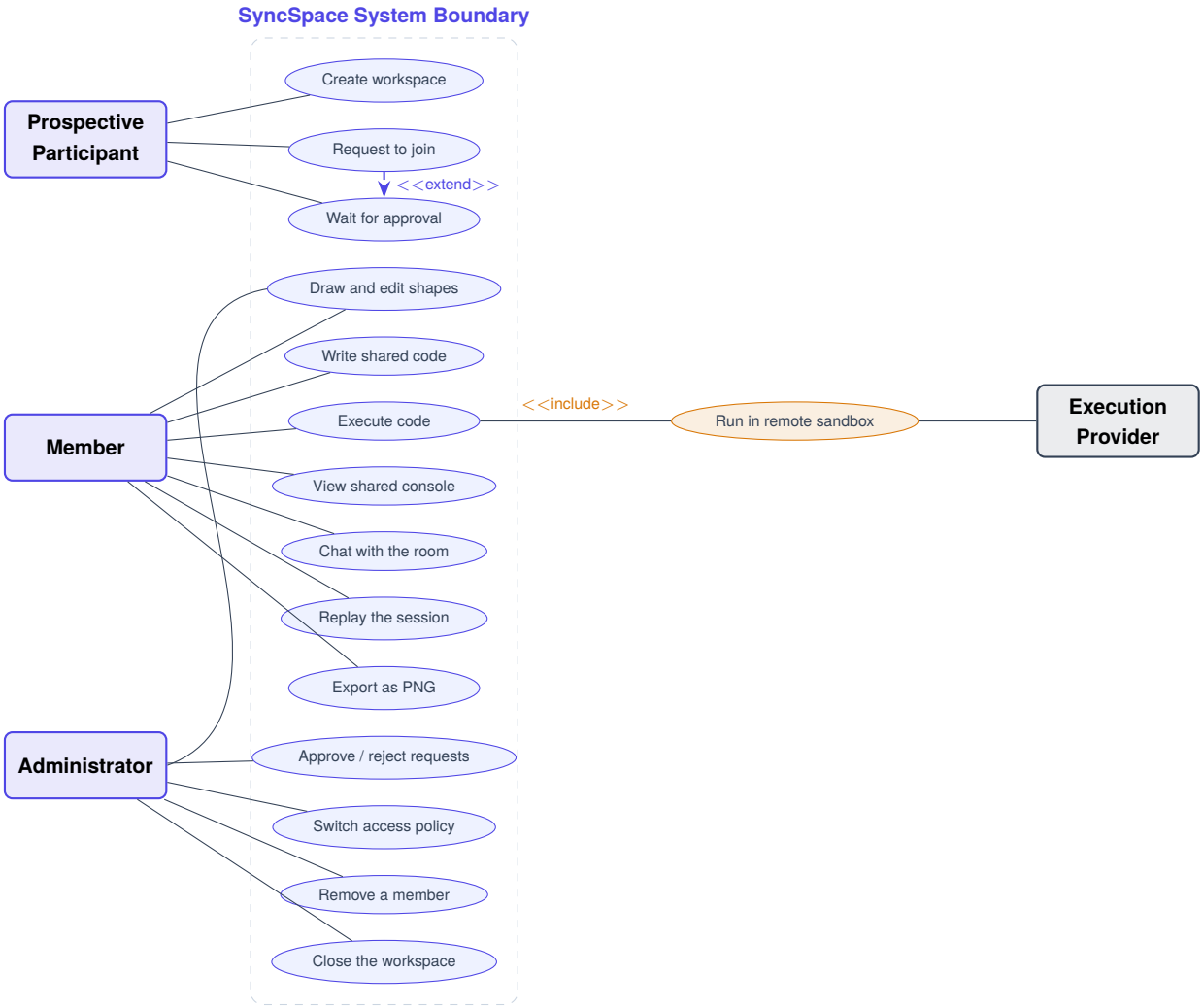


Figure 5.1: Use case diagram

5.1.1 Selected Use Case Specifications

Table 5.1: Use case specification — UC-02: Request to join a workspace

Field	Description
Identifier	UC-02
Actor	Prospective Participant
Precondition	The workspace exists, is active, and the actor knows its identifier and secret code
Trigger	The actor submits the join form
Main flow	(1) Actor supplies workspace ID, display name and secret code. (2) System applies the per-IP rate limit. (3) System validates the field formats. (4) System compares the supplied code against the stored bcrypt hash. (5) System checks the display name is not already taken. (6) If policy is *password*, the actor is added as a member and issued an access token. (7) If policy is *permission*, a pending request is recorded and a lobby ticket is issued.
Alternate A4	Wrong code or unknown workspace: an identical message is returned in both cases so the endpoint cannot be used to enumerate workspace identifiers
Alternate A5	Name already taken: the actor is asked to choose another
Alternate A2	More than ten attempts per minute from one address for one workspace: HTTP 429
Postcondition	The actor holds either an access token (member) or a lobby ticket (waiting)

Table 5.2: Use case specification — UC-09: Replay the session

Field	Description
Identifier	UC-09
Actor	Member
Precondition	The actor holds a valid access token and the socket is connected
Main flow	(1) Actor opens the replay panel. (2) Client emits `get-replay-logs`. (3) Server streams metadata, then bounded chunks, then an end marker. (4) Client reconstructs a second, local document and warms its cache in batches. (5) Actor scrubs, plays, steps or jumps to any index. (6) The panel renders that historical state with the live renderer.
Alternate	Storage read failure, corrupted chunk, incomplete history or dropped connection each produce a distinct diagnostic naming what was actually received
Postcondition	The live document is unmodified — guaranteed structurally, since the replay document is never bound to a socket

5.2 Class Design

Although the implementation is predominantly functional in style, the logical class model clarifies the relationships. Figure 5.2 shows the principal server-side and shared abstractions.

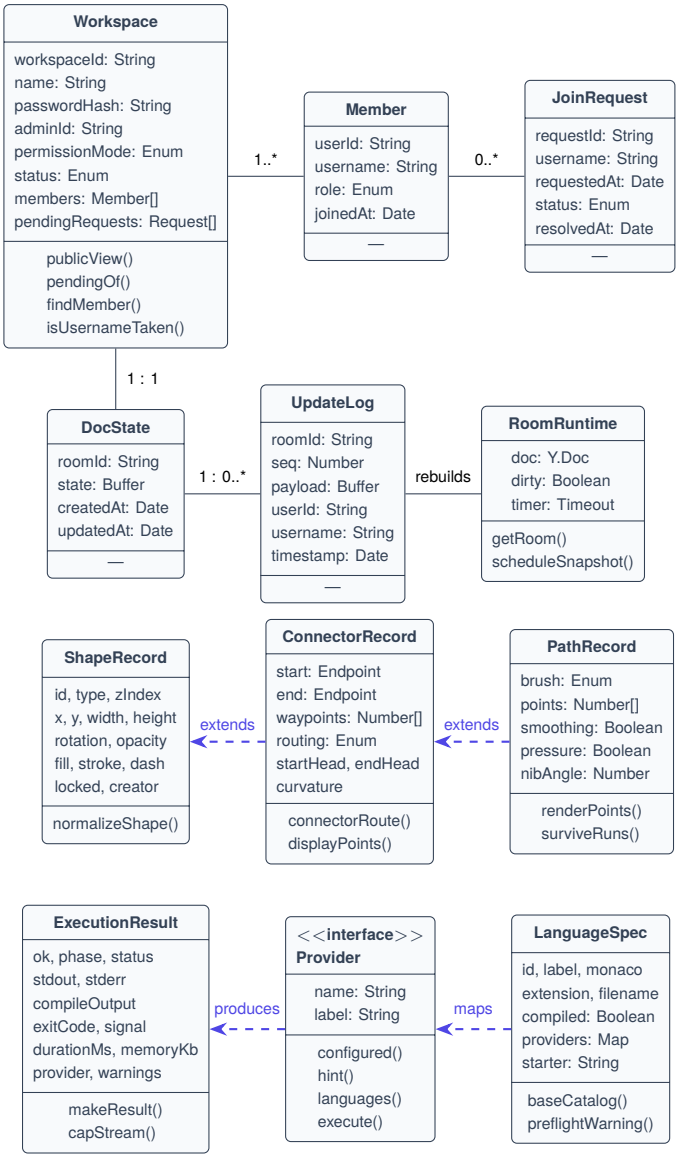


Figure 5.2: Class diagram of the principal domain and runtime abstractions

ShapeRecord is one type, not a hierarchy

The three record classes shown as specialisations are conceptual. In the implementation there is exactly *one* record shape, distinguished by a `type` field, and all records live in the same `Y.Array`. This is deliberate: it is the reason a connector, a calligraphy stroke and a rectangle all inherit selection, multi-select property editing, locking, layer ordering, duplication, undo, persistence, replication and replay without a single line of type-specific code in any of those subsystems. Adding a new object type means adding a case to one render switch — not a parallel system.

5.3 Database Design

5.3.1 Entity–Relationship Model

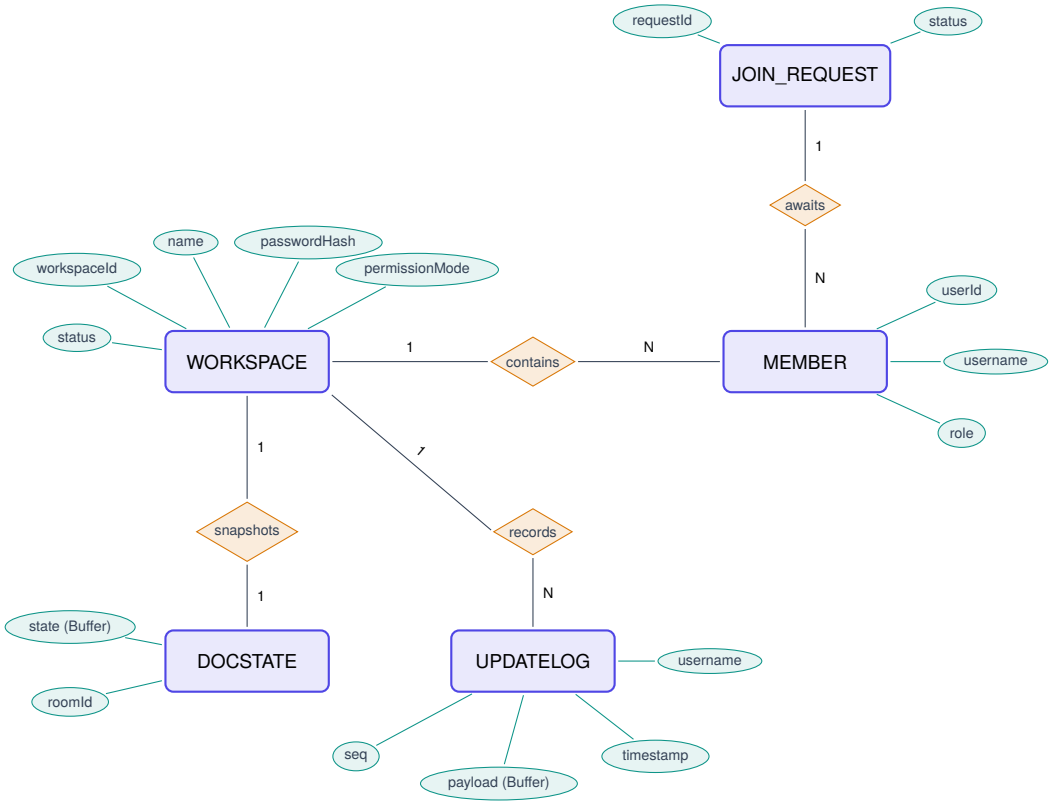


Figure 5.3: Entity–relationship diagram

5.3.2 Database Schema

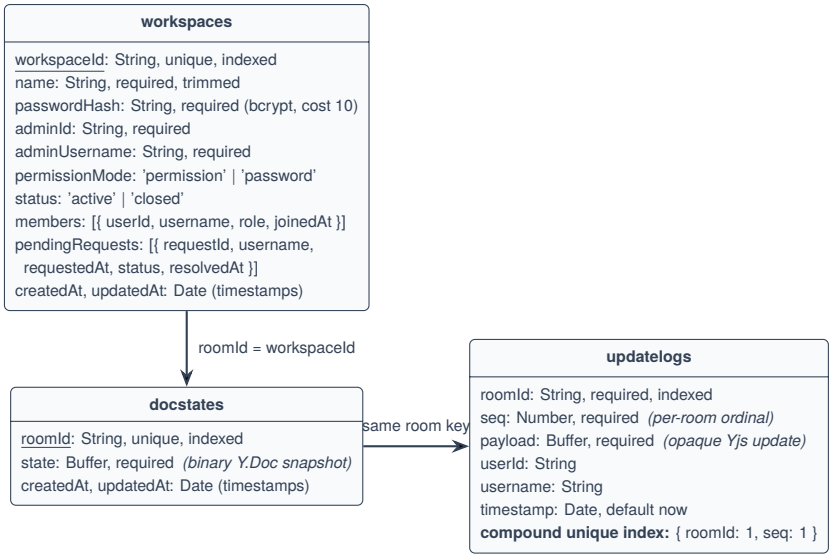


Figure 5.4: Database schema for the three MongoDB collections

Table 5.3: Collection structure and access patterns

Collection	Cardinality	Write pattern	Read pattern	Indexes
`workspaces`	1 per workspace	Read-modify-write through one mutation path	On every socket connect and REST call	`workspaceId` unique
`docstates`	1 per workspace	Upsert, debounced at 2 s	Once, on first join after a restart	`roomId` unique
`updatelogs`	1 per update	Insert-only, or in-place merge when coalescing	Full ordered scan when replay opens	{`roomId`, `seq`} unique compound

5.3.3 Two Deliberate Modelling Decisions

Why there is no global `users` collection

The project blueprint anticipated a `users` collection. It was deliberately not built. SyncSpace has no cross-workspace identity: a person is a *member of one workspace*, not a user of a platform. Membership is therefore embedded in the workspace document rather than referenced from a separate collection. This yields a single atomic read for the entire authorisation decision — one query returns the workspace, its policy, its members and its pending requests — where a normalised design would require a join or a second round trip on *every socket connection*. A global collection would carry no data that any query needs and would introduce a consistency problem (an orphaned account whose workspace was closed) with no corresponding benefit. The cost of this decision is stated honestly in Section 12.2: an "all my workspaces" listing is impossible without adding that identity layer.

Why `docstates` and `updatelogs` both exist

These two collections answer different questions and neither substitutes for the other. `docstates` answers *"what does this board look like now?"* It holds one row per workspace, overwritten every two seconds. It is a photograph.
`updatelogs` answers *"how did it get here?"* It holds one row per update, appended and never rewritten (except for in-window coalescing). It is the film reel.
A photograph cannot be scrubbed. Reconstructing the state at update N requires the individual updates in order; a periodically overwritten snapshot has by definition discarded them. Conversely, restoring a room after a restart by replaying thousands of log rows would be needlessly slow when a single encoded snapshot achieves it in one

operation. The two shapes are complementary, which is precisely the classical event-sourcing pairing of an event stream with periodic snapshots described in Chapter 2.

5.3.4 Shared Document Structure

The MongoDB schema describes only durable metadata. The collaborative content itself lives inside a single Yjs document whose top-level types are shown in Table 5.4. Every one of these is captured by the same snapshot and reproduced by the same replay.

Table 5.4: Top-level shared types within the workspace `Y.Doc`

Name	Yjs type	Contents
`shapes`	`Y.Array` of `Y.Map`	Every whiteboard object: strokes, shapes, connectors, text, images
`monaco`	`Y.Text`	The shared code buffer, bound to the Monaco text model
`editorMeta`	`Y.Map`	The active language, shared so all dropdowns and syntax modes switch together
`runHistory`	`Y.Array`	Execution results (capped at 20), each attributed to the member who ran it
`chatMessages`	`Y.Array`	Chat history (capped at 100)

5.4 Sequence Diagrams

5.4.1 Workspace Creation and First Connection

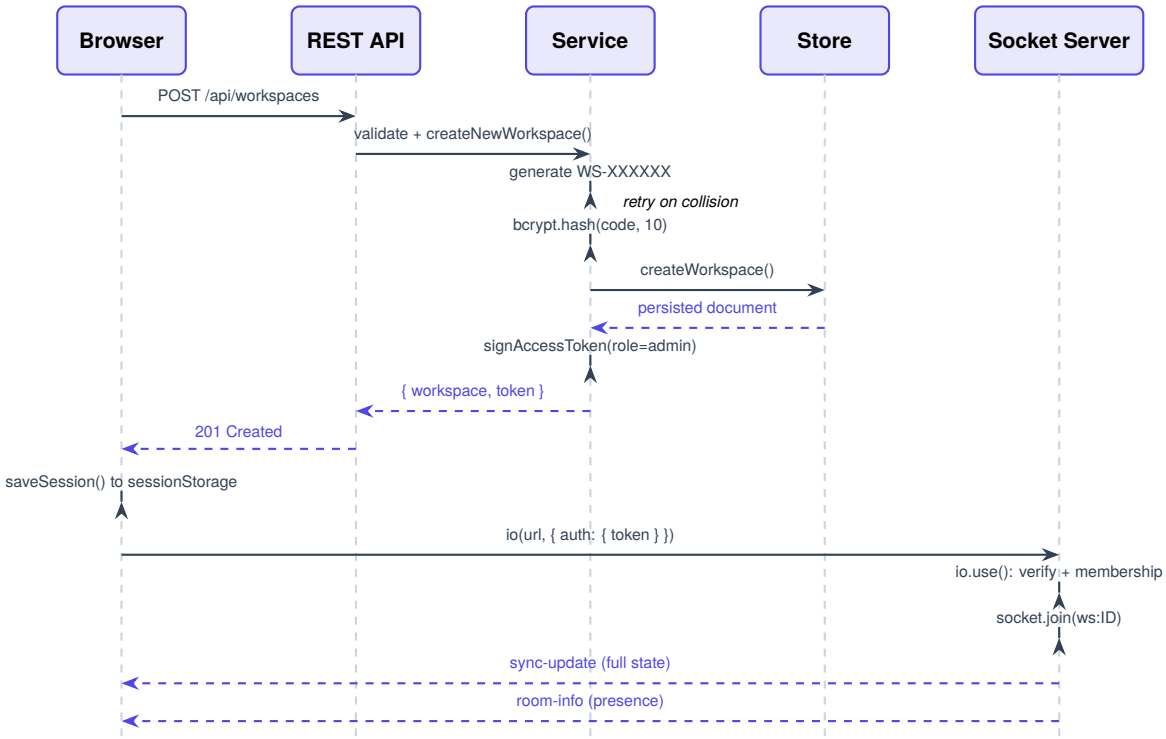


Figure 5.5: Sequence diagram — workspace creation followed by the authenticated socket handshake

5.4.2 Join with Administrator Approval

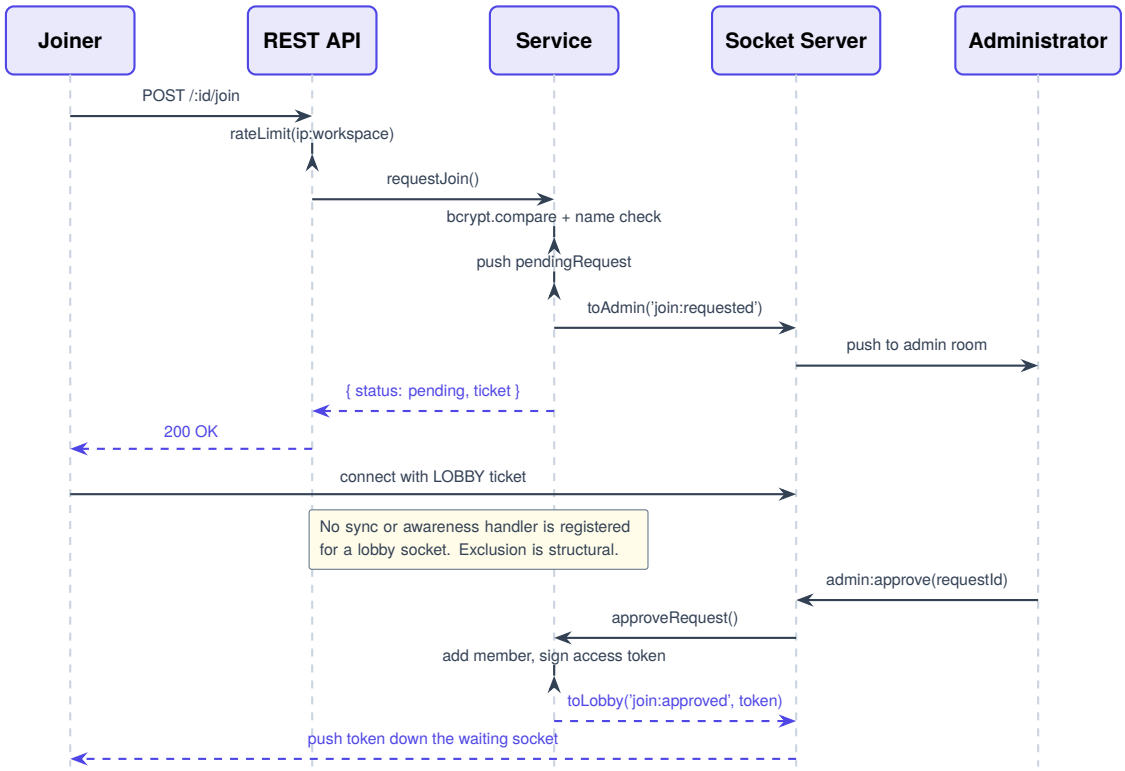


Figure 5.6: Sequence diagram — joining a permission-mode workspace

5.4.3 Concurrent Shape Edit

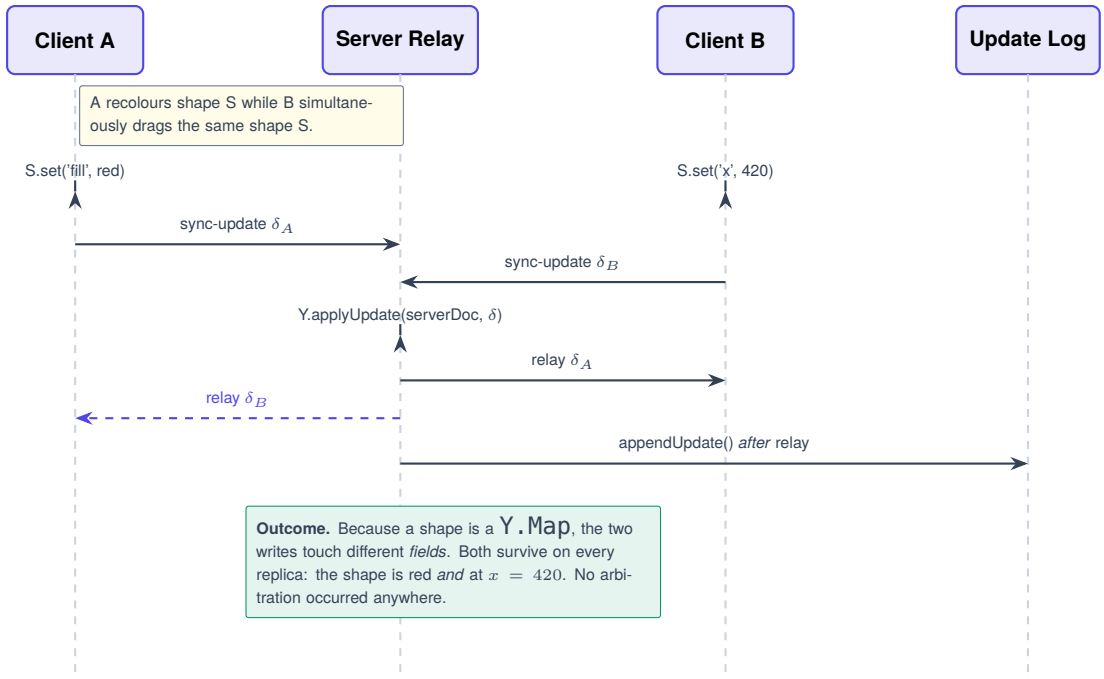


Figure 5.7: Sequence diagram — concurrent field-level edits merging without loss

5.5 Activity Diagrams

5.5.1 Joining a Workspace

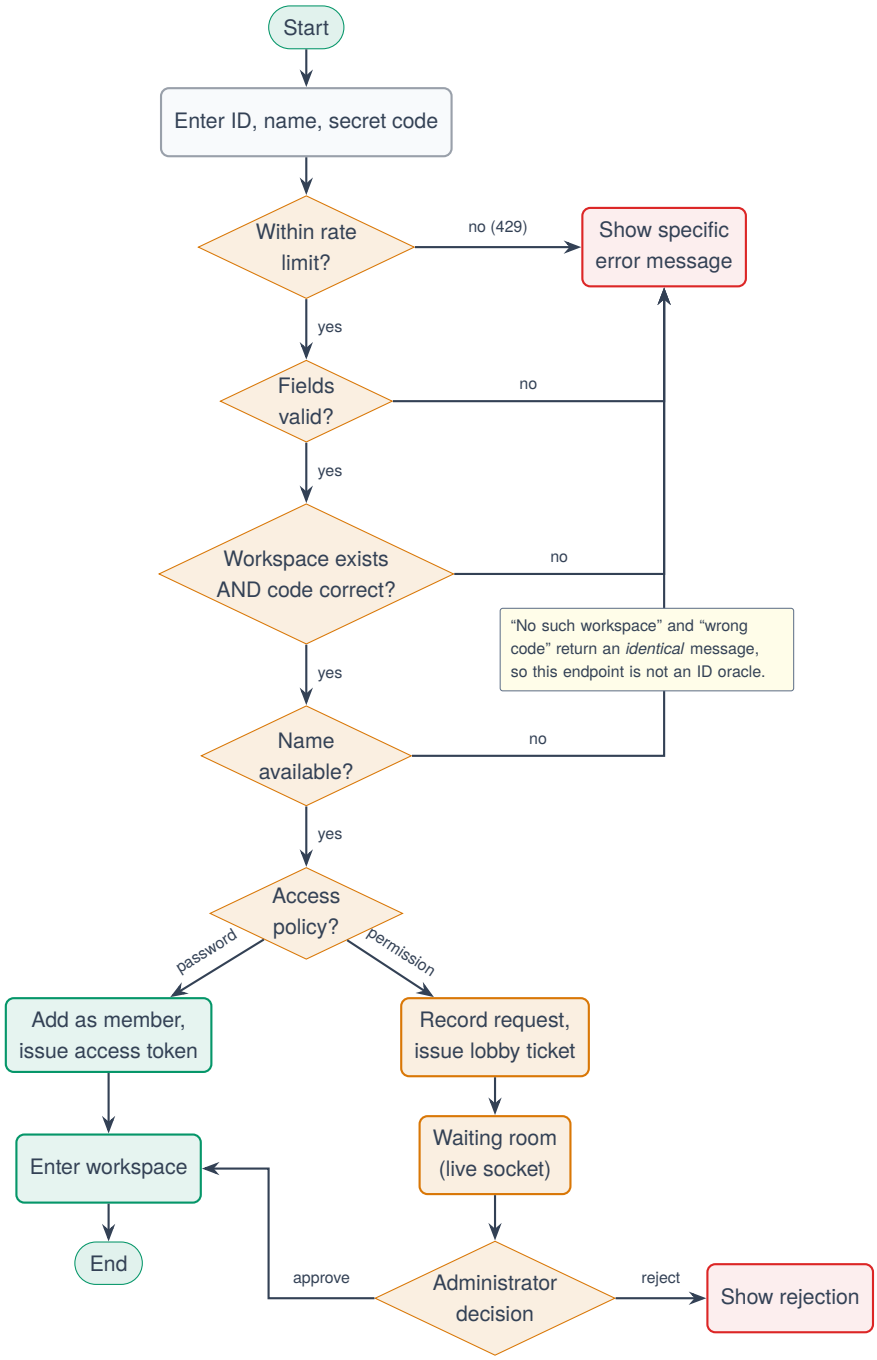


Figure 5.8: Activity diagram — joining a workspace

5.5.2 Drawing and Committing a Stroke

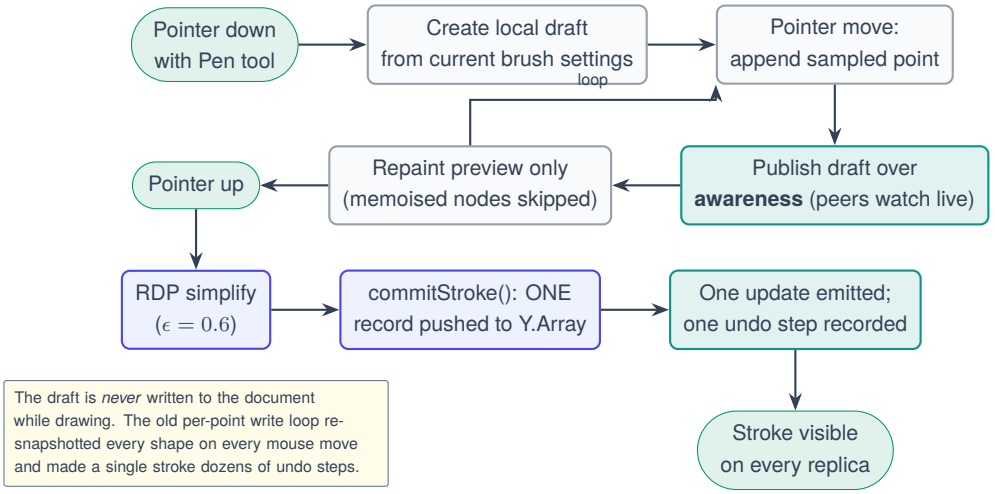


Figure 5.9: Activity diagram — freehand stroke lifecycle

5.6 State Transition Diagrams

5.6.1 Participant State

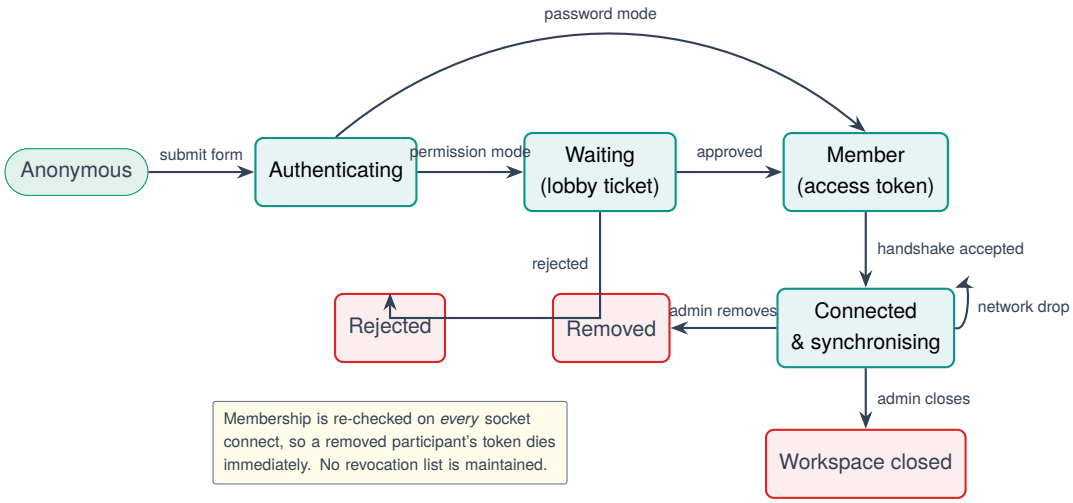


Figure 5.10: State transition diagram — participant lifecycle

5.6.2 Shape Object State

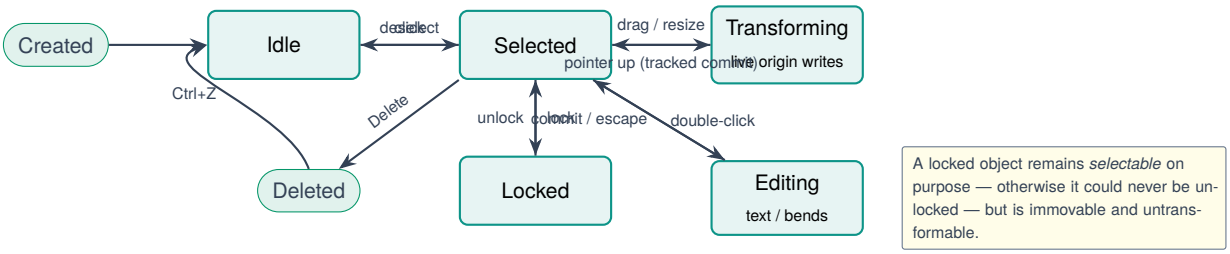


Figure 5.11: State transition diagram — whiteboard object lifecycle

CHAPTER 6

Implementation I — Server Side

This chapter documents the server module by module. For each significant module it states the purpose, the internal working, the inputs and outputs, the data structures used, the communication with neighbouring modules, and the design decisions taken — together with the reasoning that produced them.

6.1 Composition Root: `server.js`

6.1.1 Purpose and Working

`server.js` is the composition root. It performs no business logic; it assembles the application, attaches the socket server, resolves the persistence mode once at boot, and starts listening. Every stateful subsystem is told whether a database is available through a single explicit call, which is what makes the dual-mode behaviour described in Section 6.5.1 uniform rather than ad hoc.

```
const app = express();
app.set("trust proxy", 1);
app.use(cors({ origin: CLIENT_ORIGIN }));
// 1mb: the execute endpoint receives whole source files.
app.use(express.json({ limit: "1mb" }));

app.get("/api/health", (req, res) => res.json({
  status: "ok", service: "syncspace-backend",
  time: new Date(), execution: executionStats()
}));

app.use("/api/workspaces", workspaceRoutes);
app.use("/api/workspaces/:workspaceId/execute", executeRoutes);

// Malformed or oversized JSON must not become a 500 -- the editor shows
// the message verbatim, and "our side" would be a lie here.
app.use("/api", (err, req, res, next) => {
  if (err?.type === "entity.parse.failed")
    return res.status(400).json({ error: "That request was not valid JSON." });
  if (err?.type === "entity.too.large")
    return res.status(413).json({ error: "That program is too large to send (limit 1 MB)." });
  return next(err);
});

const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: CLIENT_ORIGIN, methods: ["GET","POST","PATCH","DELETE"] },
  // engine.io's default receive ceiling is 1 MB, and a message over it does
```

```
  // not error politely -- it severs the socket at the transport layer. A
  // single sync-update carrying an uploaded image can exceed 1 MB.
  maxHttpBufferSize: 8e6
});

const connected = await connectDB();
setPersistence(connected);       // Yjs snapshots
setStorePersistence(connected);  // workspaces / members / requests
setLogPersistence(connected);    // updatelogs (replay history)
setupSocket(io);
```

Listing 6.1: Express and Socket.IO initialisation with explicit persistence wiring

6.1.2 Design Decisions

Why `maxHttpBufferSize` was raised to 8 MB

Engine.IO's default *receive* ceiling is one megabyte, and a message exceeding it does not produce a polite error — it severs the socket at the transport layer. A single `sync-update` carrying an uploaded image (whose base64 source lives inside the shape record) can legitimately exceed that figure. Left at the default, uploading a photograph would silently terminate live collaboration for that participant with no diagnostic. This is the same class of defect as the replay attachment ceiling discussed in Section 8.3 — a transport-layer limit presenting as an application-layer mystery — and it was found by recognising the pattern.

Two error handlers, not one

The API-scoped handler translates body-parser failures into specific messages; the global handler logs the real stack and returns one neutral sentence. The separation exists because "Something went wrong on our side" is factually false for a request whose JSON the client malformed, and the code editor displays these messages verbatim to the user.

6.2 Authentication and the Token Model

6.2.1 Two Classes of Token

The security model rests on a distinction that is easy to state and easy to under-appreciate: SyncSpace issues *two* kinds of signed token, and the difference between them *is* the access control system.

Table 6.1: The two token classes

Kind	Claims	Lifetime	Grants
`access`	`workspaceId`, `userId`, `username`, `role`	12 hours	The collaborative socket: document synchronisation, awareness, cursors, replay, chat, execution
`lobby`	`workspaceId`, `requestId`, `username`	2 hours	The waiting room only. No document handler is ever registered for it

```
export function signAccessToken({ workspaceId, userId, username, role }) {
  return jwt.sign({ kind: "access", workspaceId, userId, username, role },
                  SECRET, { expiresIn: "12h" });
}

export function signLobbyTicket({ workspaceId, requestId, username }) {
  return jwt.sign({ kind: "lobby", workspaceId, requestId, username },
                  SECRET, { expiresIn: "2h" });
}

export function verifyToken(token) {
  try { return jwt.verify(token, SECRET); } catch { return null; }
}
```

Listing 6.2: Token issuance and verification

6.2.2 JWT Workflow

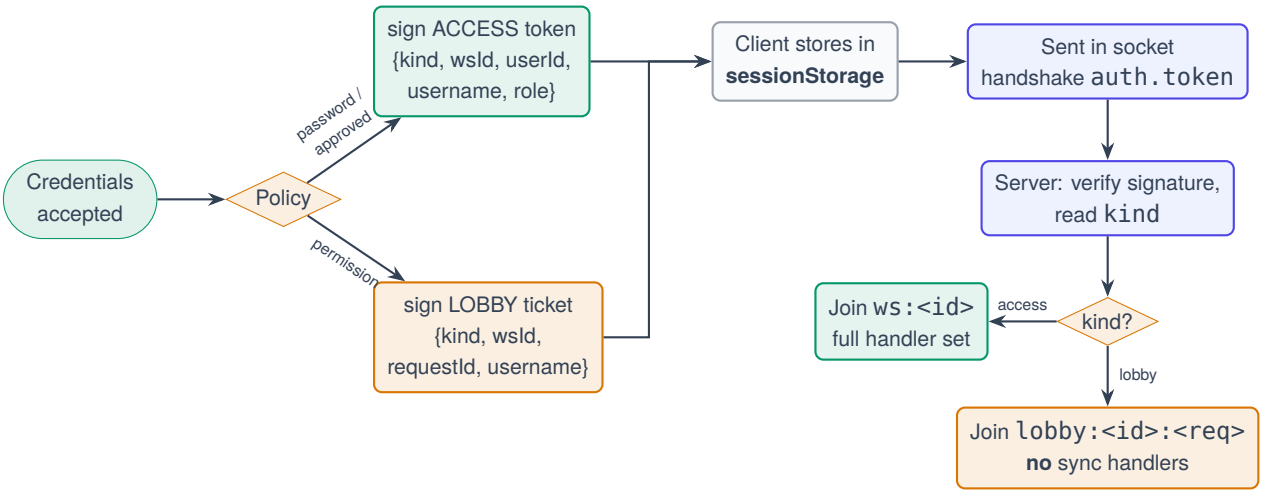


Figure 6.1: JWT issuance and consumption workflow

6.2.3 Authorisation Middleware

```
export async function requireMember(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token)
    return res.status(401).json({ error: 'You are not signed in to this workspace.' });

  const payload = verifyToken(token);
  if (!payload || payload.kind !== 'access')
    return res.status(401).json({ error: 'Your session has expired. Please join again.' });

  // The token names its own workspace. A token for workspace A can never
  // authorise a request against workspace B, whatever the URL says.
  if (payload.workspaceId !== req.params.workspaceId)
    return res.status(403).json({ error: 'That session does not belong to this workspace.' });

  const workspace = await findWorkspace(payload.workspaceId);
  if (!workspace)  return res.status(404).json({ error: 'This workspace no longer exists.' });
  if (workspace.status === 'closed')
    return res.status(410).json({ error: 'This workspace has been closed by its administrator.' });

  // Re-checked on EVERY request, so a removed member's token dies at once
  // and no revocation list has to be maintained.
  const member = findMember(workspace, payload.userId);
  if (!member) return res.status(403).json({ error: 'You have been removed from this workspace.' });

  req.workspace = workspace;
  req.user = { ...payload, role: member.role };
  next();
}
```

Listing 6.3: `requireMember` — membership is re-verified on every request

Why membership is re-checked rather than revoked

A signed token cannot be un-signed. The conventional answer is a revocation list, which introduces its own storage, expiry and consistency problems. SyncSpace instead makes the token a *claim of identity* rather than a *grant of access*: the grant is re-derived from the workspace document on every request and every socket connection. Removing a member is therefore a single array modification whose effect is instantaneous and total, with no auxiliary structure to maintain or to fall out of sync.

6.2.4 Authentication and Authorisation Flow

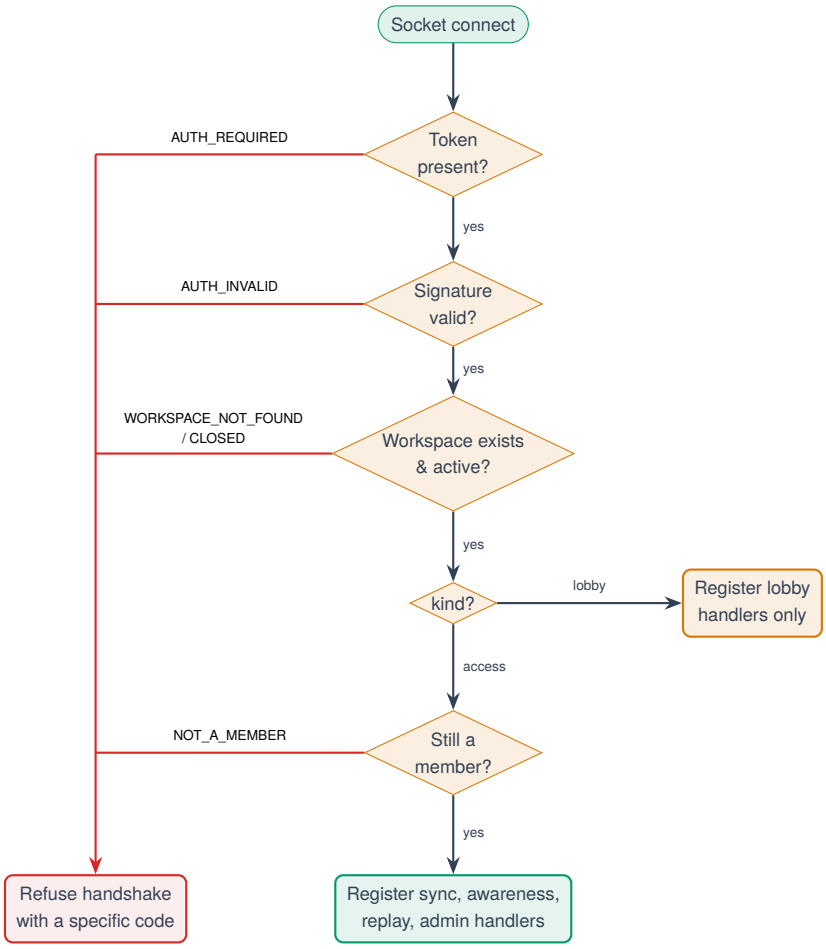


Figure 6.2: Authentication and authorisation flow at the socket handshake

6.3 Workspace Service and Lifecycle

6.3.1 Purpose

`workspaceService.js` contains every workspace business rule exactly once. It is called both by the REST controllers and by the socket administrator handlers. This matters: administrator approval is available over HTTP *and* over the socket, and if each transport implemented the rules separately the two would eventually disagree. Every function returns either `{ ok: true, ... }` or `{ ok: false, status, message }`, so a caller translates to an HTTP status or a socket acknowledgement without duplicating any decision.

```
export async function createNewWorkspace({ name, password, username, permissionMode }) {
  // Retry on the (astronomically unlikely) ID collision rather than crash.
  let workspaceId;
  for (let i = 0; i < 5; i++) {
    workspaceId = generateWorkspaceId();
    if (!(await findWorkspace(workspaceId))) break;
    workspaceId = null;
```

```
  }
  if (!workspaceId)
    return fail(500, 'Could not allocate a workspace ID. Please try again.');

  const adminId = newId();
  const passwordHash = await bcrypt.hash(password, 10);

  const workspace = await createWorkspace({
    workspaceId, name, passwordHash, adminId, adminUsername: username,
    permissionMode, status: 'active',
    members: [{ userId: adminId, username, role: 'admin', joinedAt: new Date() }],
    pendingRequests: []
  });

  const token = signAccessToken({ workspaceId, userId: adminId, username, role: 'admin' });
  return { ok: true, workspace: publicView(workspace), token };
}
```

Listing 6.4: Workspace creation with collision-tolerant identifier allocation

Identifiers are generated from a deliberately unambiguous alphabet (`23456789ABCDEFGHJKMNPQRSTUVWXYZ`) that omits `0`/`O` and `1`/`I`/`L`, because a workspace identifier is routinely read aloud over a call.

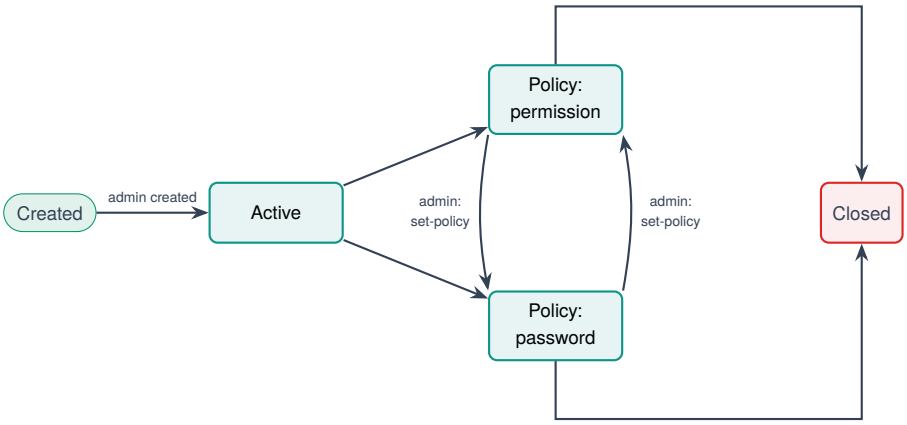
6.3.2 Join Logic

```
export async function requestJoin({ workspaceId, username, password }) {
  const workspace = await findWorkspace(workspaceId);

  // Deliberately identical message for "no such workspace" and "wrong
  // password": otherwise this endpoint becomes a workspace-ID oracle.
  if (!workspace) return fail(404, 'Workspace not found, or the secret code is incorrect.');
  if (workspace.status === 'closed')
    return fail(410, 'This workspace has been closed by its administrator.');

  const passwordOk = await bcrypt.compare(password, workspace.passwordHash);
  if (!passwordOk) return fail(401, 'Workspace not found, or the secret code is incorrect.');

  if (isUsernameTaken(workspace, username))
    return fail(409, `The name "${username}" is already taken in this workspace.`);

  // ---- Mode 2: password is enough. Straight in.
  if (workspace.permissionMode === 'password') {
    const userId = newId();
    const { workspace: saved } = await updateWorkspace(workspaceId, (ws) => {
      ws.members.push({ userId, username, role: 'member', joinedAt: new Date() });
    });
    const token = signAccessToken({ workspaceId, userId, username, role: 'member' });
    rt.toAdmin(workspaceId, 'workspace:updated', { workspace: publicView(saved) });
    return { ok: true, status: 'approved', token, workspace: publicView(saved) };
  }

  // ---- Mode 1: permission based. Into the waiting room.
  const requestId = newId();
  const request = { requestId, username, requestedAt: new Date(), status: 'pending' };
  const { workspace: saved } = await updateWorkspace(workspaceId, (ws) => {
    ws.pendingRequests.push(request);
```

```
  });

  const ticket = signLobbyTicket({ workspaceId, requestId, username });
  rt.toAdmin(workspaceId, 'join:requested', { request });  // wake the admin
  return { ok: true, status: 'pending', requestId, ticket };
}
```

Listing 6.5: Join handling for both access policies

6.3.3 Workspace Lifecycle



Runtime policy switching. Changing the policy affects only *future* joiners. Participants already collaborating stay connected; participants already in the waiting room stay waiting and can still be approved by hand. Closure emits `workspace:closed` to every member and rejects every outstanding lobby request with an explanatory reason rather than leaving them on an indefinite spinner.

Figure 6.3: Workspace lifecycle and policy transitions

6.4 Socket Service

6.4.1 Purpose and Structure

`socketService.js` is the data plane. It authenticates the handshake, places the socket into exactly one room, relays document and awareness updates, serves session history, and exposes the administrator actions. Its most important structural property is that the handler registration is *branched*: a lobby socket returns from the connection handler before any document handler is registered.

```
async function authenticate(socket, next) {
  const token = socket.handshake.auth?.token;
  if (!token) return next(new Error('AUTH_REQUIRED'));

  const payload = verifyToken(token);
  if (!payload) return next(new Error('AUTH_INVALID'));

  const workspace = await findWorkspace(payload.workspaceId);
```

```
  if (!workspace) return next(new Error('WORKSPACE_NOT_FOUND'));
  if (workspace.status === 'closed') return next(new Error('WORKSPACE_CLOSED'));

  if (payload.kind === 'access') {
    // Membership is re-checked on EVERY connect, so a removed user's token
    // dies immediately and we never maintain a revocation list.
    const member = findMember(workspace, payload.userId);
    if (!member) return next(new Error('NOT_A_MEMBER'));
    socket.data = { kind: 'access', workspaceId: workspace.workspaceId,
                    userId: payload.userId, username: member.username, role: member.role };
    return next();
  }

  if (payload.kind === 'lobby') {
    socket.data = { kind: 'lobby', workspaceId: workspace.workspaceId,
                    requestId: payload.requestId, username: payload.username };
    return next();
  }
  return next(new Error('AUTH_INVALID'));
}

io.use((socket, next) => {
  authenticate(socket, next).catch(() => next(new Error('AUTH_INVALID')));
});
```

Listing 6.6: Handshake authentication — the workspace is read from the token, never from the client

Isolation is structural, not enforced

There is no client-supplied room parameter anywhere in the protocol. The room a socket joins is `ws:` concatenated with the workspace identifier taken from inside its own signed token. A client therefore cannot request another workspace, because there is no field in which to request one. This converts isolation from a rule that must be checked into a property that cannot be violated — which is why the isolation test suite asserts it structurally rather than by attempting attacks.

6.4.2 The Yjs Relay

```
socket.on('sync-update', async (update) => {
  const bytes = new Uint8Array(update);
  const room = await getRoom(workspaceId);
  Y.applyUpdate(room.doc, bytes, socket.id);
  socket.to(rt.roomOf(workspaceId)).emit('sync-update', bytes);   // relay FIRST

  // History is appended AFTER the relay, on purpose. Collaborators must
  // never wait on a database write to see each other's edits, and
  // appendUpdate() swallows its own errors, so a broken log can disable
  // replay but can never stall or break live collaboration.
  logs.appendUpdate(workspaceId, bytes, { userId, username });
});

socket.on('awareness-update', (update) => {
```

```
15    socket.to(rt.roomOf(workspaceId)).emit('awareness-update', new Uint8Array(update));
16  });
```

Listing 6.7: Document relay — note that logging happens strictly after the broadcast

The ordering in Listing 6.7 is the single most consequential line ordering in the server. Logging is a bonus feature; it must fail like one. The automated suite asserts this directly with the check *"live collaboration still works with logging enabled"*.

6.4.3 Socket Events Documentation

Table 6.2: Socket event reference

Event	Direction	Socket kind	Payload and purpose
`sync-update`	Both	access	Binary Yjs update. Server relays to room peers and appends to the log
`awareness-update`	Both	access	Binary awareness update: cursors, selections, drafts, eraser ring, typing
`room-info`	S→C	access	{ workspaceId, users, connected[] } presence broadcast
`get-replay-logs`	C→S	access	Requests the session history; answered by the chunked stream
`replay-logs-begin`	S→C	access	{ count, capped, skipped, meta[] } — metadata only, no payloads
`replay-logs-chunk`	S→C	access	{ start, count, sizes[], data } — exactly one packed buffer
`replay-logs-end`	S→C	access	{ count } — transfer complete
`replay-logs-error`	S→C	access	{ code, message } — a specific, named failure
`admin:approve`	C→S	access (admin)	{ requestId }; acknowledged with a result object
`admin:reject`	C→S	access (admin)	{ requestId, reason }
`admin:set-policy`	C→S	access (admin)	{ permissionMode }

Continued on next page

Table 6.2 – continued from previous page

Event	Direction	Socket kind	Payload and purpose
`admin:remove-user`	C→S	access (admin)	{ userId }
`admin:pending`	C→S	access (admin)	Re-fetches the pending queue
`join:requested`	S→C	access (admin)	A new request arrived; pushed to the admin room
`join:resolved`	S→C	access (admin)	{ requestId, status }
`join:pending`	S→C	access (admin)	Queue delivered on connect, covering admin downtime
`join:waiting`	S→C	lobby	Confirms the request is still pending
`join:approved`	S→C	lobby	{ token, workspace } — the access token is *pushed*, not polled
`join:rejected`	S→C	lobby	{ reason }
`workspace:updated`	S→C	access	Membership changed
`workspace:policy-changed`	S→C	access	{ permissionMode }
`workspace:removed`	S→C	access	Sent immediately before a forced disconnect
`workspace:closed`	S→C	access	The workspace was closed by its administrator

6.4.4 Administrator Authority

```
const asAdmin = (handler) => async (payload, ack) => {
  if (socket.data.role !== 'admin')
    return ack?.({ ok: false, message: 'Only the administrator can do that.' });
  try {
    const result = await handler(payload || {});
    ack?.(result.ok ? result : { ok: false, message: result.message });
  } catch (err) {
    console.error('[socket] admin action failed:', err.message);
    ack?.({ ok: false, message: 'Something went wrong. Please try again.' });
  }
};

socket.on('admin:approve',    asAdmin(({ requestId }) => svc.approveRequest({ workspaceId, requestId }))
    ));
socket.on('admin:reject',     asAdmin(({ requestId, reason }) => svc.rejectRequest({ workspaceId,
    requestId, reason }))));
socket.on('admin:set-policy', asAdmin(({ permissionMode }) => svc.setPermissionMode({ workspaceId,
    permissionMode }))));
socket.on('admin:remove-user',asAdmin(({ userId: target }) => svc.removeMember({ workspaceId, userId:
    target, actorId: userId }))));
```

Listing 6.8: Administrator authority is read from the signed token, never from the payload

The role comes from `socket.data`, populated at handshake time from the signed token. A member emitting `admin:approve` receives an error, not an approval — and the test suite asserts exactly that, because privilege escalation through a forged payload field is the obvious attack on this design.

6.5 Persistence

6.5.1 Dual-Mode Repository Contract

Both `workspaceStore.js` and `updateLogService.js` implement the same contract: identical exported functions, backed either by Mongoose or by an in-memory structure, selected by a single boolean set once at boot. No caller is aware of which mode is active.

```
let persistent = false;
export function setPersistence(flag) { persistent = flag; }

const memory = new Map();                       // workspaceId -> plain object
const clone = (o) => (o ? JSON.parse(JSON.stringify(o)) : null);

/**
 * Apply a mutation function to a workspace and persist the result.
 * ONE single write path, which is what keeps concurrent joins from
 * clobbering each other.
 */
export async function updateWorkspace(workspaceId, mutate) {
  if (persistent) {
    const doc = await Workspace.findOne({ workspaceId });
    if (!doc) return { workspace: null, result: null };
```

```
    const result = mutate(doc);
    await doc.save();
    return { workspace: doc.toObject(), result };
  }
  const record = memory.get(workspaceId);
  if (!record) return { workspace: null, result: null };
  const result = mutate(record);
  record.updatedAt = new Date();
  return { workspace: clone(record), result };
}

/** Everything safe to send to a browser. NEVER includes passwordHash. */
export function publicView(ws) {
  if (!ws) return null;
  return {
    workspaceId: ws.workspaceId, name: ws.name,
    permissionMode: ws.permissionMode, status: ws.status,
    adminUsername: ws.adminUsername, createdAt: ws.createdAt,
    members: (ws.members || []).map((m) => ({
      userId: m.userId, username: m.username, role: m.role, joinedAt: m.joinedAt
    }))
  };
}
```

Listing 6.9: Dual-mode repository, showing the single write path

Two things are worth drawing out. First, `updateWorkspace` takes a mutation *function* rather than a patch object, which forces every write through one read–modify–save path and makes concurrent joins safe to reason about. Second, `publicView` is a whitelist rather than a blacklist: the password hash cannot leak by omission, because fields are copied in explicitly rather than deleted out.

6.5.2 Binary Snapshot Persistence

```
async function getRoom(workspaceId) {
  if (rooms.has(workspaceId)) return rooms.get(workspaceId);
  const doc = new Y.Doc();

  if (persistenceEnabled) {
    const saved = await DocState.findOne({ roomId: workspaceId }).lean();
    if (saved?.state) {
      Y.applyUpdate(doc, new Uint8Array(saved.state));
      console.log(`[yjs] "${workspaceId}" restored from MongoDB`);
    }
  }

  const entry = { doc, dirty: false, timer: null };

  doc.on('update', () => {
    if (!persistenceEnabled) return;
    entry.dirty = true;
    if (entry.timer) return;                // a snapshot is already scheduled
    entry.timer = setTimeout(async () => {
      entry.timer = null;
      if (!entry.dirty) return;
```

```
      entry.dirty = false;
      const state = Buffer.from(Y.encodeStateAsUpdate(doc));
      await DocState.findOneAndUpdate({ roomId: workspaceId },
        { roomId: workspaceId, state }, { upsert: true });
    }, 2000);
  });

  rooms.set(workspaceId, entry);
  return entry;
}
```

Listing 6.10: Room hydration and the debounced binary snapshot

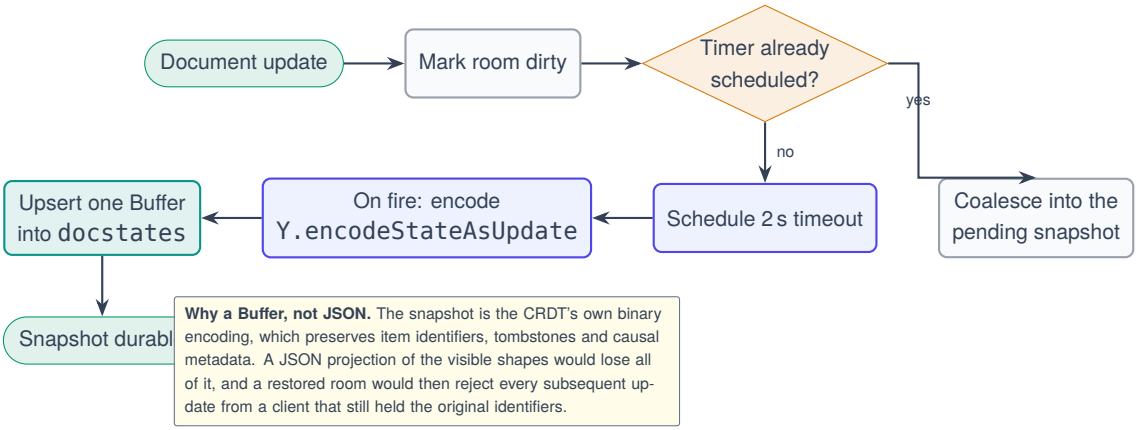


Figure 6.4: MongoDB persistence workflow — debounced binary snapshot

6.5.3 Update Logging

The update log implements two capacity mechanisms that were added after measurement showed the original design was inadequate in practice.

```
export const MAX_LOGS_PER_ROOM      = 50000;
export const MAX_LOG_BYTES_PER_ROOM = 32 * 1024 * 1024;
export const COALESCE_WINDOW_MS     = 400;
export const MAX_COALESCED_BYTES    = 16 * 1024;

export async function appendUpdate(roomId, payload, meta = {}) {
  try {
    if (!roomId || !payload || !payload.length) return null;
    await ensureSeeded(roomId);
    const buf = Buffer.from(payload);
    const now = Date.now();

    // ---- coalesce into the previous entry when part of the same burst
    const prev = lastEntry.get(roomId);
    const sameBurst = prev
      && (prev.userId || null) === (meta.userId || null)
      && now - prev.atMs <= COALESCE_WINDOW_MS
      && prev.payload.length + buf.length <= MAX_COALESCED_BYTES;

    if (sameBurst) {
      const merged = mergeSafely(prev.payload, buf);
```

```
      if (merged && merged.length <= MAX_COALESCED_BYTES) {
        /* rewrite the previous row in place; keep its ORIGINAL start time so
           a continuous drag cannot coalesce forever into one entry */
        lastEntry.set(roomId, { ...prev, payload: merged });
        return { roomId, seq: prev.seq, payload: merged, coalesced: true };
      }
    }

    // ---- otherwise append, if there is budget for it
    const seq  = nextSeq.get(roomId);
    const used = bytesUsed.get(roomId) || 0;
    if (seq >= MAX_LOGS_PER_ROOM) return null;                    // count budget
    if (used + buf.length > MAX_LOG_BYTES_PER_ROOM) return null;  // byte budget
    nextSeq.set(roomId, seq + 1);                                 // SYNCHRONOUS
    /* ... insert the row, update bytesUsed and lastEntry ... */
  } catch (err) {
    console.warn('[replay] append failed:', err.message);
    return null;   // never throws: this runs on the hot path of every edit
  }
}
```

Listing 6.11: Append with burst coalescing and a two-sided budget

A defect found by testing, not by users: duplicate sequence numbers

The original implementation awaited the lazy sequence seed *between* reading the counter and advancing it. Two updates arriving back to back — and a drag emits roughly twenty-two per second — could both resume from that `await` holding the same value and be written under duplicate sequence numbers, silently corrupting the replay slider's axis on precisely the long sessions replay exists for. Seeding now runs once behind a shared promise and allocation is a synchronous read-and-increment, making interleaving impossible. A long-session ordering assertion pins the behaviour permanently.

Why the cap trims the tail and never the head

Yjs updates form a causal chain: an update that edits a shape references the update that created it. A *prefix* of the log is therefore always a complete, internally consistent document — which is exactly what the slider scrubs through. A *suffix* is not: discard the early updates and the later ones reference items the document has never seen, so Yjs parks them as pending and the board replays blank.

A ring buffer is therefore the one structure that must not be used here. When a room reaches its ceiling, recording stops and a `capped` flag is set, which the interface displays honestly as a "history full" indicator. An honest partial history is strictly better than a corrupted history that looks complete.

6.6 Execution Service

6.6.1 Architecture

The execution subsystem is a self-contained module that imports nothing from the collaboration server. Its structure separates four concerns: the language registry (data), the canonical result shape, the network client, and one adapter per provider.

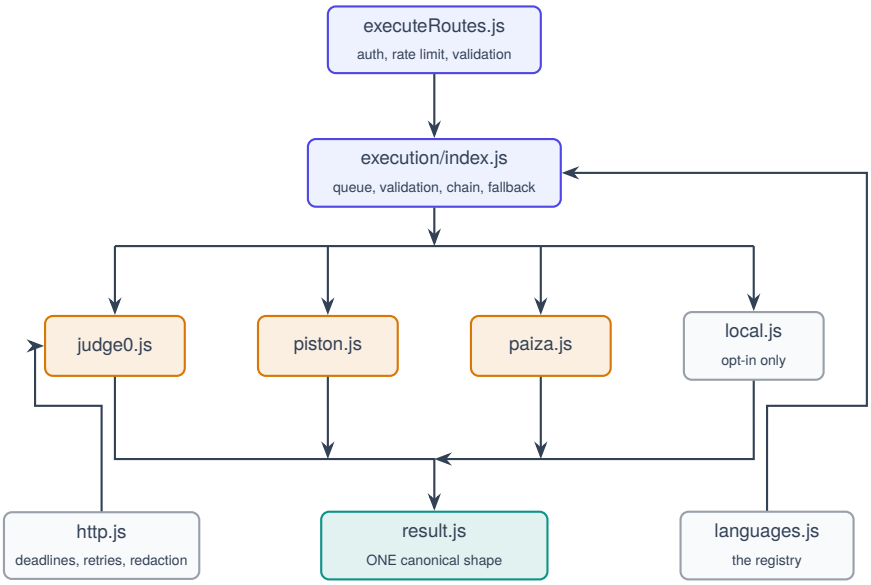


Figure 6.5: Execution service architecture — the orchestrator knows nothing about any specific provider

6.6.2 The Canonical Result

Every adapter must return exactly one shape, constructed by `makeResult()` so that no field can be omitted. The user interface never sees a Judge0 `status.id`, a Piston `signal` or a Paiza `build_result`, which is precisely why the output panel is identical for every language and every provider.

Table 6.3: Canonical execution result fields

Field	Type	Meaning
`ok`	Boolean	Compiled, ran and exited zero
`phase`	Enum	`setup` | `compile` | `run` — where it stopped
`stdout`, `stderr`	String	Program output, each capped at 64 KB by us
`compileOutput`	String	Compiler diagnostics, kept separate from stderr
`exitCode`, `signal`	Number, String	Termination detail
`exitReason`	Enum	`exit` | `signal` | `timeout` | `error`
`status`	Enum	One of ten machine-readable outcomes
`statusText`	String	One human sentence describing the outcome
`timedOut`, `truncated`	Boolean	Bound conditions
`durationMs`, `memoryKb`	Number	Measured cost
`provider`, `providerLabel`	String	Which backend actually ran it
`attempts`	Number	How many providers or retries were consumed
`warnings`	String[]	Non-fatal notes, e.g. the Java class-name check

6.6.3 Two Decisions Worth Explaining

Judge0 language identifiers are resolved at runtime

`C (GCC 9.2.0)` is identifier `50` on one Judge0 instance and something else on the next. Hard-coding `50` is the single most common way these integrations break. The adapter fetches `GET /languages`, matches on the language *family* by regular expression, and selects the newest match — which is also how it avoids selecting Python 2.7 when Python 3 was requested. The registry's `fallbackId` is used only when that call fails.

Everything is base64-encoded

GCC emits non-printable bytes in compiler diagnostics, and any program printing emoji or CJK text breaks a plain-text submission. Setting `base64_encoded=true` on both request and response is the only way Unicode survives intact end to end. This is verified in the test suite with a string containing accented Latin, CJK and emoji

characters across all five languages.

6.6.4 Failure Handling

Table 6.4: Execution failure modes and behaviour

Situation	Behaviour
Compile error	`status: compile_error`, diagnostics in `compileOutput`, phase `compile`
Runtime crash	`status: runtime_error` with a plain-English signal reason (e.g. "Segmentation fault — the program touched memory it does not own")
Infinite loop	Killed by the provider; `status: timeout`
Large output	Capped at 64 KB per stream by us, on top of the provider's own limit; trimmed again to 8 KB before entering the shared document
HTTP 429	Retried with full-jitter backoff, then falls through to the next provider
HTTP 5xx / network failure	Same, and the message names which providers failed and why
Malformed response	Rejected rather than parsed into nonsense
All providers unavailable	One readable message listing each provider and its reason — never a crash

Retries fire only on genuinely transient failures. A 4xx that is not 429 means the service will reject the request just as firmly next time, so retrying is pure added latency — an explicit distinction encoded in the `retryable` flag of `ProviderError`.

6.7 REST API Documentation

Table 6.5: REST API endpoint reference

Method	Path	Auth	Purpose and response
GET	`/api/health`	None	Liveness, service name and execution queue statistics
POST	`/api/workspaces`	None	Create a workspace. 201 with { workspace, token }
GET	`/api/workspaces/:id`	None	Minimal teaser: id, name, status. Access policy is deliberately *not* revealed
POST	`/api/workspaces/:id/join`	None	Join. 200 with either { status: approved, token } or { status: pending, ticket }. Rate limited to 10/min per IP per workspace
GET	`/api/workspaces/:id/me`	Member	Re-hydrate after reload: user, workspace, pending requests (admin only)
GET	`/api/workspaces/:id/requests`	Admin	The pending join queue
POST	`/api/workspaces/:id/requests/:rid/approve`	Admin	Approve; pushes an access token down the waiting socket
POST	`/api/workspaces/:id/requests/:rid/reject`	Admin	Reject with an optional reason (truncated to 140 characters)
PATCH	`/api/workspaces/:id/policy`	Admin	Switch access policy at runtime
DELETE	`/api/workspaces/:id/members/:uid`	Admin	Remove a member and force-disconnect their sockets
POST	`/api/workspaces/:id/close`	Admin	Close the workspace; notifies members and outstanding requests
GET	`/api/workspaces/:id/execute/languages`	Member	The language catalogue with per-language availability
GET	`/api/workspaces/:id/execute/providers`	Member	Provider diagnostics for a misconfigured deployment
POST	`/api/workspaces/:id/execute`	Member	Run { language, code, stdin }. Rate limited to 20 runs / 30 s per user

6.8 Module Responsibility Summary

Table 6.6: Server module responsibilities, inputs and outputs

Module	Input	Output	Key data structure
`server.js`	Environment, HTTP requests	Running server	Express app, Socket.IO server
`socketService.js`	Signed token, binary updates	Relayed updates, streamed history	`Map` of workspaceId to { doc, dirty, timer }
`workspaceService.js`	Command parameters	{ ok, ... } result objects	Plain workspace documents
`workspaceStore.js`	Mutation functions	Persisted documents	`Map` (memory) or Mongoose model
`updateLogService.js`	Room id, binary payload	Ordered log entries	Per-room arrays plus counter, byte and last-entry maps
`authMiddleware.js`	Authorization header	`req.user`, `req.workspace`	Decoded JWT payload
`validate.js`	Raw client strings	Cleaned values or errors	`Map` of rate-limit buckets
`realtime.js`	Event name, payload	Socket emissions	Bound Socket.IO reference
`execution/index.js`	{ language, code, stdin }	Canonical result	FIFO queue array, provider probe cache
`execution/providers/*`	Normalised request	Canonical result	Provider-specific wire formats

CHAPTER 7

Implementation II — Client Side

7.1 Application Shell and Routing

The client is a React 19 single-page application with five routes, each corresponding to one stage of the participant lifecycle established in Figure 5.10.

Table 7.1: Client routes and their responsibilities

Route	Component	Responsibility
`/`	`Landing`	Two doors only: create or join
`/create`	`CreateWorkspace`	Name, secret code, access policy; issues an admin token
`/join`	`JoinWorkspace`	Workspace id, display name, secret code
`/waiting/:id`	`WaitingRoom`	Live lobby socket; receives the verdict without polling
`/workspace/:id`	`Workspace`	The collaborative session shell

An earlier iteration included a dashboard on which a participant typed a room identifier to enter directly. It was removed rather than refactored, because type-an-identifier-to-enter is precisely the insecure behaviour the permission system exists to replace. The whole application is wrapped in an `ErrorBoundary`, and individual shapes are additionally wrapped in a `ShapeErrorBoundary` so that one malformed record cannot blank the board.

7.2 The Collaboration Provider

7.2.1 Purpose

`useCollaboration.js` is the Yjs provider: a custom React hook that owns the `Y.Doc` and the awareness instance for the session, bridges both to the socket, and additionally carries the workspace-management channel so the administrator panel needs no second connection.

7.2.2 Internal Working

```
const ydoc      = new Y.Doc();
const awareness = new Awareness(ydoc);
const socket    = createSocket(session.token);

awareness.setLocalStateField('user', {
  name: session.username, color: colorFor(session.username), role: session.role
});

// ---- outgoing ------------------------------------------------------------
const onDocUpdate = (update, origin) => {
  if (origin === 'remote') return;          // <- this stops the echo loop
  socket.emit('sync-update', update);
};
ydoc.on('update', onDocUpdate);

const onAwareness = ({ added, updated, removed }, origin) => {
  if (origin === 'remote') return;
  const changed = [...added, ...updated, ...removed];
  socket.emit('awareness-update', encodeAwarenessUpdate(awareness, changed));
};
awareness.on('update', onAwareness);

// ---- incoming ------------------------------------------------------------
socket.on('sync-update', (update) => {
  Y.applyUpdate(ydoc, new Uint8Array(update), 'remote');
});
socket.on('awareness-update', (update) => {
  applyAwarenessUpdate(awareness, new Uint8Array(update), 'remote');
});
```

Listing 7.1: The synchronisation core — outgoing and incoming paths

The origin tag is the whole loop-prevention mechanism

Every Yjs transaction carries an *origin*. A remote update is applied with origin `'remote'`; the outgoing handler returns immediately for that origin. Without this single guard, applying a received update would fire the document's own update event, which would emit it straight back to the server, which would relay it onward — an infinite amplification loop. The same origin mechanism is later reused for two further purposes: excluding high-frequency mid-drag writes from the undo stack, and excluding a peer's edits from local undo (Section 7.7).

7.2.3 Data Flow of a Single Edit

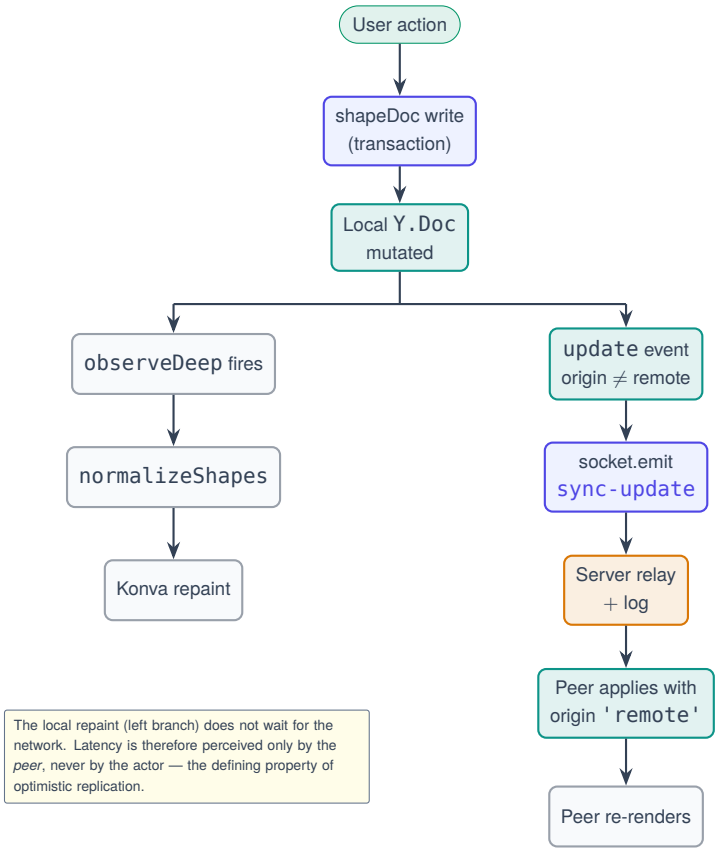


Figure 7.1: Data flow of a single edit from action to peer repaint

7.3 The Shared Document Layer

7.3.1 The Single Write Path

`canvas/shapeDoc.js` is the only module that writes to the shared document. Concentrating every mutation in one file is what makes concurrent behaviour tractable to reason about and keeps the Canvas component from accumulating scattered transaction calls.

```
export const shapesArray = (ydoc) => ydoc.getArray('shapes');

/** Turn a plain object into a Y.Map, converting `points` into a Y.Array. */
function toYMap(obj) {
  const map = new Y.Map();
  for (const [k, v] of Object.entries(obj)) {
    if (k === 'points' && Array.isArray(v)) {
      const arr = new Y.Array(); arr.push(v); map.set(k, arr);
    } else map.set(k, v);
  }
  return map;
}

/**
 * `undefined` means "remove this field". Yjs happily STORES undefined
```

```
 * otherwise, leaving a dead key that reads back as a real (undefined)
 * property and defeats every `?? default` in the renderer.
 */
function applyField(map, k, v) {
  if (v === undefined) map.delete(k); else map.set(k, v);
}

export function updateShape(ydoc, id, patch) {
  const map = findMap(ydoc, id);
  if (!map) return;
  ydoc.transact(() => {
    for (const [k, v] of Object.entries(patch)) applyField(map, k, v);
    map.set('updatedAt', Date.now());
  });
}
```

Listing 7.2: Shape creation and field-level patching

Why a shape is a `Y.Map` and not a plain object

If a shape were stored as a plain JavaScript object inside a `Y.Array`, any modification would replace the *entire* object, and two concurrent modifications would resolve as last-writer-wins across all fields — one participant's colour change would silently erase another's position change. Storing each shape as a `Y.Map` moves the merge boundary down to the individual field. This is the concrete mechanism behind the convergence claim, and it is proved directly by `test-shapes.mjs`, which has two clients edit the same shape (one recolouring, one moving) and asserts both survive.

7.3.2 Live Drag Streaming

```
export const LIVE_ORIGIN = 'live';

export function updateShapeLive(ydoc, id, patch) {
  const map = findMap(ydoc, id);
  if (!map) return;
  ydoc.transact(() => {
    for (const [k, v] of Object.entries(patch)) map.set(k, v);
  }, LIVE_ORIGIN);          // <- the UndoManager does not track this origin
}
```

Listing 7.3: Untracked live writes during a drag

During a drag, position commits are emitted every 45 ms under the `'live'` origin. Because `Y.UndoManager` tracks only origin-`null` transactions by default, remote participants watch the object glide in real time — with every attached connector re-routing frame by frame — yet the entire drag collapses into a single undo step, because only the final `dragEnd` commit is tracked.

7.3.3 The Normalisation Gate

Yjs guarantees that all replicas converge on the *same* document. It does not guarantee that the document is *well-formed*. A record can legitimately arrive half-built: from an older client version, from an interrupted transaction, from a hand-edited snapshot, from a field an undo rolled back to `undefined`, or from a shape whose creation raced with a peer's deletion.

Konva reacts badly to non-finite geometry — a single `NaN` in a points array silently poisons an entire canvas path — so `canvas/normalize.js` coerces every record to something drawable once, before it can reach a Konva node.

```
/**
 * Non-finite entries are dropped in PAIRS so x/y never desynchronise --
 * a half-dropped coordinate would shear the whole stroke.
 */
export function cleanPoints(v) {
  if (!Array.isArray(v)) return [];
  const out = [];
  for (let i = 0; i + 1 < v.length; i += 2) {
    const x = Number(v[i]), y = Number(v[i + 1]);
    if (Number.isFinite(x) && Number.isFinite(y)) out.push(x, y);
  }
  return out;
}

export function normalizeShapes(list) {
  if (!Array.isArray(list)) return [];
  const seen = new Set();
  const out = [];
  for (let i = 0; i < list.length; i++) {
    const s = normalizeShape(list[i], i);
    // Two records sharing an id give React duplicate keys, which makes it
    // reuse or destroy the wrong Konva node -- "shapes that won't select".
    if (seen.has(s.id)) s.id = `${s.id}__dup${i}`;
    seen.add(s.id);
    out.push(s);
  }
  return out;
}
```

Listing 7.4: Point-array sanitisation: coordinates are dropped in pairs

The module also repairs legacy records: an early freehand format carried `{ id, color, points }` with no `type` field, and is silently promoted to a modern `path` record with `color` mapped onto `stroke`. This is why strokes drawn in the first milestone still render correctly today.

7.4 Whiteboard Rendering Pipeline

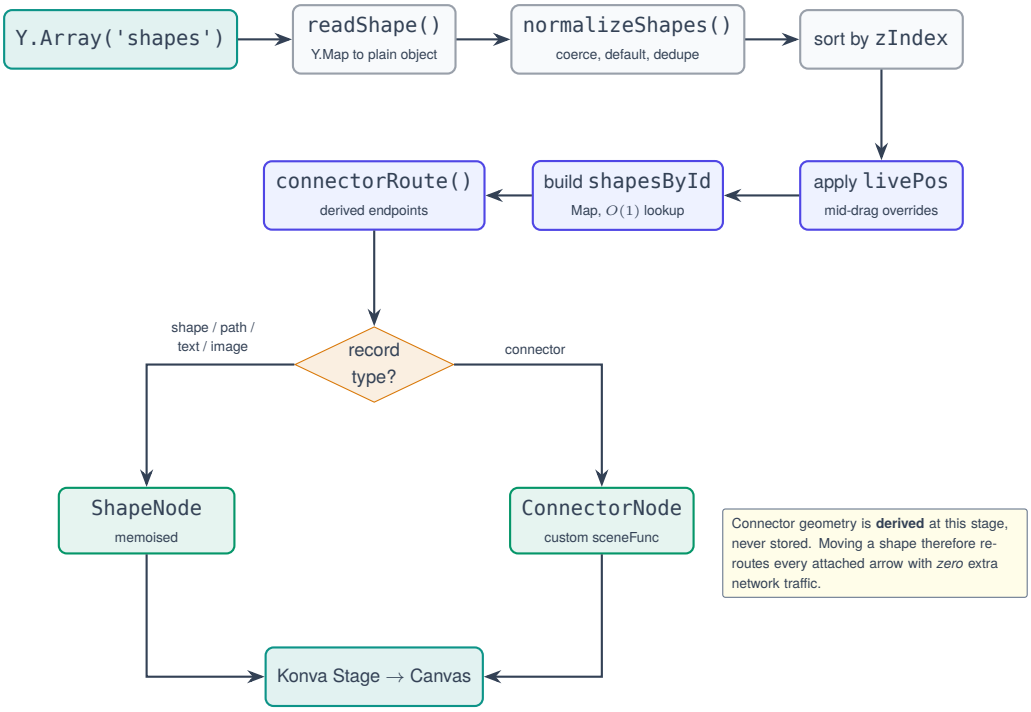


Figure 7.2: Whiteboard rendering pipeline

7.4.1 Shape Registry

Sixteen primitives are exposed in five toolbar groups. Every polygon routes through one shared `shapePoints(type, w, h)` generator, so the outline used for rendering is the same outline used for connector attachment and hit testing — they cannot disagree.

Table 7.2: Shape primitives by toolbar group

Group	Primitives
Flowchart	Process (rectangle), Start/End (rounded rectangle), Decision (diamond), Input/Output (parallelogram)
Geometric	Circle, Ellipse, Triangle, Pentagon, Hexagon, Star, Heart
Extra	Trapezoid, Cross, Speech bubble, Cloud
Lines	Line
Connectors & Arrows	Twelve presets over the single connector type: connector, elbow, curved, arrow, double, dashed, dotted, thick, thin, hollow head, open, block

Every one of the twelve connector presets is nothing more than a set of field values on the one connector record type — one render path, many appearances.

7.5 The Connector System

7.5.1 The Core Idea

A connector is a flat record in the same `shapes` array as every rectangle, but its endpoints may *reference* other shapes:

```
{
  type: 'connector',
  start: { shapeId: 'abc', anchor: 'auto', x: 300, y: 150 },  // attached
  end:   { x: 620, y: 240 },                                  // free
  waypoints: [x1, y1, x2, y2, ...],       // user-inserted bend points
  routing: 'straight' | 'elbow' | 'curved',
  curvature, cornerRadius,
  startHead / endHead: 'none'|'filled'|'hollow'|'open'|'block'|'bar',
  stroke, strokeWidth, dash, opacity, locked, zIndex
}
```

Listing 7.5: Connector record structure

Derivation replaces mutation

An attached endpoint's position is *derived at render time* from wherever the referenced shape currently is. Nothing is ever "re-attached", because nothing is ever detached. Move a box and every connector touching it recomputes on the next frame — locally, remotely, with zero additional network messages, because no connector record was modified. The cached x/y on an endpoint is used only as a fallback if the referenced shape is later deleted, which is why deleting a box never breaks its arrows.

The corollary is that because a connector is an ordinary `Y.Map` inside the ordinary `shapes` array, it inherits selection, deletion, undo, multi-select property editing, locking, layer ordering, duplication, persistence, replication and replay *for free* — the same code paths as every other object, not a parallel system.

7.5.2 Smart Edge Attachment

```
/**
 * The point where the segment centre->towards leaves the shape's REAL
 * outline (circle edge, diamond edge, star point -- not the bounding box).
 * This is what makes 'auto' anchors slide around the perimeter as the
 * other end moves.
 */
export function edgePoint(shape, towards) {
  const c = shapeCenter(shape);
  const poly = shapeOutline(shape);
  let best = null;
  for (let i = 0; i < poly.length; i++) {
    const hit = segIntersect(c, towards, poly[i], poly[(i + 1) % poly.length]);
    if (hit && (!best || hit.t > best.t)) best = hit;   // furthest from centre
  }
  return best ? { x: best.x, y: best.y } : c;
```

```
16 }
```

Listing 7.6: Attachment to the real outline, not the bounding box

7.5.3 Connector Routing Flow

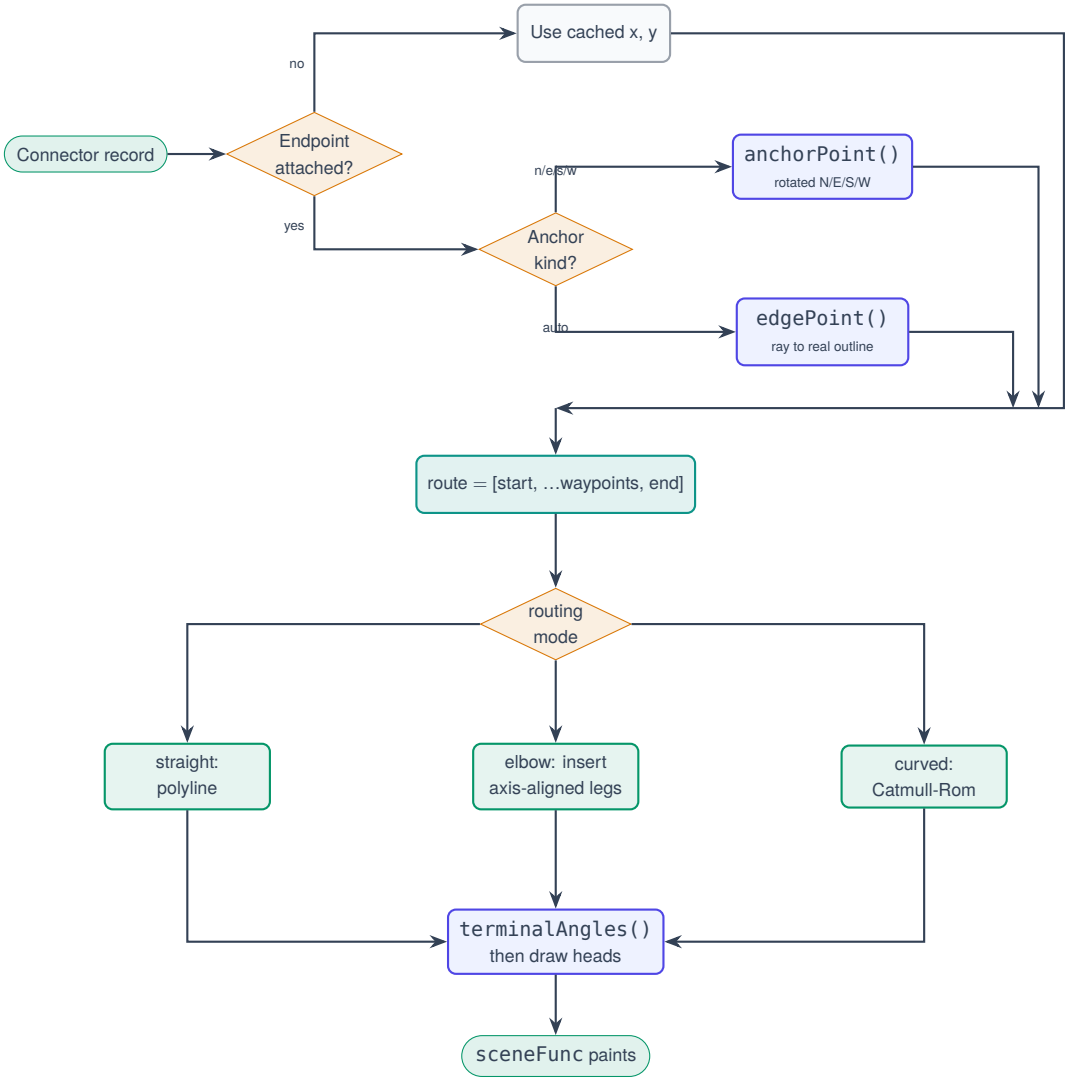


Figure 7.3: Connector routing flow, executed per connector per frame

7.5.4 Rendering and Hit Testing

`ConnectorNode` is a single custom Konva `Shape`. Its `sceneFunc` traces the body — with dash support and quadratic-arc rounded elbows — and then draws the two heads, trimming the body back so it never protrudes through a hollow head. Its `hitFunc` strokes the same path at a minimum width of 14 px and is *never* filled: filling an open polyline would make the entire enclosed region clickable. Consequently a one-pixel dotted connector is still easy to grab.

7.6 Brush Engine and Eraser

7.6.1 Brush Registry

Table 7.3: The seven brush styles

Brush	Width	Opacity	Composite	Character
Pen	4	1.00	source-over	Baseline round-cap stroke
Pencil	2	0.75	source-over	Thin and slightly translucent
Marker	12	0.92	source-over	Broad and near-opaque
Highlighter	18	0.35	multiply	Overlaps darken, as physical ink does
Calligraphy	14	1.00	source-over	Variable-width nib ribbon (filled polygon)
Dashed	4	1.00	source-over	Dash array scaled to stroke width
Dotted	4	1.00	source-over	Round cap with a near-zero dash

Calligraphy is the only brush that is not a constant-width line. It is a filled polygon whose visible thickness at each point is the projection of a flat nib, held at `nibAngle`, onto the stroke's normal — so travel perpendicular to the nib swells and travel parallel to it tapers, producing the classic copperplate swell.

7.6.2 Live Drawing Performance

The in-progress stroke lives in local component state and is broadcast over *awareness*, so peers watch it being drawn; it is written to the document exactly once, on pointer-up. This replaced an earlier loop that pushed every sampled point into the shared array, which re-snapshotted every shape on every pointer move and made a single stroke dozens of undo steps and dozens of network messages. `ShapeNode` is additionally wrapped in `React.memo`, so during a draw the hundreds of *existing* nodes are skipped and only the preview repaints.

7.6.3 The Partial Eraser

The eraser is a true vector eraser: dragging across a stroke removes only the portion touched, and the remainder survives as independent strokes.

```
/**
 * Mark every vertex swept by an eraser of `radius` dragged from (ax,ay)
 * to (bx,by) -- i.e. everything inside the capsule the circle traces.
 *
 * This replaces stamping the circle at N sampled points along the segment.
 * Sampling was both slower (O(samples x vertices) every mousemove, with the
 * sample count growing with pointer speed) and approximate -- too few
```

```
 * samples left scalloped gaps. The capsule is the EXACT swept region at one
 * distance test per vertex, so a fast flick and a slow rub behave identically
 * and frame cost stops depending on pointer speed.
 */
export function markErasedSweep(flat, ax, ay, bx, by, radius, into) {
  const r2 = radius * radius;
  const n = flat.length / 2;
  for (let i = 0; i < n; i++) {
    if (segDist2(flat[i*2], flat[i*2+1], ax, ay, bx, by) <= r2) into.add(i);
  }
  return into;
}

/**
 * Given a stroke's points and the erased vertex indices, return the
 * SURVIVING runs. The gap becomes a clean break, so one stroke split down
 * the middle yields two independent strokes.
 */
export function surviveRuns(flat, erased) {
  const n = flat.length / 2;
  const runs = []; let cur = [];
  for (let i = 0; i < n; i++) {
    if (erased.has(i)) { if (cur.length >= 4) runs.push(cur); cur = []; }
    else cur.push(flat[i*2], flat[i*2+1]);
  }
  if (cur.length >= 4) runs.push(cur);
  return runs;
}
```

Listing 7.7: Swept-capsule erasure and run reconstruction

```
export function applyErase(ydoc, user, edits) {
  if (!edits.length) return [];
  const arr = shapesArray(ydoc);
  const newIds = [];
  ydoc.transact(() => {                       // <- ONE transaction
    for (const { id, runs } of edits) {
      const base = readShape(orig);
      arr.delete(origIndex, 1);               // remove the original
      for (const run of runs) {               // push one fresh path per run
        const frag = { ...base, id: makeId(ydoc), points: run, updatedAt: Date.now() };
        arr.push([toYMap(frag)]);
        newIds.push(frag.id);
      }
    }
  });
  return newIds;
}
```

Listing 7.8: Atomic commit of a whole eraser drag

The single transaction is the load-bearing idea

Deleting the original and pushing the surviving fragments inside *one* transaction produces three properties simultaneously: the whole erase is one undo step (undo restores the original stroke intact; redo re-splits it); collaborators receive one atomic update, so no half-erased intermediate state can be observed and nothing can interleave to corrupt the split; and no change to the freehand representation was required, because a stroke was already a point list — "erase" is simply "split the point list and re-emit the surviving runs as ordinary strokes".

Why the eraser once felt like it removed particles

Erasing removes whole vertices, so *the vertex spacing is the erase granularity* — and strokes are RDP-simplified on commit, which is precisely the operation that deletes redundant vertices. A long straight drag could commit as two points tens of pixels apart, so rubbing over it removed the entire span at once and the erased edge snapped between surviving vertices instead of tracking the cursor. The fix is to densify each stroke to roughly two-pixel spacing once per erase session, making the erasable unit far smaller than the eraser itself. Fragments are re-simplified before commit, so the densified points never leak into the document, the network or the undo history.

7.7 Undo and Redo

```
// trackedOrigins defaults to {null}: the throttled 'live' mid-drag writes are
// NOT tracked, so a whole drag still undoes as one step.
const undoMgr = useMemo(
  () => new Y.UndoManager(yshapes, { captureTimeout: 400 }),
  [yshapes]
);
```

Listing 7.9: Undo manager scoped to the shapes array and to local origins

Three origins therefore carry three different meanings, and together they produce correct collaborative undo:

Table 7.4: Transaction origins and their undo semantics

Origin	Produced by	Undo behaviour
`null`	A local, deliberate edit	Tracked — this is what Ctrl+Z reverses
`'live'`	Throttled mid-drag position commits, and z-order bumps on selection	Not tracked, so a drag is one step and merely clicking a shape does not fill the undo stack
`'remote'`	An update received from a peer	Not tracked, so local undo can never revert a collaborator's work

Because every action — pen strokes, partial erasing, shapes, connectors, text, moves, resizes, rotations, colour and brush changes, deletions and layer ordering — is a tracked transaction on the *one* shapes array, all of them undo and redo in chronological order with no per-feature code. The claim that local undo never reverts a remote stroke is asserted directly in `test-brushes.mjs`.

A subtle undo defect that was fixed

An earlier version called `bringToFront()` on *every* selection click. This had two consequences: it produced a document mutation and a network broadcast for an action that changed nothing visible, and it pushed an entry onto the undo stack, so pressing Ctrl+Z after merely selecting something undid an invisible z-order bump instead of the user's last real edit. The function is now a no-op when the shape is already frontmost and writes under the untracked `'live'` origin; explicit reordering still goes through `reorderShape()` and remains undoable.

7.8 Camera, Export and Object Verbs

The viewport is *local* state: every participant keeps their own camera. The wheel zooms to the pointer between $0.2\times$ and $4\times$, space-drag pans, and footer controls provide explicit increments. All document coordinates remain world coordinates — the stage transform is purely presentational — which is why every pointer read goes through stage.get Relative Pointer Position(). Selection handles, remote cursors and selection chrome are scaled by $1/\text{zoom}$ so they retain a constant on-screen size at any magnification.

The object system provides copy, paste and duplicate with connector re-wiring: copies attached to shapes *inside* the copied set re-attach to the new copies, while attachments to shapes *outside* the set become free endpoints, so a pasted arrow never grabs the original. This is implemented as a two-pass identifier remap in `duplicateShapes`. Select-all, lock/unlock, bring-forward, send-backward and PNG export complete the verb set.

7.9 The Collaborative Code Editor

7.9.1 Shared State

Table 7.5: Editor state and where each element lives

Element	Location	Effect
Code buffer	`Y.Text('monaco')`	Everyone types in one file, remote cursors included
Active language	`Y.Map('editorMeta')`	The dropdown *and* Monaco's syntax mode switch for everyone together
Run console	`Y.Array('runHistory')`, capped at 20	Every result — stdout, stderr, compiler output, exit code, duration, who ran it — appears for all collaborators and survives a reload
Program input	Local component state	Deliberately private: each person experiments with their own stdin
Theme, wrap, console height	Local component state	Personal display preferences

```
const ytext = ydoc.getText('monaco');
bindingRef.current = new MonacoBinding(
  ytext, editor.getModel(), new Set([editor]), awareness
);
```

Listing 7.10: Binding the Monaco text model to the shared text type

Passing `awareness` as the fourth argument is what produces remote cursors and coloured remote selections inside the editor, using the same awareness channel that drives the whiteboard cursors.

7.9.2 Editor Synchronisation Workflow

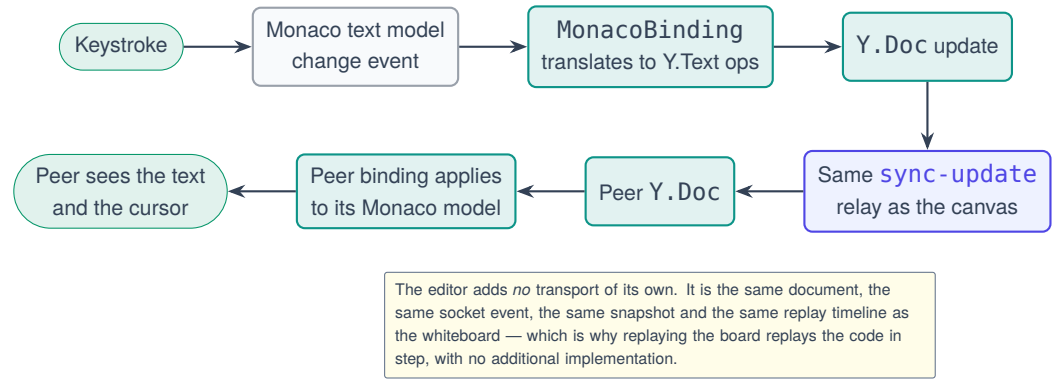


Figure 7.4: Editor synchronisation workflow

7.9.3 Execution Workflow

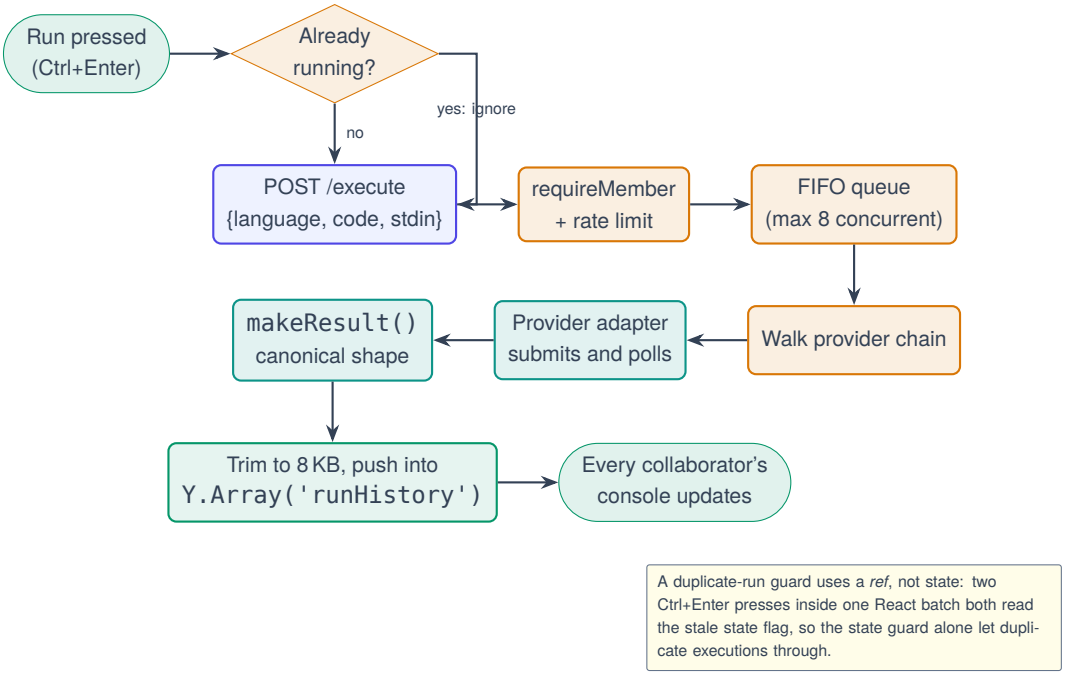


Figure 7.5: Code execution workflow, from keystroke to shared console

The result written into the shared document is trimmed to 8 KB per stream before it enters the CRDT, on top of the 64 KB the backend already applies. The reason is that a shared console entry is replicated to every peer *and* persisted in the snapshot: without the second trim, one print-loop would bloat the document for everyone.

7.10 Session Replay Interface

`ReplaySlider.jsx` does not rewind the live document; it builds a second one. The scrub document is a local object that no socket writes to and that never emits, so there is no code path by which scrubbing could affect the live board. Nothing had to be locked, frozen or guarded to make that true — it is a property of the structure.

It also reuses the same renderer: `normalizeShapes` → `ShapeNode`/`ConnectorNode`. History is therefore drawn by the code under test rather than by a second implementation that could drift, and a connector still re-routes against the shapes as they existed at that moment, because routing is derived at render time from whatever document it is handed. This is also why uploaded images and stickers required no replay-specific work at all: replay applies opaque updates and never inspects the shape schema.

The panel loads in two visible phases — *downloading*, as chunks arrive, and *preparing*, as the reconstruction cache is warmed in batches of 200 updates through `setTimeout(0)`. The second phase exists because the first frame shown is "now", which requires applying the entire log once; doing that synchronously would lock the interface thread for thousands of updates. Warming also lays down the cache checkpoints, so the very first backward scrub is already fast. Controls provide play and pause, single stepping, speeds of $0.5\times$ to $4\times$, fit and zoom, and keyboard operation.

7.11 Supporting Panels

Table 7.6: Supporting client components

Component	Responsibility
`Toolbar`	Tool selection, shape menu with five groups, undo/redo buttons with live enabled state
`PropertyPanel`	Contextual editing of the selection: fill (solid, linear, radial), stroke, dash, width, opacity, corner radius, shadow, blur, rotation, text formatting, connector routing and heads, lock, arrange, duplicate, reset appearance. Every change writes straight to the document, so it is live for all peers
`BrushPanel`	Floating panel shown while the pen or eraser is active: seven styles, fifteen-swatch palette plus picker plus recent colours, thickness, opacity, smoothing and pressure toggles, and a live preview. Settings persist to `localStorage`; each stroke copies them once at creation, so adjusting the panel never alters strokes already on the board
`AdminPanel`	Live join-request queue with approve and reject, runtime policy switch, connected-member list with removal
`ChatPanel`	Workspace chat over `Y.Array('chatMessages')` with awareness-based typing indicators. Requires no server code at all
`WaitingRoom`	Lobby socket, live status, and receipt of the pushed access token on approval
`Toast`, `ErrorBoundary`	Transient notifications; failure containment at application and per-shape granularity

The chat feature deserves a specific remark, because it illustrates the payoff of the architecture more compactly than any other subsystem: it is a complete real-time messaging feature with history, capping, attribution, unread counts and typing indicators, and it required **zero lines of server code and zero new dependencies**. Messages are an array in the existing document, and typing indicators are a field in the existing awareness state.

CHAPTER 8

Algorithms and Workflows

This chapter isolates the algorithmic content of the system. Each section states the problem, the algorithm chosen, its complexity, and the specific reason it was preferred over the obvious alternative.

8.1 CRDT Conflict Resolution

8.1.1 The Convergence Property

A CRDT is a data type whose merge operation $\sqcup$ satisfies three algebraic properties over the set of replica states S:

$$a \sqcup b = b \sqcup a \qquad \text{(commutativity)} \tag{8.1}$$

$$(a \sqcup b) \sqcup c = a \sqcup (b \sqcup c) \qquad \text{(associativity)} \tag{8.2}$$

$$a \sqcup a = a \qquad \text{(idempotence)} \tag{8.3}$$

Commutativity means delivery order does not matter; associativity means grouping does not matter; idempotence means duplicate delivery does not matter. Given all three, any two replicas that have received the same *set* of updates are in the same state — regardless of the order or multiplicity in which those updates arrived. This is the entire correctness argument, and it is why the server does not arbitrate.

8.1.2 Merge Granularity in SyncSpace

Yjs implements YATA for sequences and a last-writer-wins register per key for maps. The design decision that matters in SyncSpace is therefore not *which* CRDT, but *at what granularity* the shape record is modelled.

Table 8.1: Merge outcomes by modelling choice

Concurrent operations	**Plain object in array**	**`Y.Map` per shape**	**Verdict**
A sets `fill`; B sets `x`	One overwrites the other entirely — a change is lost	Different keys; *both* survive	Field-level granularity is essential
A sets `fill=red`; B sets `fill=blue`	Last writer wins	Last writer wins, resolved identically on every replica	Deterministic either way
A appends to `points`; B appends to `points`	Whole array replaced	`Y.Array` interleaves both	Nested type needed for strokes
A deletes shape; B moves shape	Undefined; may resurrect	Deletion wins; the move applies to a tombstoned item and is discarded	Correct and consistent

The second row is worth dwelling on because it is often mistaken for a defect. When two participants set the *same* field concurrently, one value must win — there is no meaningful "merge" of red and blue. What the CRDT guarantees is not that both survive, but that *every replica chooses the same winner*, deterministically, without communication. Convergence is the guarantee; preservation of both intentions is available only when the intentions touch different fields, which is exactly why the granularity choice matters.

8.1.3 Yjs Synchronisation Workflow

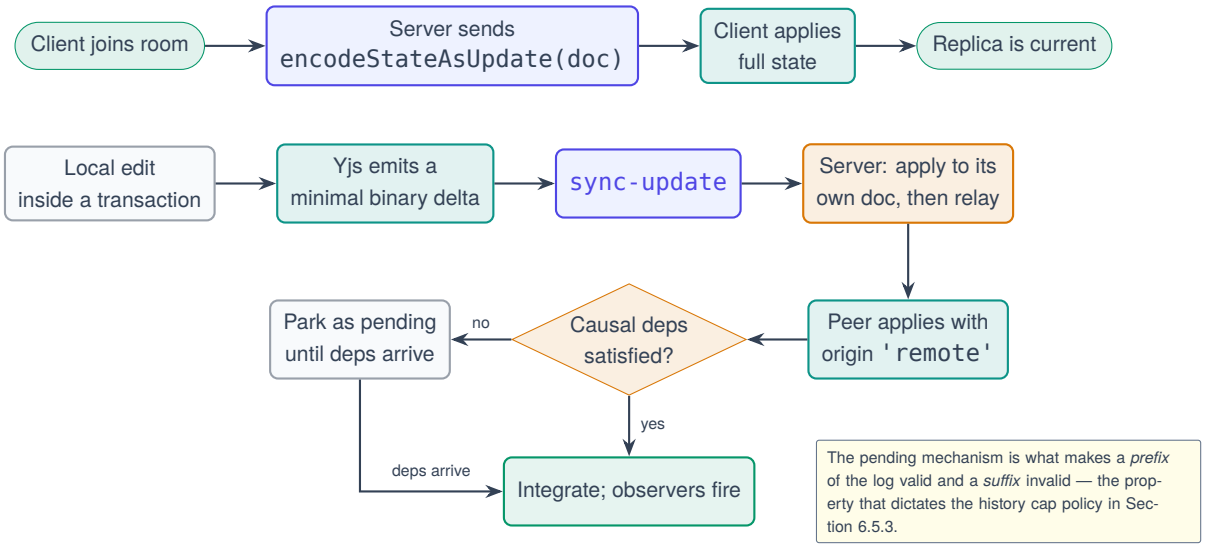


Figure 8.1: Yjs synchronisation workflow, including late-joiner state transfer

8.2 Session Replay Reconstruction

8.2.1 The Core Algorithm

Because a Yjs update is a self-contained, causally ordered delta, applying the first n updates of a log to an empty document reconstructs exactly the document that existed after update n — not an approximation, not a re-simulation, but the actual state. The entire reconstruction is therefore:

```
export function rebuildTo(entries, n) {
  const doc = new Y.Doc();
  const upto = Math.max(0, Math.min(n, entries.length));
  for (let i = 0; i < upto; i++) {
    const bytes = toBytes(entries[i]?.payload);
    if (!bytes) continue;
    try { Y.applyUpdate(doc, bytes, 'replay'); }
    catch (err) {
      // Losing one frame of history is far better than showing the user a
      // blank replay because one buffer arrived malformed.
      console.warn('[SyncSpace] replay: skipped a malformed update at', i, err);
    }
  }
  return doc;
}
```

Listing 8.1: Replay reconstruction — the complete idea

No diffing engine, no inverse operations and no snapshot ladder are required. Everything else in the replay subsystem is plumbing around these seven lines.

8.2.2 The Forward-Only Cache with Checkpoints

Calling `rebuildTo` on every slider tick costs $\Theta(n)$ per frame, which makes playback of a long session stutter. The observation that makes this cheap is that updates only ever move *forward*.

Table 8.2: Replay cache complexity by access pattern

Operation	Naive	Cache, no checkpoints	Cache with checkpoints
Step forward by one	$\Theta(n)$	$\Theta(1)$ amortised	$\Theta(1)$ amortised
Playback of n frames	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$
Jump backward to k	$\Theta(k)$	$\Theta(k)$	$\Theta(n/20)$ expected
Memory	$\Theta(1)$	$\Theta(1)$ document	$\Theta(1)$ plus $\leq$ 20 encoded snapshots

Checkpoints are encoded document states laid down every $\max(250, \lceil n/20 \rceil)$ updates, so a session carries at most roughly twenty of them regardless of its length, and a tiny session — which rebuilds instantly in any case — carries none.

```
at(n) {
  const target = Math.max(0, Math.min(n, this.entries.length));

  if (target < this.builtTo) {                // moving BACKWARD
    let base = 0, snap = null;
    for (const [k, v] of this.checkpoints) {
      if (k <= target && k > base) { base = k; snap = v; }
    }
    this.doc = new Y.Doc();
    this.builtTo = 0;
    if (snap) {
      Y.applyUpdate(this.doc, snap, 'replay');
      this.builtTo = base;                    // resume from the checkpoint
    }
  }

  for (let i = this.builtTo; i < target; i++) {   // then walk forward
    const bytes = toBytes(this.entries[i]?.payload);
    if (bytes) Y.applyUpdate(this.doc, bytes, 'replay');
    const applied = i + 1;
    if (applied % this.checkpointEvery === 0 && !this.checkpoints.has(applied)) {
      this.checkpoints.set(applied, Y.encodeStateAsUpdate(this.doc));
    }
  }
  this.builtTo = target;
  return this.doc;
}
```

Listing 8.2: Backward jump restoring from the nearest checkpoint

This is the one component that could silently corrupt replay, so it is proved rather than asserted: `test-replay.mjs` walks forward, backward and in random jumps, checking at *every index* that the cached document is byte-identical to a from-scratch rebuild.

8.2.3 Replay Reconstruction Flow

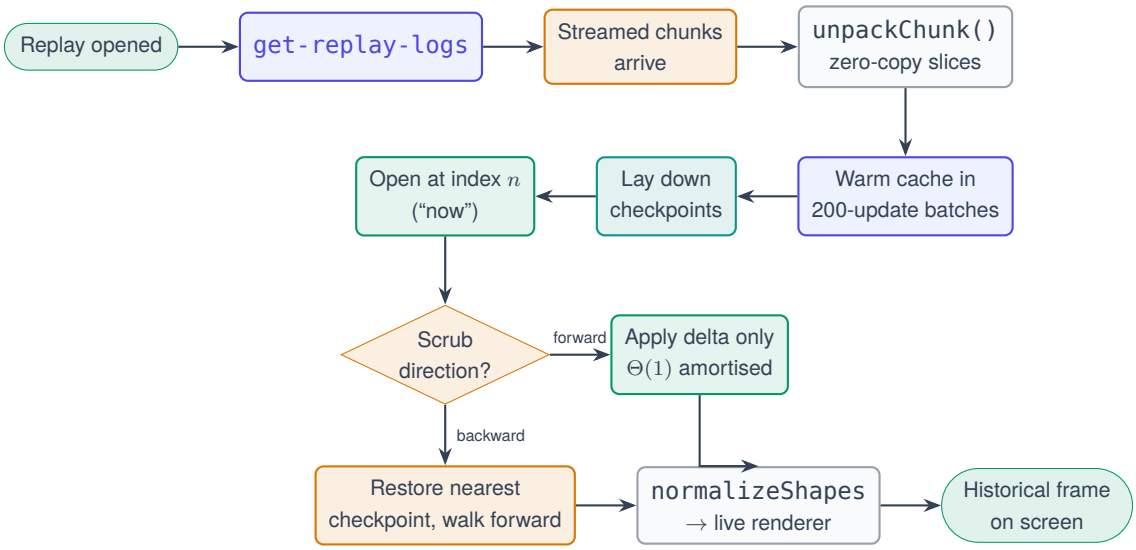


Figure 8.2: Replay reconstruction flow

8.3 The Replay Streaming Protocol

8.3.1 The Defect

The feature originally shipped the entire history in one emission, with each entry carrying its own buffer. It worked for sessions of about ten updates and failed silently for everything real: replay of any drawing past roughly fifteen seconds hung on “Loading session history” and then displayed a false “the server did not answer in time”.

The root cause was neither in recording, storage, ordering nor reconstruction. Socket.IO encodes *every* buffer nested anywhere within an emitted value as a separate binary attachment, and `socket.io-parser` version 4.2.5 and later — pulled in on both server and client — refuses any packet carrying more than ten attachments as a denial-of-service defence. Therefore:

- Three or four quick strokes gave ten or fewer entries, ten or fewer attachments, a packet that decoded, and replay that worked.
- Anything longer — and a drag alone emits roughly twenty-two updates per second — produced eleven or more attachments, so the *client’s* parser threw, the engine tore the connection down with a parse error, the response never arrived, and the client’s fixed fifteen-second timer blamed the server.

The server had answered every time. It answered unreadably. No increase in the timeout could ever have fixed it, which is why the resolution had to be a protocol change.

8.3.2 The Protocol

Table 8.3: The chunked replay transfer protocol

Message	Payload
`replay-logs-begin`	{ workspaceId, count, capped, skipped, meta: [{ seq, timestamp, username, size }] } — metadata only, *no* binary
`replay-logs-chunk`	{ workspaceId, start, count, sizes: [...], data: *one* packed Buffer } — repeated; closes at 400 entries or 512 KB
`replay-logs-end`	{ workspaceId, count }
`replay-logs-error`	{ workspaceId, code, message } — instead of any of the above

No message ever carries more than *one* binary attachment, regardless of session length. The acknowledgement now answers a summary rather than the payloads, because an acknowledgement is itself a single packet and would hit the identical ceiling.

```
let i = 0;
while (i < count) {
  const start = i, sizes = [], parts = [];
  let chunkBytes = 0;
  while (i < count && sizes.length < REPLAY_CHUNK_ENTRIES &&
         (chunkBytes === 0 || chunkBytes + rows[i].bytes.length <= REPLAY_CHUNK_BYTES)) {
    sizes.push(rows[i].bytes.length);
    parts.push(rows[i].bytes);
    chunkBytes += rows[i].bytes.length;
    i++;
  }
  if (!socket.connected) return;              // viewer left mid-transfer
  socket.emit('replay-logs-chunk', {
    workspaceId, start, count: sizes.length, sizes,
    data: Buffer.concat(parts)                 // exactly ONE attachment
  });
  // Yield between chunks so a huge history can never block the relay.
  await new Promise((r) => setImmediate(r));
}
```

Listing 8.3: Server-side chunk packing

```
export function unpackChunk(chunk) {
  if (!chunk || !Array.isArray(chunk.sizes)) return null;
  const data = toBytes(chunk.data) || new Uint8Array(0);
  const out = []; let off = 0;
  for (const s of chunk.sizes) {
    const size = Number(s);
    if (!Number.isInteger(size) || size < 0 || off + size > data.length) return null;
    out.push(data.subarray(off, off + size));   // zero-copy view
    off += size;
```

```
  }
  if (off !== data.length) return null;        // sizes disagree with the data
  return out;
}
```

Listing 8.4: Client-side unpacking with integrity validation

Returning `null` on inconsistency lets the caller report "corrupted replay data" rather than render a lie.

8.3.3 Three Supporting Hardenings

1. **The watchdog became an inactivity timer**, re-armed on every message. A long transfer making progress can therefore never falsely time out, and a genuinely stalled one fails saying exactly how much arrived ("stalled: 312 of 4,980 updates received"). The generic "server did not answer" text was removed entirely and replaced with distinct messages for storage read failure, corrupted chunk, incomplete history, interrupted connection and not-connected.
2. **Completeness resolves the transfer, not the end marker**, so a lost final packet degrades to the stall message instead of hanging.
3. **Stored payloads are normalised on the way out.** Memory-mode buffers and Mongo-mode BSON binary rows both ship correctly, and a single corrupt row is skipped and counted rather than aborting the whole fetch.

8.4 Update Coalescing

8.4.1 The Problem Measured

The original history ceiling was 5,000 entries. Measurement showed what actually fills it: dragging a shape emits a throttled position commit every 45 ms — about twenty-two updates per second. Under four minutes of ordinary dragging exhausted a room's entire replay history, on precisely the long collaborative sessions replay exists for.

8.4.2 The Solution

Raising the number alone would have traded one failure for another, so pressure was reduced from both ends.

Coalescing. Consecutive updates from the *same* user within a 400 ms window are merged into one entry using `Y.mergeUpdates`. A sixty-frame drag becomes a handful of entries. Different users are never merged, because attribution is displayed in the scrubber, and a deliberate pause keeps separate actions as separate replay steps. A merged entry is capped at 16 KB; without that cap one continuous drag would coalesce into a single ever-growing

blob rewritten on every frame — quadratic work, and a replay step so coarse the slider would jump across the whole gesture.

A two-sided budget. 50,000 entries *and* 32 MB, whichever is reached first. Neither a flood of tiny updates nor a few enormous ones can run away.

8.4.3 Why Merging is Safe

mergeUpdates(a, b) produces an update equivalent to applying a then b. A prefix of the merged log is therefore still exactly the document as it stood at that point — the one property on which the entire replay design rests. `test-updatelog.mjs` asserts directly that *every* prefix of a coalesced log is a valid document.

Table 8.4: Effect of coalescing on a representative session

Action	Raw updates	After coalescing	Reduction
One freehand stroke (commits on release)	1	1	—
One 3-second shape drag	$\approx$ 66	$\approx$ 8	$\approx$ 88%
Typing one line of code	$\approx$ 12	$\approx$ 3	$\approx$ 75%
Placing a shape	1	1	—
One eraser drag (atomic commit)	1	1	—

8.5 Connector Geometry

8.5.1 Edge Intersection

The `auto` anchor requires the point at which the ray from a shape's centre towards a target leaves the shape's real outline. The outline is sampled as a polygon — 24 points for circles and ellipses, 10 for stars, the exact vertex list for polygons via the shared `shapePoints` generator — and the segment–segment intersection with the greatest parameter t is selected, since that is the crossing furthest from the centre.

For a segment P_1P_2 and an edge P_3P_4, with $d = (P_{2x} - P_{1x})(P_{4y} - P_{3y}) - (P_{2y} - P_{1y})(P_{4x} - P_{3x})$, the intersection exists when $|d| \geq \epsilon$ and both parameters lie in $[0, 1]$. Cost is $\Theta(m)$ per endpoint for an m-vertex outline, with $m \leq 24$, so it is effectively constant.

8.5.2 Elbow Routing

```
export function displayPoints(conn, route) {
  if (conn.routing !== 'elbow') return route;
  const out = [route[0]];
  for (let i = 1; i < route.length; i++) {
    const p = out[out.length - 1], q = route[i];
    if (Math.abs(p.x - q.x) > 0.5 && Math.abs(p.y - q.y) > 0.5) {
      // alternate horizontal-first / vertical-first so chains of waypoints
      // give the H-V-H-V pattern instead of a staircase of same-direction Ls
      const horizontalFirst = i % 2 === 1;
      out.push(horizontalFirst ? { x: q.x, y: p.y } : { x: p.x, y: q.y });
    }
    out.push(q);
  }
  return out;
}
```

Listing 8.5: Axis-aligned elbow insertion with alternating orientation

8.5.3 Curved Routing

Curved connectors convert the route into cubic Bézier segments using a Catmull-Rom formulation with tension t taken from the curvature slider. For consecutive points $P_0 \dots P_3$, the control points of the segment $P_1 \to P_2$ are

$$C_1 = P_1 + \frac{(P_2 - P_0)}{6}\,2t, \qquad C_2 = P_2 - \frac{(P_3 - P_1)}{6}\,2t.$$

The curve passes through every user waypoint, which is the property a draw.io-style bend editor requires. Cost is $\Theta(k)$ for k route points.

8.6 Stroke Geometry

8.6.1 Ramer–Douglas–Peucker Simplification

RDP is applied at commit time with $\epsilon = 0.6$. The implementation is iterative rather than recursive, using an explicit stack and a `Uint8Array` keep-mask, so a pathological stroke cannot overflow the call stack.

```
export function simplify(flat, epsilon = 0.6) {
  const n = flat.length / 2;
  if (n < 3) return flat.slice();
  const keep = new Uint8Array(n);
  keep[0] = 1; keep[n - 1] = 1;
  const stack = [[0, n - 1]];
  while (stack.length) {
    const [a, b] = stack.pop();
    let maxD = -1, idx = -1;
    const ax = px(a), ay = py(a), bx = px(b), by = py(b);
```

```
    const dx = bx - ax, dy = by - ay;
    const len2 = dx * dx + dy * dy || 1;
    for (let i = a + 1; i < b; i++) {
      const t = ((px(i) - ax) * dx + (py(i) - ay) * dy) / len2;
      const cx = ax + t * dx, cy = ay + t * dy;
      const d = (px(i) - cx) ** 2 + (py(i) - cy) ** 2;
      if (d > maxD) { maxD = d; idx = i; }
    }
    if (maxD > epsilon * epsilon && idx > 0) {
      keep[idx] = 1;
      stack.push([a, idx], [idx, b]);
    }
  }
  /* collect kept points */
}
```

Listing 8.6: Iterative RDP with an explicit stack

Complexity is $\Theta(n \log n)$ expected and $\Theta(n^2)$ in the worst case, which is entirely acceptable because it runs once per stroke on pointer-up, not per frame. Note that distances are compared *squared* throughout, avoiding n square-root operations.

8.6.2 Chaikin Smoothing

One Chaikin pass replaces each interior vertex with two points at one quarter and three quarters along its adjacent edges, preserving the endpoints and converging towards a quadratic B-spline. It runs in $\Theta(n)$ at render time.

Ordering: simplify first, then smooth

The composition is `chaikin(simplify(points, 0.4), 1)`. Simplifying first yields a compact stored record — smaller documents, smaller network messages, cheaper redraws — that nonetheless renders smoothly, because Chaikin reintroduces the rounding at draw time from the compact representation. Doing it the other way round would double the point count *before* storage and achieve no additional visual quality.

8.6.3 The Calligraphy Ribbon

For a nib held at angle θ with unit direction $\mathbf{n} = (\cos\theta, \sin\theta)$ and unit stroke tangent $\mathbf{t}$ at a point, the half-width is

$$h = \frac{w}{2}\Big(r_{\min} + (1 - r_{\min})\,\big|\,t_x n_y - t_y n_x\,\big|\Big), \qquad r_{\min} = 0.18.$$

The cross-product magnitude is the sine of the angle between travel and the nib, so travel perpendicular to the nib gives maximum width and travel parallel gives the floor $r_{\min}$. With pressure enabled, a speed factor derived from point spacing relative to the stroke mean scales h further into $[0.45, 1.3]$. Left and right offsets are emitted along the normal and

joined — left edge forward, right edge backward — to form a closed polygon rendered as one filled Konva line. Cost is $\Theta(n)$.

8.6.4 The Eraser Sweep

Table 8.5: Eraser implementation comparison

	Stamped circles (original)	**Swept capsule (current)**
Region tested	Approximation of the swept area	The exact swept area
Cost per frame	$\Theta(s \cdot v)$, s = samples	$\Theta(v)$
Dependence on pointer speed	s grows with speed	None
Failure mode	Too few samples leave scalloped gaps	None
Fast flick vs. slow rub	Different results	Provably identical

The capsule test computes, for each vertex, the squared distance from the point to the drag segment and compares it against the squared radius — one distance test per vertex, no square roots. A bounding-box broad phase skips strokes entirely outside the affected region, and surviving runs are cached on the erase session so the render pass does not re-split every stroke on every frame.

8.7 Complexity Summary

Table 8.6: Time complexity of the principal algorithms

Operation	Complexity	Frequency	Notes
Apply one Yjs update	$\Theta(1)$ amortised	Per edit	YATA integration; $\Theta(\log n)$ worst case
Encode full state	$\Theta(n)$	Every 2 s (debounced)	n = document items
Snapshot read on shape change	$\Theta(n)$	Per document change	n = shapes; dominated by normalisation
Normalise one record	$\Theta(p)$	Per record	p = points in the record
Connector route (per connector)	$\Theta(m + k)$	Per frame	$m \leq 24$ outline samples, k waypoints
Elbow / Catmull-Rom tessellation	$\Theta(k)$	Per frame	k route points
Snap target search	$\Theta(S)$	Per pointer move while wiring	S = shapes; only while a connector tool is armed
RDP simplify	$\Theta(n \log n)$ exp.	Once per stroke	$\Theta(n^2)$ worst case; not on the frame path
Chaikin smooth	$\Theta(n)$	Per stroke render	One pass
Calligraphy ribbon	$\Theta(n)$	Per stroke render	
Eraser sweep	$\Theta(v)$	Per pointer move	After a bounding-box broad phase
Survive-runs split	$\Theta(v)$	Per erase commit	
Replay: forward step	$\Theta(1)$ amortised	Per playback frame	Cache applies the delta only
Replay: backward jump	$\Theta(n/20)$ expected	Per backward scrub	Restores from the nearest checkpoint
Replay: initial warm	$\Theta(n)$	Once, batched	200 updates per event-loop slice
Log append (coalescing)	$\Theta(b)$	Per relayed update	b = merged entry size, capped at 16 KB
Log read for replay	$\Theta(n)$	Once per replay open	Indexed by {roomId, seq}
Membership check	$\Theta(M)$	Per request / connect	M = members, embedded in one document

CHAPTER 9

Security, Validation and Performance

9.1 Security Architecture

9.1.1 Defence Layers

Security in SyncSpace is applied in seven layers, each of which assumes the previous one may have been bypassed.

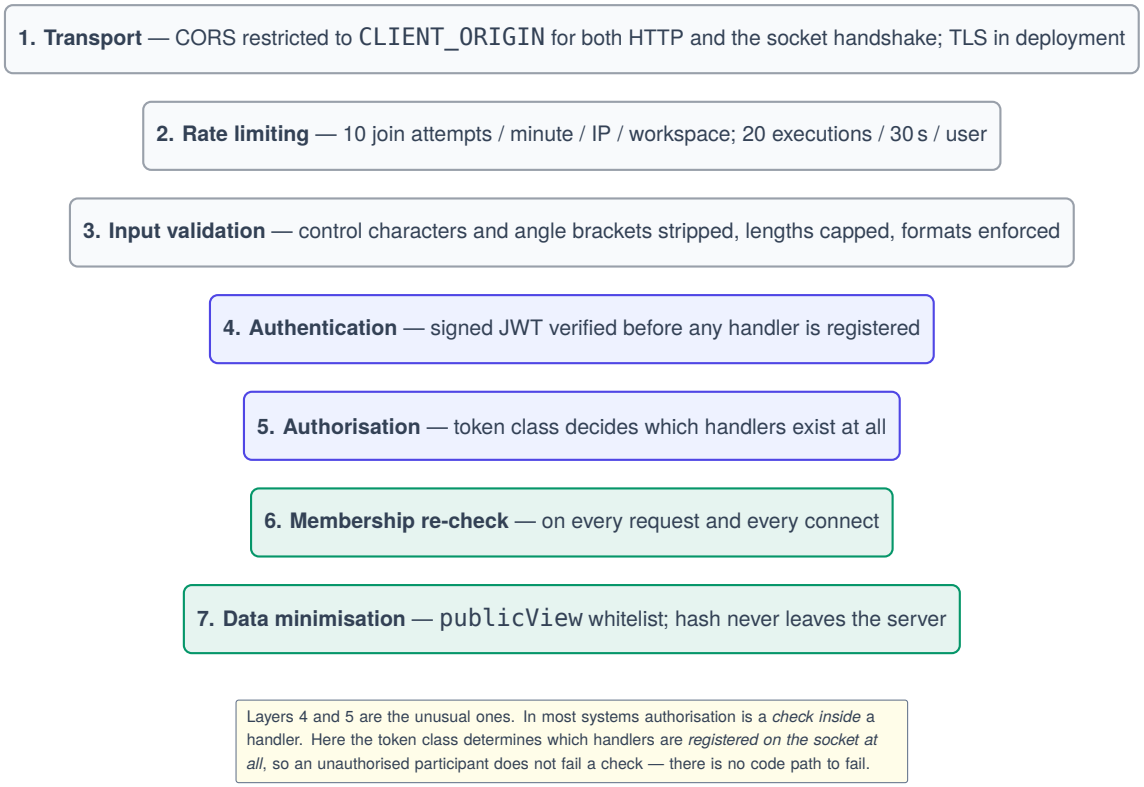


Figure 9.1: Defence-in-depth layers

9.1.2 Threat Model and Mitigations

Table 9.1: Threat model and implemented mitigations

Threat	Attack	Mitigation
Workspace enumeration	Probe identifiers to discover live workspaces	"No such workspace" and "wrong secret code" return an *identical* message and the peek endpoint deliberately withholds the access policy
Secret-code brute force	Automated guessing of a short code	Ten attempts per minute per IP per workspace; successful joins clear the bucket so a legitimate user is never penalised
Cross-workspace access	Send another workspace's identifier in an event	Impossible: the room is derived from the signed token, and no client-supplied room field exists in the protocol
Privilege escalation	A member emits `admin:approve`	Role is read from `socket.data`, populated at handshake from the signed token, never from the payload
Lobby data exfiltration	A waiting participant reads the document	No synchronisation, awareness or replay handler is registered for a lobby socket
History exfiltration	A waiting participant reads past state	`get-replay-logs` is registered inside the member branch, after the lobby branch has already returned
Stale-token access	A removed member reuses a valid token	Membership re-verified on every request and every connect; sockets force-disconnected on removal
Credential leakage	Password hash reaching the browser	`publicView` is a whitelist; the hash is never copied into any response
API key leakage	Execution provider key reaching the client	Keys are read only inside adapters; the browser receives results, never credentials. URLs are redacted before logging
Stored XSS	Script injected via a name or chat message	Control characters and angle brackets stripped at validation; React escapes all text nodes by default
Denial of service via payload	Enormous update or program	8 MB socket receive ceiling, 1 MB JSON body limit, 256 KB code limit, 64 KB stdin limit

9.1.3 Security Features Summary

Table 9.2: Implemented security features

Feature	Implementation	Property guaranteed
Password hashing	bcrypt, cost factor 10	Secret codes are never recoverable from the database
Token signing	HMAC via `jsonwebtoken`	Claims cannot be forged or modified
Token scoping	`workspaceId` inside the payload	A token for one workspace cannot authorise another
Token classes	`access` (12 h) and `lobby` (2 h)	Waiting is a genuinely distinct authorisation state
Handshake gate	`io.use()` before any handler	An unauthenticated socket is refused outright
Session storage	`sessionStorage`, not `localStorage`	Tokens are tab-scoped, enabling two-user demonstration and reducing exposure
Uniform error text	Identical message for two failure causes	The join endpoint is not an identifier oracle
Rate-limit sweeping	Expired buckets dropped every 60 s	A long-lived server does not accumulate keys
Reason-bounded input	Rejection reasons truncated to 140 characters	An administrator cannot inject an unbounded string

9.2 Validation

Every value arriving from a browser passes through `utils/validate.js` before reaching any store.

```
/** Strip control chars + angle brackets, collapse whitespace, cap length. */
export function clean(value, max = 200) {
  if (typeof value !== 'string') return '';
  return value.replace(/[\u0000-\u001F\u007F<>]/g, '').trim().slice(0, max);
}
const USERNAME_RE = /^[A-Za-z0-9 ._-]{2,24}$/;
export function validateWorkspaceId(raw) {
  const workspaceId = clean(raw, 20).toUpperCase();
  if (!/^WS-[A-Z0-9]{4,12}$/.test(workspaceId))
    return { ok: false, message: 'That does not look like a valid workspace ID.' };
  return { ok: true, workspaceId };
}
```

Listing 9.1: Input sanitisation and format validation

Table 9.3: Validation rules

Field	Rule	Rationale
Username	2–24 characters; letters, digits, spaces, dots, hyphens, underscores	Bounded and typographically safe; excludes markup characters entirely
Workspace name	Minimum 3 characters, maximum 60	Meaningful yet bounded for display
Secret code	4–128 characters, any content	Length-bounded to prevent a bcrypt denial-of-service via a megabyte password
Access policy	Exactly `permission` or `password`	Closed enumeration; anything else is rejected
Workspace ID	`WS-` followed by 4–12 upper-case alphanumerics	Normalised to upper case, so a lower-case paste still works
Language	Must exist in the registry	Prevents an arbitrary identifier reaching a provider
Code	String, maximum 256 KB	Bounded submission size
Stdin	String, maximum 64 KB	Clipped with a warning rather than rejected
Chat message	Maximum 2,000 characters; history capped at 100	Bounds both message and document growth

Client-supplied *document* content is validated differently, because it arrives as opaque CRDT bytes that the server deliberately does not interpret. The defence there is `normalize.js` on the reading side: every record is coerced to drawable values before it can reach a renderer, which converts a malformed or hostile record from a rendering failure into a harmlessly defaulted object.

9.3 Error Handling

9.3.1 Principles

Three principles govern error handling throughout the system.

1. **An error message must name the actual failure.** The generic “the server did not answer” text was removed from the replay subsystem specifically because it was false and misdirected debugging for a considerable time.
2. **A bonus feature must fail like a bonus feature.** `appendUpdate` swallows its own errors and returns `null`; a broken log degrades replay and never stalls live collabora-

tion.

3. **Failure is data, not an exception, where the user must see it.** `executeCode` always resolves with the canonical result shape, so the editor always has something meaningful to render.

Table 9.4: Error handling by subsystem

Subsystem	Failure	Behaviour
Body parsing	Malformed JSON	400 with "That request was not valid JSON" — not a 500
Body parsing	Oversized body	413 naming the 1 MB limit
REST, general	Unhandled exception	Real stack logged server-side; one neutral sentence returned
Socket handshake	Any of five causes	A distinct error code, mapped client-side to a plain-English sentence
Document read	Corrupt shape array	Last good snapshot retained; the board is not torn down
Shape render	One malformed record	`ShapeErrorBoundary` isolates it; the rest of the board renders
Connector routing	Non-finite route	Falls back to the endpoints' cached coordinates so the line still draws
Persistence	Snapshot write failure	Warning logged; collaboration unaffected
Update log	Append failure	Swallowed; returns `null`; replay degrades only
Replay transport	Read failure, corrupt chunk, incomplete history, dropped connection	Four distinct messages, each stating what was actually received
Execution	Provider unreachable	Falls through the chain; final message lists each provider and its reason
Execution	Missing toolchain	A human sentence, never a crash
Client fetch	Timeout	Deadline on every request; "the server took too long" rather than a permanently spinning button

9.3.2 A Worked Example of Error-Message Quality

The replay failure path illustrates the principle concretely. The original implementation had one message. The current implementation distinguishes:

- **Not connected** — "Not connected to this workspace."
- **No response begun** — "The server did not start sending the session history."
- **Stalled mid-transfer** — "The history transfer stalled (312 of 4,980 updates received)."
- **Corrupt chunk** — "The session history arrived corrupted and could not be parsed."
- **Incomplete** — "The session history arrived incomplete (312 of 4,980 updates)."
- **Interrupted** — "The connection was interrupted while loading the session history."
- **Storage failure** — carried from the server with the code `DB_READ_FAILED`.

Each of these points at a different component. The single generic message pointed at the wrong one.

9.4 Performance Optimisations

Table 9.5: Performance optimisations and their effect

Optimisation	Mechanism	Effect
Stroke commit on release	The in-progress stroke lives in local state and awareness; written to the document once	One network message and one undo step per stroke instead of dozens
Memoised shape nodes	`React.memo` on `ShapeNode`	During a draw, hundreds of existing nodes are skipped; only the preview repaints
Konva render flags	`perfectDrawEnabled` and `shadowForStrokeEnabled` disabled	Thousands of strokes stay inexpensive
Throttled live drags	45 ms commits under the `'live'` origin	Peers follow smoothly without one message per frame, and the drag remains one undo step
Derived connector geometry	Endpoints computed at render time	Moving a shape re-routes every attached arrow with *zero* network traffic

Continued on next page

Table 9.5 – continued from previous page

Optimisation	**Mechanism**	**Effect**
Indexed shape lookup	`shapesById` `Map` rebuilt per snapshot	$\Theta(1)$ resolution instead of a linear scan per connector endpoint
Memoised route passes	`useMemo` over routes and live positions	Routes recomputed only when inputs change
RDP simplification	Applied at commit, $\epsilon = 0.6$	Smaller records, smaller messages, faster redraws, compact undo history
Eraser broad phase	Bounding-box rejection before per-vertex tests	Distant strokes skipped entirely
Swept-capsule erasure	Exact swept region, one test per vertex	Frame cost independent of pointer speed
Cached survive-runs	Computed once per erase session	Render pass does not re-split every stroke every frame
Debounced snapshots	2 s coalescing window	At most one database write per room per two seconds regardless of activity
Update coalescing	`Y.mergeUpdates` within 400 ms per user	An 88% reduction in log entries for drag-heavy sessions
Chunked replay stream	400 entries or 512 KB per message	Bounded memory and a real progress indicator
Event-loop yield	`setImmediate` between chunks	Serving a long history cannot starve live collaboration
Batched cache warm	200 updates per `setTimeout(0)` slice	Opening a long replay cannot freeze the tab
Replay checkpoints	≤ 20 encoded snapshots per session	Backward scrub becomes a short delta, not a full rebuild
Provider probe cache	5-minute time-to-live	The language catalogue answers instantly on editor mount
Execution queue	FIFO, 8 concurrent, 32 queued	Protects the socket loop and the provider's rate limit

9.4.1 Measured Results

The replay transport was measured directly because it is the subsystem whose performance had previously failed. A capped session of 5,000 updates, including interleaved 300 KB image payloads:

Table 9.6: Measured replay transport and reconstruction performance

Metric	Measured	Comment
Complete history streamed	$\approx$ 43 ms	Across 17 bounded chunks
Full reconstruction	$\approx$ 175 ms	Byte-for-byte identical to the live document
Chunks required	17	Each carrying exactly one binary attachment
Maximum attachments per packet	1	Against a hard ceiling of 10
Live collaboration during transfer	Unaffected	Asserted by a dedicated check
Log entries for a 3-second drag	$\approx$ 8 (from $\approx$ 66)	Coalescing at a 400 ms window

9.4.2 Known Performance Weakness

One weakness is stated openly. **Cursor emissions are not throttled.** The canvas writes the cursor position to awareness on every pointer move. With two or three participants this is invisible; with six or more it will produce noticeable jitter. The remedy is already proven in the codebase — shape drags are throttled at 45 ms using a two-line pattern — and applying the identical pattern to cursors is a small, well-understood change. It is documented here rather than concealed because the mitigation is known and the trade-off was deliberate: awareness updates are ephemeral and never persisted, so the cost is bandwidth and repaint frequency, not correctness.

9.5 Deployment Security Checklist

Table 9.7: Pre-deployment security checklist

Item	Requirement
`JWT_SECRET`	Replace the development placeholder with a long random value. Failing to do so makes every token forgeable
`CLIENT_ORIGIN`	Must exactly match the deployed frontend URL, or both CORS and the socket handshake fail
TLS	Terminate HTTPS and WSS at the host or proxy; `trust proxy` is already enabled
`MONGO_URI`	Use a scoped database user, not an administrative account
Execution provider	Configure Judge0 with a key; the no-signup fallback is best-effort and rate-limited
`EXEC_ALLOW_LOCAL`	Must remain unset. Local execution is unsandboxed and is a development escape hatch only
Rate limits	Review `EXEC_RATE_MAX` against expected class size
Logging	`EXEC_LOG` redacts keys in URLs, but review the host's log retention

CHAPTER 10

Testing Methodology and Results

10.1 Testing Philosophy

The testing strategy follows a single organising rule stated in Section 3.6 and applied consistently: **the parts of the system that can actually be wrong import nothing that requires a browser**. The connector geometry, the brush and eraser mathematics, the record normalisation and the replay reconstruction each live in modules whose only dependency is Yjs or nothing at all. Every one of them is therefore tested headlessly against the *genuinely shipped source*, bundled with esbuild — not against a reimplementation written for the test, which is the failure mode that makes a green suite meaningless.

Three further commitments shape the suites:

1. **Test the property, not the implementation detail.** Two backend assertions originally counted log *entries* — exactly the quantity that coalescing reduces by design. They were rewritten to assert reconstruction *fidelity*, which is the property that actually matters and which coalescing must not disturb.
2. **Test sandboxes with real processes.** The execution suite runs against a mock that speaks each provider's real wire format and, underneath, genuinely compiles and runs the submitted program. The adapters are therefore tested against real GCC diagnostics, real signals, real exit codes and real Unicode rather than canned fixtures.
3. **Assert the guarantee, not the happy path.** The isolation suite does not merely check that a member can read their own workspace; it checks that another workspace's token is refused, that a lobby socket cannot read the document or the history, and that a member emitting an administrator event receives an error.

10.2 Test Architecture

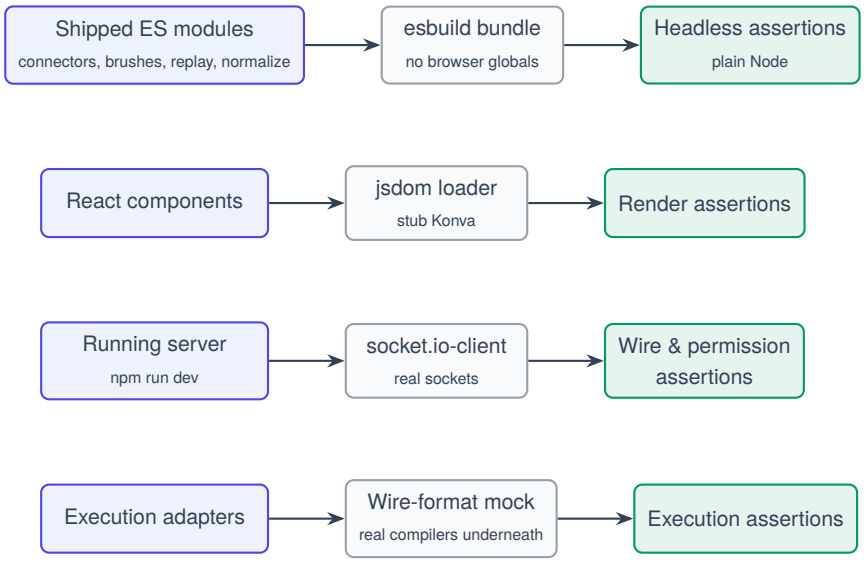


Figure 10.1: Test architecture — four execution modes

10.3 Unit Testing

Unit tests cover the pure modules: geometry, stroke mathematics, record normalisation and reconstruction.

Table 10.1: Unit test coverage by module

Suite	Checks	Properties asserted
`test-connectors`	28	Anchor points; rotated anchors; edge intersection on circles and diamonds rather than bounding boxes; auto-routing; endpoints following moved shapes; cached fallback after deletion; waypoint ordering; axis-aligned elbows; spline segment counts; snapping priorities and exclusions; bend insertion; cross-client connector synchronisation; live-origin writes invisible to undo; duplicate re-wiring and detachment; layer reordering
`test-brushes`	43	Dash scaling with width; RDP simplification; Chaikin smoothing; calligraphy nib swell; eraser vertex marking; stroke splitting (middle gives two runs, end gives one, whole gives none, distant gives no change); densification; “a fast flick erases exactly what a slow drag does”
`test-shapes`	11	Field-level CRDT merge; two clients editing the same shape (one recolouring, one moving) both survive; late-joiner receives the whole canvas
`test-replay` (frontend)	38	Full and partial reconstruction; a prefix shows a shape at its position *before* a later move and an object that was *later deleted*; cache invariant forward, backward and in random jumps; index clamping; `toBytes` across all four wire formats; malformed entry skipped rather than fatal; empty logs; frame bounds; images and stickers replay with their source intact; `unpackChunk` splits, validates and rejects the streamed format; checkpoints on a 700-update log byte-identical to a from-scratch rebuild
`test-rendering`	156	Component rendering under jsdom: gradient and solid fill precedence, blur caching, corner-radius gating, rotation about the centre, multi-select property derivation, border style derivation from both fields, panel layout constraints
`test-updatelog`	16	Coalescing correctness; *every prefix* of a coalesced log is a valid document; sequence ordering under rapid append; two-sided budget engagement; tail-trim policy

10.4 Integration Testing

Integration suites run against a genuinely running server over real sockets.

Table 10.2: Integration test coverage

Suite	Checks	Properties asserted
`test-workspace`	15	The whole permission system over live sockets: creation, both join policies, waiting room, approval pushing a token down the lobby socket, rejection, policy switching at runtime, member removal with forced disconnect, privilege escalation blocked, room isolation, handshake refusal without a token
`test-replay` (backend)	27	Every update logged; the streamed events answer and the acknowledgement returns a summary; ordering, timestamps and attribution; full replay reproduces the live document exactly; a prefix is a true intermediate state; the editor replays too; workspace isolation in both directions; a lobby socket is refused; live synchronisation unaffected by logging; a late joiner's edits are logged; the long-session regression block — 400+ mixed updates stream back with no parse error, complete, sequence-ordered, and live synchronisation survives serving them
`test-execute`	100	Per language (all five) and per provider (all three): correct runtime invoked, stdin with and without trailing newline, multiline stdin, Unicode and emoji round trip, compile errors landing in the compile phase with real compiler text, runtime errors with correct signal and plain-English reason, non-zero exit codes reported honestly, output floods capped, infinite loops timed out, ten simultaneous runs from different workspaces staying isolated, five languages at once with no leaked state; plus fault injection — 503 retried, persistent 429 falling through, malformed JSON rejected, polling fallback, total outage producing one readable message

10.5 Stress Testing

`stress-replay.mjs` drives a room past five thousand updates with 300 KB image payloads interleaved, and verifies that the transfer remains chunk-bounded, complete, correctly ordered and fast. It is the suite that directly guards against a recurrence of the attachment-ceiling defect, because that defect was invisible at every scale a functional test would naturally use.

Table 10.3: Stress test results

Scenario	Result	Observation
5,000 updates with 300 KB images	Pass	Streams complete in ≈ 43 ms across 17 chunks
Reconstruction of the same session	Pass	Byte-for-byte identical in ≈ 175 ms
5,206-update session after coalescing	Pass	Records in full instead of capping, where the old flat 5,000 limit engaged
Live synchronisation during history service	Pass	Unaffected, owing to the event-loop yield between chunks
Cap engagement when genuinely reached	Pass	Recording stops, `capped` flag set, tail trimmed — never the head
100 MB print flood from executed code	Pass	Truncated at 64 KB per stream
Infinite loop submission	Pass	Killed at the wall-clock limit
Six-way parallel execution burst	Pass	Serialised correctly through the FIFO queue

10.6 Security Testing

Table 10.4: Security test cases and outcomes

Test case	Expected	Actual	Verdict
Socket connect with no token	Handshake refused	`AUTH_REQUIRED`	Pass
Socket connect with a forged token	Handshake refused	`AUTH_INVALID`	Pass
Workspace A token against workspace B	403 Forbidden	403	Pass
Removed member reconnects	Handshake refused	`NOT_A_MEMBER`	Pass
Lobby socket emits `sync-update`	No handler exists	Silently ignored	Pass
Lobby socket requests replay history	No handler exists	Silently ignored	Pass
Member emits `admin:approve`	Error acknowledgement	“Only the administrator can do that”	Pass
Eleven join attempts in one minute	429 with Retry-After	429	Pass
Twenty-first execution in 30 seconds	429	429	Pass
Unknown workspace vs. wrong code	Identical message	Identical	Pass
Response inspected for `passwordHash`	Absent	Absent	Pass
Response inspected for provider key	Absent	Absent	Pass
Angle brackets in a username	Stripped at validation	Stripped	Pass
2 MB program submitted	413 with a stated limit	413	Pass

10.7 Consolidated Test Results

Table 10.5: Consolidated test suite results

Suite	Checks	Passed	Failed	Mode
`frontend/test-rendering`	156	156	0	jsdom, stubbed Konva
`frontend/test-brushes`	43	43	0	Headless, esbuild-bundled
`frontend/test-replay`	38	38	0	Headless
`frontend/test-connectors`	28	28	0	Headless, esbuild-bundled
`frontend/test-shapes`	11	11	0	Headless
`backend/test-execute`	100	100	0	Wire-format mock, real processes
`backend/test-replay`	27	27	0	Live sockets
`backend/test-updatelog`	16	16	0	Headless, no server
`backend/test-workspace`	15	15	0	Live sockets
`backend/stress-replay`	8	8	0	Live sockets, long session
Total	**442**	**442**	**0**	
`vite build`	—	Clean	—	Production bundle

Table 10.6: Growth of the test suite across development milestones

Milestone	Total checks	Added
Synchronisation core	26	Shapes and workspace permissions
Connectors and executable IDE	105	Connector geometry, execution, HTTP end-to-end
Brushes, eraser and undo	105	Brush and eraser suite
Session replay	287	Replay wire protocol, reconstruction, stress
Quality pass	442	Rendering suite repaired and expanded, update-log suite added, execution suite broadened across providers

10.8 Defects Found by Testing

The following defects were found by the automated suites before any user encountered them. They are listed because the value of a test suite is measured by what it catches, not by its size.

Table 10.7: Defects discovered by the test suites

Defect	Symptom	Resolution
Duplicate sequence numbers	Two rapid updates could be logged under the same `seq`, corrupting the replay axis on long sessions	Seeding moved behind a shared promise; allocation made a synchronous read-and-increment
Address-space limit killing managed runtimes	V8 and the JVM terminated at startup under `ulimit -v` because they reserve gigabytes of virtual address space	Managed runtimes capped by their own flags instead; native code retains the virtual-memory cap
Shell rejecting a process limit	`ulimit -u` unsupported in the invoking shell	Detected and handled rather than crashing
Gradient fills never rendering	Linear and radial were dead controls: the button highlighted, the record updated, the document synced, and the canvas never changed	Paint ownership consolidated; `fillPriority` set explicitly rather than relying on library fallback ordering
Blur slider inert	A filter does nothing until the node is cached, so no pixel ever changed at any blur value	`useBlurFilter` hook with a merged ref, caching with padding, and cache clearing at zero
Image nodes filled with flat colour	A refactor routed images through the unified paint, covering every uploaded photograph with the schema's default fill	Images take shadow only
Layer order destroyed by selection	Sending a shape backward then clicking it returned it to the top, making the arrange controls appear broken	`bringToFront` on selection removed entirely
Undefined written into the document	Yjs stored `undefined` as a real value, leaving a dead key that defeated every default	The field is deleted instead
Stale test API	20 pre-existing rendering failures came from a test passing a long-removed prop	Test updated to the real API, plus new multi-select coverage

10.9 Limitations of the Testing Performed

Stated honestly, the following were *not* covered and should not be considered verified:

- **Live interaction testing.** Clicking through every control, first-click responsiveness and hover, active and disabled states in a real browser were not systematically exercised.
- **Visual quality assurance.** Layout at multiple window widths, zoom levels, small and large screens and high-DPI displays was not verified. The stylesheet changes made during the quality pass are reasoned but visually unconfirmed.
- **Multi-participant field testing.** Sessions with several genuine concurrent human users, as opposed to programmatically driven clients, were not conducted.
- **Live provider calls.** The execution suite runs against a mock speaking the real wire formats; a call to a live Judge0 or paiza.io endpoint was not made from the development environment. The wire formats are implemented from the official API documentation.
- **Network resilience.** Dropping the connection, continuing to draw offline and reconnecting is expected to merge correctly by Yjs's design, but was not tested.
- **Monaco feature verification.** The editor was read through and no defect was identified, but its features were not individually exercised.

Declaring these openly is itself a testing decision. A suite that reports 442 green assertions invites the reader to assume more coverage than exists; naming the gaps prevents that.

CHAPTER 11

User Interface and Results

11.1 How to Use This Chapter

Each section below documents one screen: what it presents, why it is arranged as it is, and what the reader should observe in the corresponding capture. Every figure provides a dashed placeholder sized for the real screenshot. To insert a capture, replace the `\screenshotslot` command in the figure with an `\includegraphics` directive, for example:

```
% Before:
\screenshotslot{13}{7}

% After:
\includegraphics[width=0.92\textwidth]{figures/landing-page.png}
```

Listing 11.1: Replacing a placeholder with a real capture

Alongside each placeholder, an annotated wireframe records the intended layout and labels the interface elements referenced in the surrounding discussion, so the report remains self-explanatory even before captures are pasted in.

11.2 Landing Page

The landing page offers exactly two doors: create a workspace or join one. An earlier version also allowed a participant to type a workspace identifier and enter directly; that path was removed, because type-an-identifier-to-enter is precisely the behaviour the permission system exists to replace.

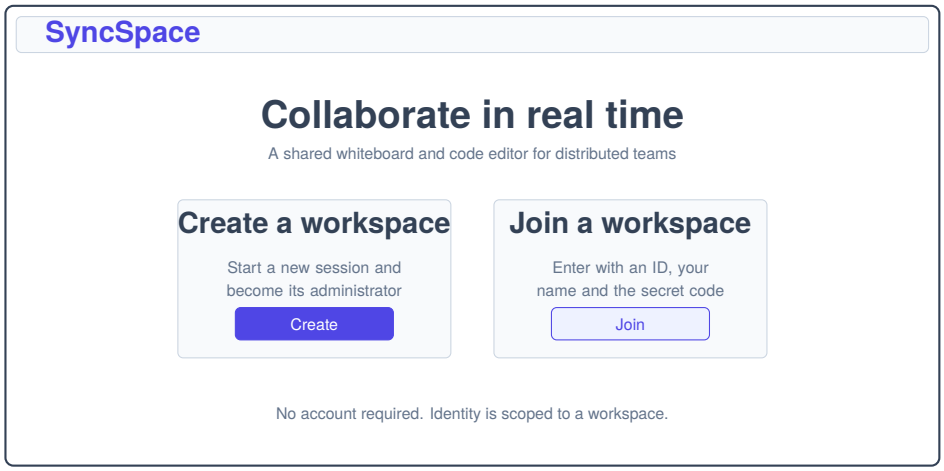


Figure 11.1: Landing page — annotated wireframe

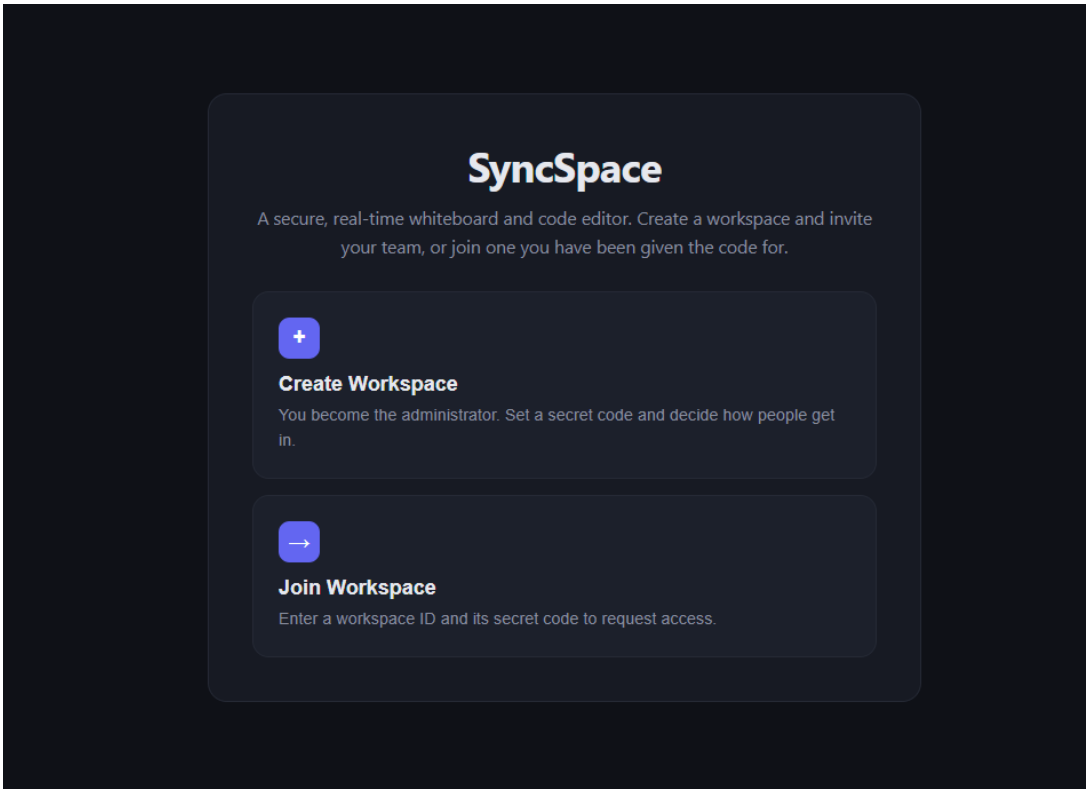


Figure 11.2: Landing page — the two entry points presented to a new visitor

11.3 Workspace Creation

Creation collects four values: a workspace name, the administrator's display name, a secret code and the initial access policy. The policy control is presented at creation time rather than buried in settings, because it is the single most consequential decision about the session and because it remains changeable at runtime afterwards.

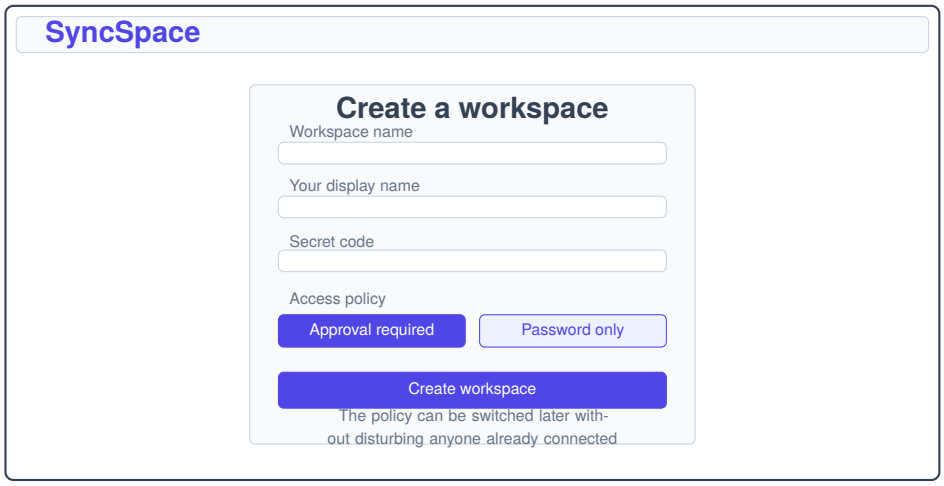


Figure 11.3: Workspace creation — annotated wireframe

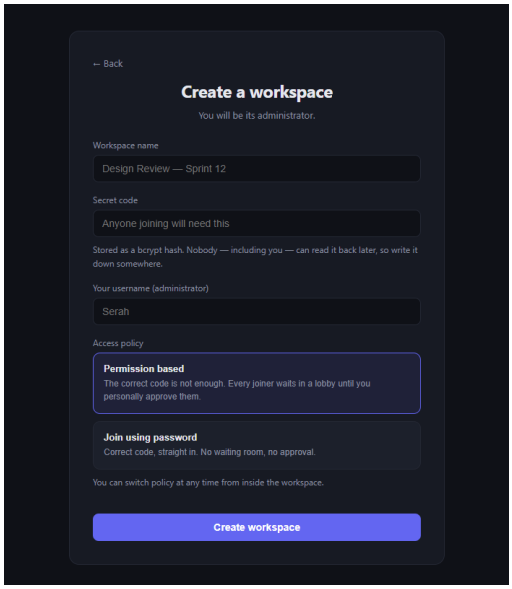


Figure 11.4: Workspace creation form, showing the two selectable access policies

On success the administrator is routed into the workspace and a notification displays the generated identifier — formatted `WS-XXXXXX` from an alphabet that omits the visually ambiguous characters, because the identifier is routinely read aloud over a call.

11.4 Waiting Room

A participant joining a permission-mode workspace is held here. The screen is not a polling loop: the client holds an open socket carrying a *lobby ticket*, and the administrator's approval pushes an access token down that socket directly. Refreshing the page is safe, because the server replays the current request status on reconnection rather than leaving the participant on an indefinite spinner.

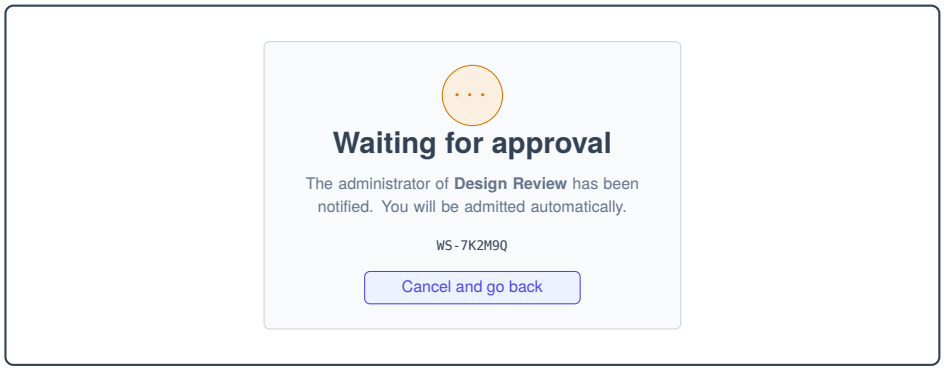


Figure 11.5: Waiting room — annotated wireframe

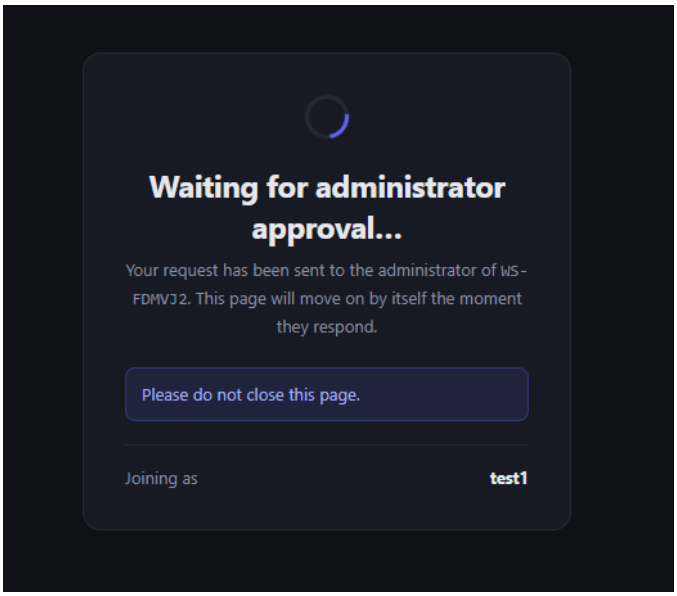


Figure 11.6: Waiting room — a lobby socket with no document handlers registered

The security property to observe here is negative rather than visible: this socket has *no* synchronisation, awareness or replay handler registered. The participant cannot read the document, enumerate peers or request history, and this required no additional checks — the handlers simply do not exist on that connection.

11.5 The Collaborative Workspace

This is the principal screen. A top bar carries identity, presence and global actions; the body is a two-pane split with the whiteboard on the left and the code editor on the right; contextual panels overlay as needed.

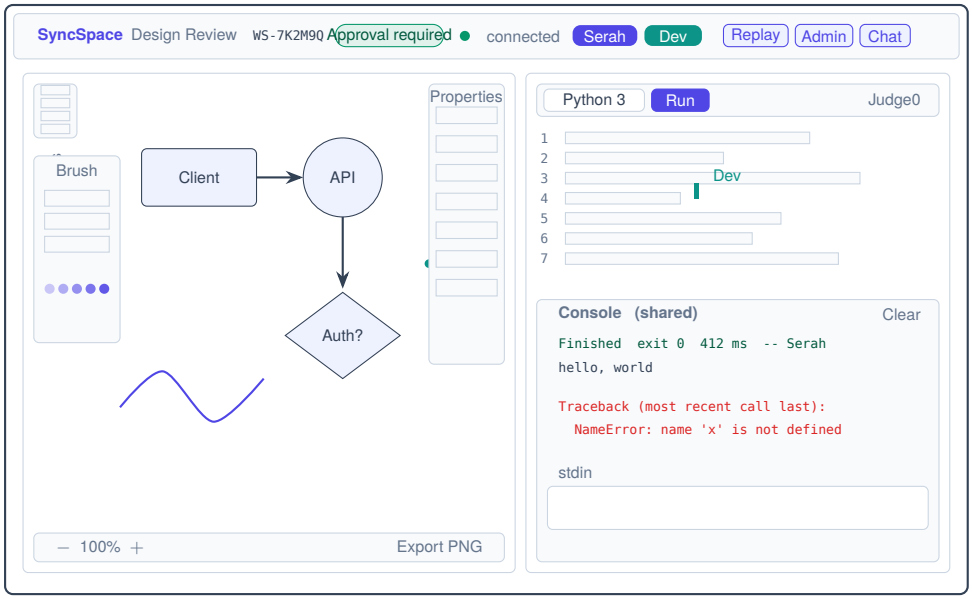


Figure 11.7: Collaborative workspace — annotated wireframe of the two-pane shell

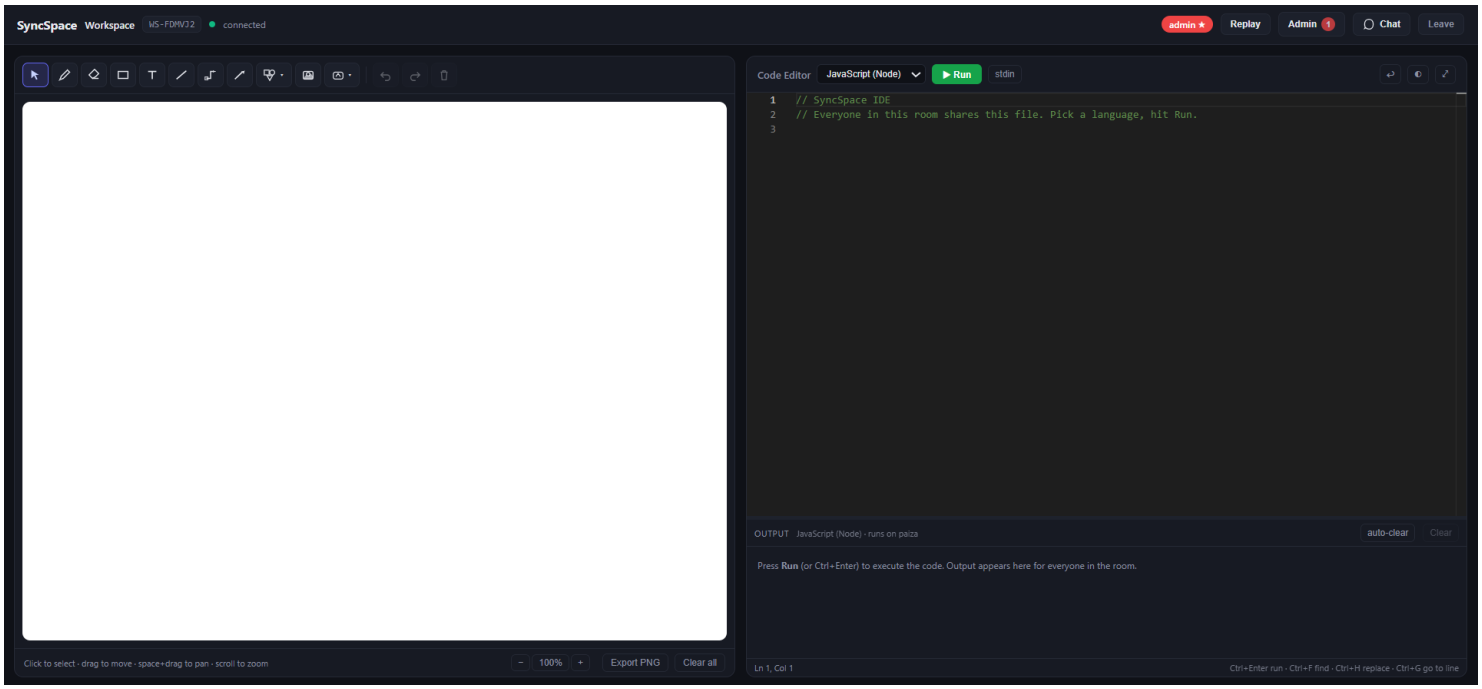


Figure 11.8: The collaborative workspace: whiteboard on the left, executable code editor on the right, with peer presence chips in the top bar

11.5.1 Elements to Observe

Table 11.1: Workspace interface elements

Element	Location	What it conveys
Workspace name and ID	Top bar, left	The identifier to share; rendered monospaced so it can be read aloud accurately
Policy chip	Top bar	The current access policy, visible to every member rather than to the administrator alone
Connection dot	Top bar	Green when the socket is live, amber while reconnecting
Peer chips	Top bar	One per connected participant, in their deterministic awareness colour; a star marks the administrator
Replay button	Top bar	Disabled while disconnected, because history cannot be fetched without a socket
Admin button	Top bar	Visible only to the administrator; carries a pulsing badge when requests are pending
Chat button	Top bar	Unread count when the panel is closed
Toolbar	Canvas, left edge	Select, pen, eraser, text, shapes menu, connector, arrow, undo, redo
Brush panel	Floating, top-left	Appears only while the pen or eraser is active, anchored away from the property panel
Property panel	Canvas, right edge	Contextual to the selection; empty when nothing is selected
Canvas footer	Canvas, bottom	Zoom controls and PNG export
Language dropdown	Editor, top	Shared: changing it switches every participant's syntax mode
Provider label	Editor, top-right	Names the sandbox actually executing code
Console	Editor, bottom	Shared and attributed; colour-coded by stream
Stdin panel	Editor, bottom	Collapsible and deliberately *local* to each participant

11.6 Multi-User Collaboration

The capture below should be taken with two browser windows side by side, signed in as two different participants — which the use of `sessionStorage` rather than `localStorage` makes possible in one browser profile.

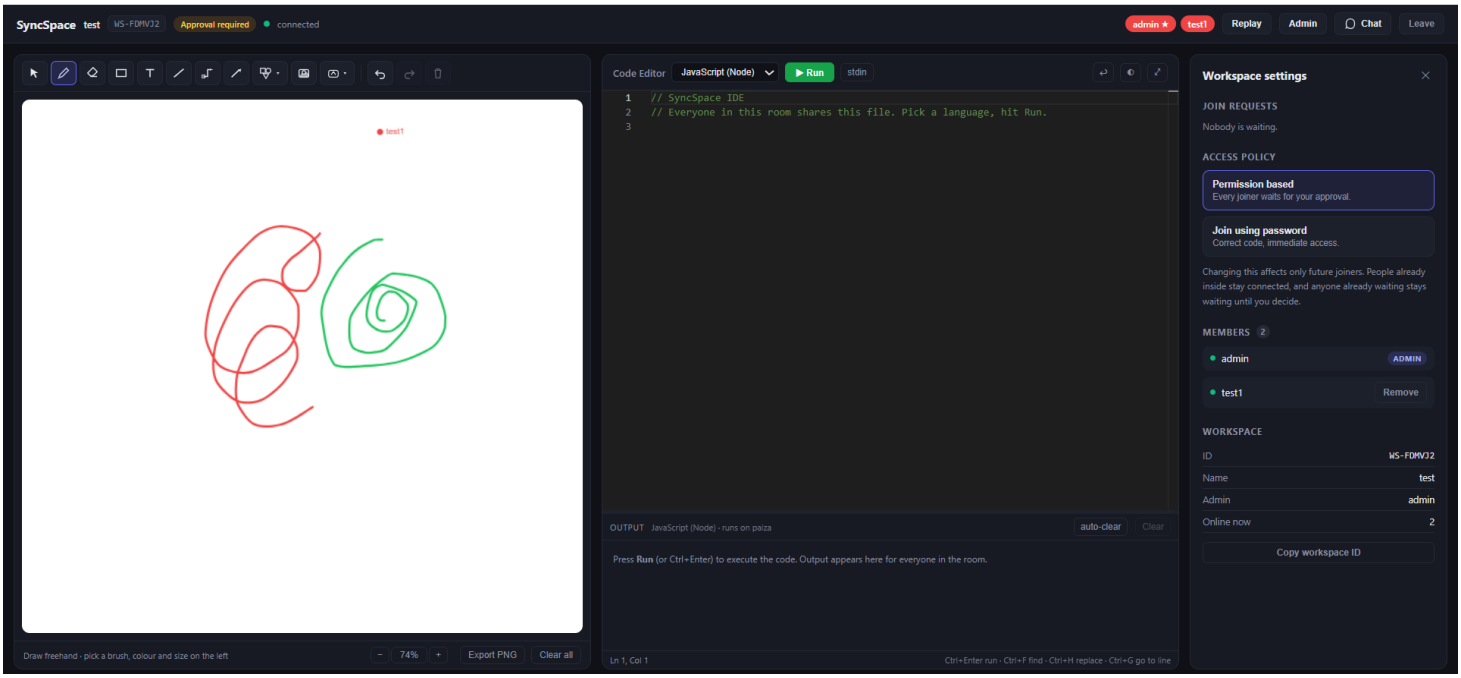


Figure 11.9: Two participants in one workspace: named remote cursors, coloured remote selection boxes, and a connector re-routing live as the attached shape is dragged in the other window

Three behaviours are worth capturing specifically, because each demonstrates a distinct architectural claim:

1. **A named cursor and a coloured selection box** for the remote participant — the awareness channel operating independently of the document.
2. **A shape dragged in one window gliding in the other**, with every attached arrow re-routing frame by frame — the throttled live-origin commits combined with derived connector geometry.
3. **The whole drag undoing as a single step** — the untracked `'live'` origin excluded from the undo stack.

11.7 Administrator Panel

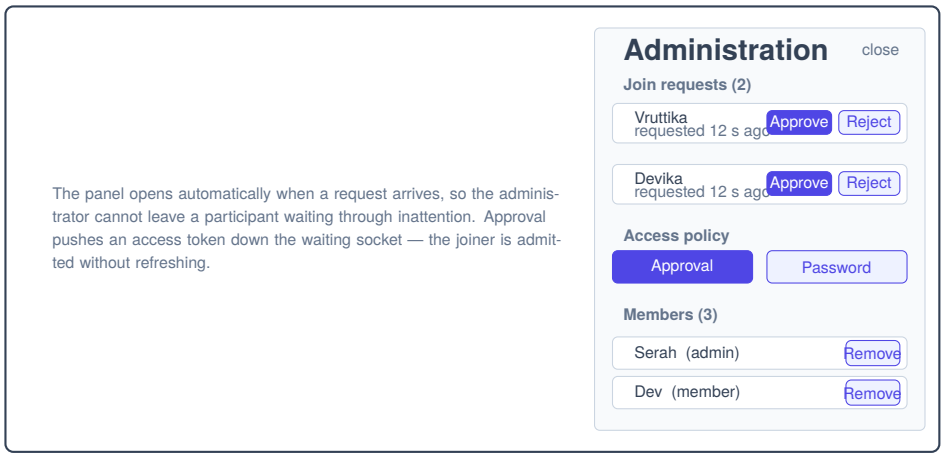


Figure 11.10: Administrator panel — annotated wireframe

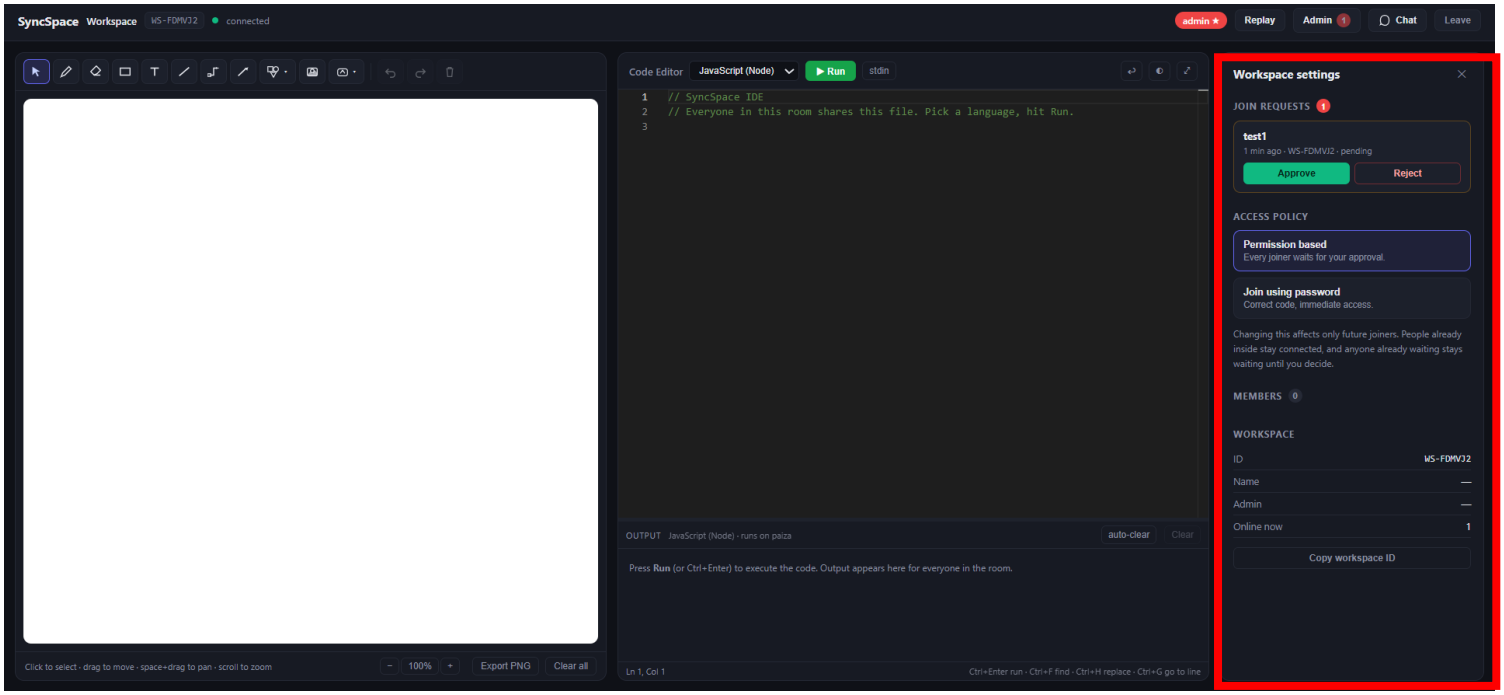


Figure 11.11: Administrator panel with a live join-request queue, the runtime policy switch and member management

Switching the policy while participants are connected is the behaviour worth demonstrating: existing collaborators keep working uninterrupted, anyone already in the waiting room stays there and can still be approved by hand, and only *future* joiners follow the new rule.

11.8 Session Replay

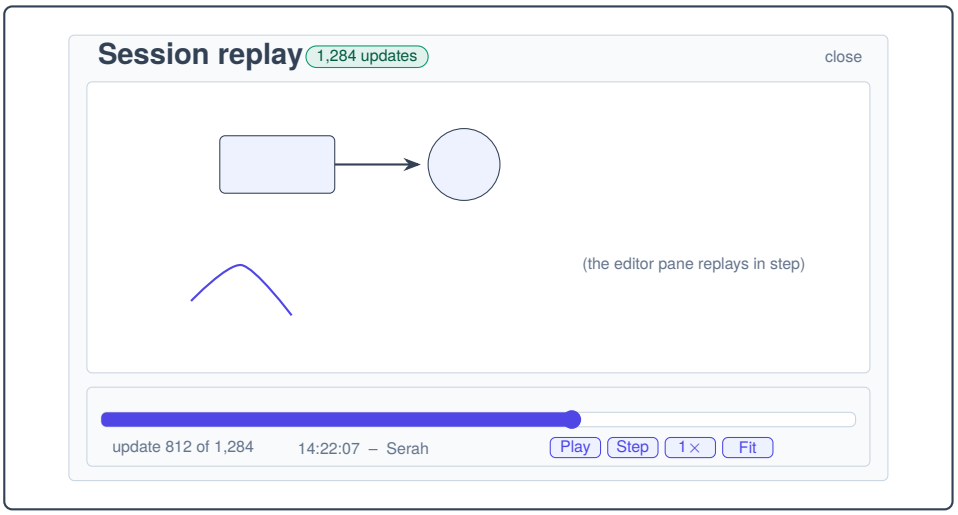


Figure 11.12: Session replay — annotated wireframe

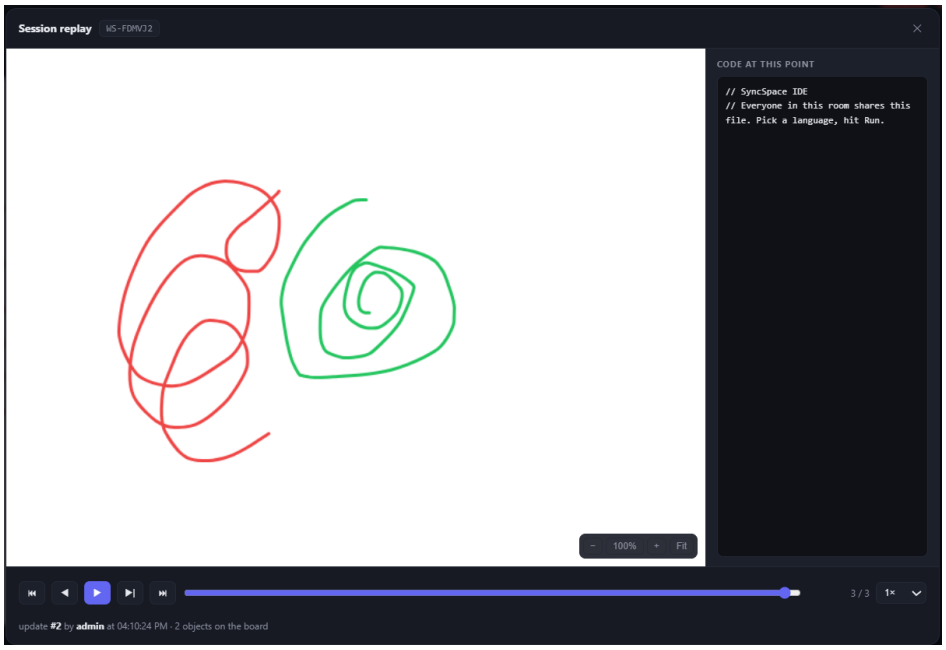


Figure 11.13: Session replay scrubbed to an intermediate point, showing the whiteboard as it stood after update 812 with per-update attribution

The demonstration that best conveys the architecture is the following sequence: open the workspace in two tabs, draw several shapes, connect two of them, type in the code pane, then open replay and drag the slider to zero. The board is blank. Press play and watch it rebuild itself shape by shape, each update attributed to whoever made it, while the code pane fills in alongside. Then point at the other tab: *it never moved*. Replay is a second document, and that is a structural property rather than a precaution.

11.9 Results Summary

Table 11.2: Objectives against delivered outcomes

No.	Objective	Evidence	Status
1	Low-latency bidirectional synchronisation with late-joiner support	Socket relay with full-state transfer on join; asserted in `test-shapes` and `test-workspace`	Met
2	Provable conflict-free convergence	Field-level `Y.Map` modelling; concurrent recolour and move both survive (11 assertions)	Met
3	Structured whiteboard object model	16 primitives, connectors, text, images and strokes as one record type; 28 + 43 assertions	Met
4	Collaborative executable editor	`MonacoBinding` with remote cursors; five languages; shared language and console	Met
5	Dual-shape persistence degrading gracefully	`docstates` and `updatelogs` behind a dual-mode repository	Met
6	Session replay with exact reconstruction	Prefix reconstruction proven byte-identical at every index (38 + 27 assertions)	Met
7	Transport-layer authorisation	Two token classes; handlers registered by class; 15 assertions	Met
8	Safe multi-language execution	Remote sandboxed providers with fallback; 100 assertions	Met
9	Empirical validation	442 assertions across 10 suites, all passing; clean production build	Met

Against the original four-week development plan, all four weeks are complete, and the whiteboard, editor and history subsystems each extend materially beyond what was specified: a connector system with attachment, three routing modes and twelve presets where an arrow was requested; a seven-brush engine with a true partial eraser where freehand lines were requested; a five-language executable environment with a shared console where a synchronised editor was requested; and a streamed, checkpointed replay subsystem with per-update attribution where a backward scrub was requested.

CHAPTER 12

The Account and Identity Layer

Chapters 6 and 7 documented a system in which identity was scoped entirely to a single room. A participant existed only as an entry in one workspace's `members` array, and the sentence "SyncSpace has no cross-workspace account system" appeared in Section 1.8.2 as a deliberate limitation. Table 12.1 of the original edition listed a *global identity layer* among the proposed future enhancements, at medium effort and medium priority.

That enhancement has since been implemented. This chapter documents it. The account layer is additive in the strictest sense: every flow described in Chapters 6 and 7 continues to work unchanged for a user who never creates an account, and no existing endpoint changed its contract. What was added is a second, orthogonal notion of identity that sits *above* the workspace-scoped model rather than replacing it.

12.1 Why an Account Layer, and Why It Stays Optional

The original design's refusal to model users was well motivated. A workspace was self-contained; its members were meaningful only inside it; and the absence of a user table removed an entire class of concerns — password reset flows, email verification, orphaned accounts, cross-workspace authorisation — from a three-week project. Section 6.3 of the original edition defended this explicitly.

Two costs emerged in use, and neither is architectural:

1. **Workspace identifiers were unrecoverable.** A participant who closed the browser tab and lost the identifier had no route back into a room they had legitimately joined. The secret code did not help, because the identifier is what the join form asks for first.

2. **Re-entry required re-approval.** In permission mode, an approved member who reconnected after their twelve-hour access token expired went back into the waiting room and had to be approved a second time by an administrator who had already approved them once.

Both are convenience failures rather than correctness failures, which is why the solution is a convenience layer. The account system's entire purpose is to answer one question

— *which workspaces does this person already belong to?* — and to shorten the path back into them. It grants no authority of its own.

> **The load-bearing invariant**
>
> An account token proves *who* you are. A workspace access token proves you may collaborate in *that* room. Signing in never, by itself, opens a workspace: every entry re-checks membership server-side against `Workspace.members` and mints a fresh access token. The account's `workspaces` array is an index for the dashboard, never an authorisation source.

This separation is what keeps the account layer from widening the attack surface described in Section 9.1. A stolen account token lets an attacker enumerate the rooms the victim belongs to; it does not let them enter one whose membership the server does not independently confirm.

12.2 The Third Token Class

Section 6.2.1 documented two classes of signed token. There are now three. Table 12.1 supersedes Table 6.1.

Table 12.1: The three token classes (supersedes Table 6.1)

Kind	Claims	Lifetime	Grants
`access`	`workspaceId`, `userId`, `username`, `role`	12 hours	The collaborative socket: Yjs sync, awareness, cursors, chat, execution and the AI assistant
`lobby`	`workspaceId`, `requestId`, `username`	2 hours	The waiting room only. No document handler is ever registered for it
`user`	`userId`, `email`, `username`	30 days	The account layer: `/api/auth/me`, the dashboard, and re-entry into workspaces already joined. Opens no room by itself

The lifetimes encode the threat model. An access token is short because it carries real collaborative authority and cannot be revoked once signed — the mitigation is that `requireMember` re-checks live membership on every request, so a removed member's unexpired token stops working immediately. A user token is long because losing it costs

only convenience: the holder is bounced to the sign-in page, and the token grants nothing that is not re-authorised at the point of use.

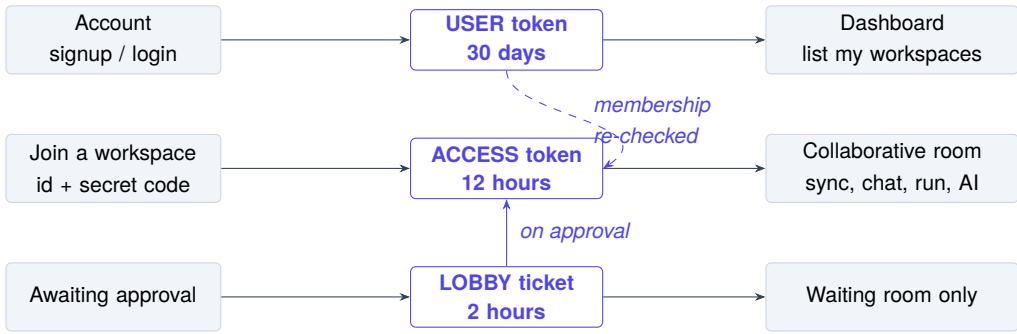


Figure 12.1: The three identities and what each one opens. The dashed edge is the only path from an account to a room, and it passes through a server-side membership check

12.3 The User Model

`backend/models/User.js` holds one document per account.

Table 12.2: The `User` collection

Field	Type	Purpose
`userId`	String, unique, indexed	The *same* value used in `Workspace.members[].userId`. One value links an account to every membership it holds
`email`	String, unique, lowercased, trimmed, indexed	The login identifier. Lowercasing at the schema level makes uniqueness case-insensitive without a second index
`username`	String, trimmed	Display name, seeded into new memberships
`passwordHash`	String	bcrypt, cost factor 10
`workspaces`	Array of {`workspaceId`, `role`}	Denormalised membership index so the dashboard can be built without scanning every workspace document

Two modelling decisions deserve the same treatment Section 5.3.3 gave the originals.

12.3.1 The account id *is* the member id

The obvious design gives an account its own primary key and joins it to memberships through a foreign key. This one reuses a single identifier in both places: when a signed-in

user creates or joins a workspace, the member record written into `Workspace.members` carries their `userId` verbatim rather than a freshly generated one.

The consequence is that membership lookup is a single indexed query on an existing array, and — more importantly — that the authorisation path did not change at all. `requireMember` still compares the `userId` claim in an access token against `Workspace.members`; it neither knows nor cares whether that identifier originated in an account or in `newId()`. An anonymous member and an account-linked member are the same kind of object.

12.3.2 The `workspaces` array is an index, not a fact

Denormalising memberships onto the user document introduces the classic risk: two copies of the same truth, free to diverge. The mitigation is to declare which copy is authoritative and to make the other one incapable of granting anything. `Workspace.members` is the source of truth. `User.workspaces` is consulted only to *list* rooms on the dashboard, and every subsequent action — including opening a listed room — re-reads the workspace document.

A stale entry therefore produces a dashboard card that fails with "You are not a member of this workspace" when clicked, which is a display bug. The inverse error, a workspace that the dashboard fails to list, costs the user nothing they did not already have before accounts existed. Neither can escalate into unauthorised access, and `addMembership` is idempotent, so a repeated join never duplicates a row.

12.4 The Account Service

`services/authService.js` holds the business rules. It follows the same contract as the rest of the project's service layer: return a reason, never throw.

Table 12.3: Account service operations

Operation	Rules enforced	Returns
`signup`	Email not already registered; password hashed with bcrypt at cost 10	409 on duplicate; otherwise a user token and the public account view
`login`	Rate limited to 12 attempts per minute per email address; bcrypt comparison	401 with a message identical for “no such account” and “wrong password”
`getUser`	Reads the account, then resolves the live membership list from the workspace store	The account plus an enriched workspace list for the dashboard

Three details are worth drawing out.

The failure message is deliberately ambiguous. `login` returns “Incorrect email or password.” whether the account does not exist or the password is wrong. Distinguishing them would turn the endpoint into an account-existence oracle — the same reasoning that made the join endpoint return one message for “no such workspace” and “wrong secret code” in Section 6.3.2.

The rate limit is keyed on the email, not the IP. An attacker distributing a credential-stuffing run across many addresses is throttled per target account rather than per source, which is the direction that matters here. It reuses `utils/validate.js`’s existing limiter rather than introducing a second mechanism.

`getUser` does not trust the denormalised array. It calls `findWorkspacesByUser`, which queries `Workspace` documents on `members.userId`, so the dashboard reflects live membership including any role change or removal that happened since the account row was last written.

12.5 Middleware Additions

Two middlewares join `requireMember` and `requireAdmin` from Section 6.2.3.

`requireUser` demands a valid token whose `kind` is exactly `'user'`. The explicit kind check is the whole point: without it, an access token — which also carries a `userId` claim — would satisfy the account endpoints, and a workspace membership would silently become an account.

`optionalUser` is the more interesting of the two. It attaches `req.accountId` when a valid user token is present and *never rejects a request that lacks one*. It is mounted on workspace creation and joining, which is what allows one endpoint to serve both populations: an anonymous visitor creates a workspace exactly as before, while a signed-in visitor creates the same workspace and additionally has it indexed on their account. No branch in the client, no second endpoint, no duplicated validation.

> **Why `optionalUser` cannot fail closed**
>
> A middleware that rejected malformed tokens here would break the anonymous flow for any user holding an *expired* account token in local storage — they would be unable to create a workspace at all, having done nothing wrong. It therefore ignores anything it cannot verify and proceeds as if no token were sent. Nothing downstream is authorised by `req.accountId`; it only decides whether an index entry is written.

12.6 Account-Aware Workspace Entry

Four existing flows learned to recognise an account. In each case the change is the same shape — reuse the account identifier instead of generating a fresh one, then record the membership on the account.

Table 12.4: Where an account identifier is threaded through the existing flows

Flow	Behaviour when signed in	Behaviour when anonymous
Create workspace	Administrator member uses the account's `userId`; membership indexed with role `admin`	Administrator member uses a generated id (unchanged)
Join, password mode	Member row uses the account's `userId`; membership indexed	Generated id (unchanged)
Join, permission mode	Account id is carried on the pending request so approval can reuse it	Request carries no id; approval generates one
Approve request	Reuses the id carried on the request; indexes the membership	Generates an id (unchanged)

Two behaviours emerge from this that were not designed separately but fall out of the identifier reuse.

Joining a room you already belong to becomes re-entry. If a signed-in user submits the join form for a workspace in which they are already a member, `requestJoin` detects the existing membership and delegates straight to `enterWorkspace`. Previously this path produced "the name is already taken in this workspace" — the user colliding with themselves. The secret code is still verified first, so this is a shortcut through the queue, not around the door.

Re-approval disappears. An approved member whose access token expires re-enters from the dashboard without touching the administrator's queue, because `enterWorkspace` re-checks membership rather than re-running the join protocol.

12.6.1 The entry endpoint

`POST /api/workspaces/:id/enter` is the only genuinely new workspace route. It requires a user token, and it performs the check that keeps the whole layer honest:

1. Reject if not signed in to an account.
2. Load the workspace; 404 if absent, 410 if closed.
3. Look up `members` for the account's `userId`; 403 if absent.
4. Only then sign a fresh access token carrying the member's stored role.

Step 3 is the reason a dashboard listing cannot become an unauthorised entry. The role in the issued token is read from the workspace document at that moment, so a member demoted or removed since the dashboard was rendered gets the current answer, not the cached one.

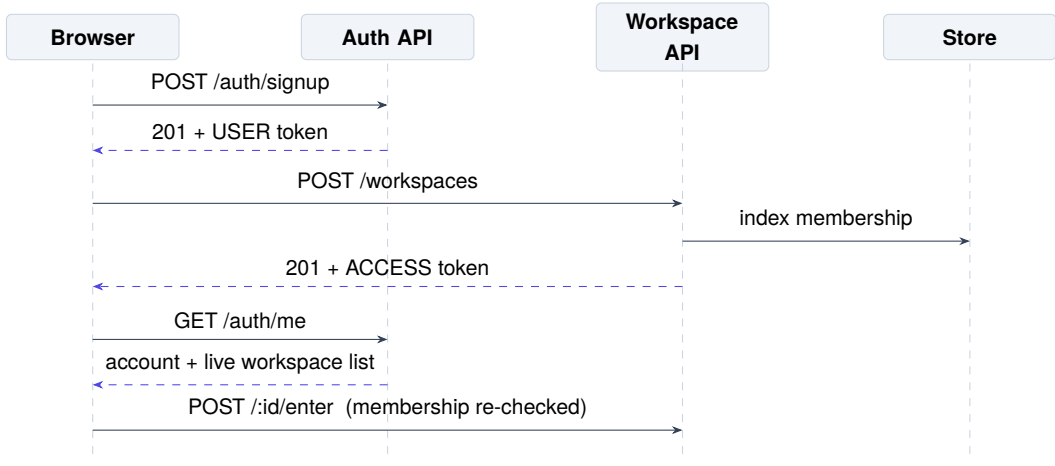


Figure 12.2: Sequence — signing up, creating an account-linked workspace, and re-entering it later from the dashboard

12.7 REST API Additions

Table 12.5 extends Table 6.5. No endpoint documented there changed its contract; the two workspace rows below gained *optional* behaviour only.

Table 12.5: REST endpoints added or extended by the account layer

Method	Path	Auth	Purpose and response
POST	`/api/auth/signup`	None	Create an account. 201 with { token, user }; 409 if the email is taken
POST	`/api/auth/login`	None	200 with a fresh user token. Rate limited to 12 attempts / min / email
GET	`/api/auth/me`	User	The signed-in account plus its live workspace list
POST	`/api/workspaces/:id/enter`	User	Re-enter as an existing member. Membership re-checked; returns a fresh access token
POST	`/api/workspaces`	Optional	*Extended*: links the new workspace to the creator's account when a user token is present
POST	`/api/workspaces/:id/join`	Optional	*Extended*: links the membership; short-circuits to re-entry for an existing member

12.8 Client Implementation

12.8.1 The `useAuth` hook

`hooks/useAuth.js` holds account state for the whole client. It seeds from local storage synchronously so the first paint already knows whether a user is signed in, then revalidates against `/api/auth/me` on mount. A stored token that fails validation is cleared rather than left in place, so the failure mode of an expired session is a sign-in prompt, not a dashboard that errors on every card.

The hook exposes a deliberately small surface — `account`, `ready`, `login`, `signup`, `logout` — and the `ready` flag matters more than it looks. Without it, every consumer would briefly see `account === null` during validation and redirect a legitimately signed-in user to the login page.

12.8.2 Pages

Table 12.6: Client components added by the account layer

Component	Route	Responsibility
`pages/Auth.jsx`	`/login`, `/signup`	One component in two modes, switchable in place without navigation. Submits on Enter; surfaces the server's message verbatim
`pages/Dashboard.jsx`	`/dashboard`	Lists every workspace the account belongs to with its role, access policy and member count; opens one through `/enter`
`hooks/useAuth.js`	—	Account state, storage hydration and token re-validation
`pages/Landing.jsx`	`/`	*Extended*: the navigation bar now reflects signed-in state (see Section 14.2)

The dashboard's "Open workspace" button is worth one note. It calls `/enter`, receives an access token, writes it into the same per-workspace session slot that the join flow uses, and only then navigates. The workspace screen is therefore unchanged — it reads its token from exactly where it always did and has no knowledge that an account layer exists.

12.9 Security Assessment

Table 12.7: Threats introduced by the account layer and their mitigations

Threat	Attack	Mitigation
Account enumeration	Probe `/login` for which emails exist	One message for both failure causes; per-email rate limit
Credential stuffing	Replay leaked credentials at volume	bcrypt cost 10; 12 attempts per minute per email
Privilege escalation via token confusion	Present an access token to an account endpoint, or a user token to a room	Every middleware asserts an exact `kind`; `/enter` re-checks membership before signing
Stale dashboard entry	A removed member's card remains listed	Opening it fails the server-side membership check with 403
Password in transit or at rest	Interception or database disclosure	Only the hash is stored; the plaintext is never logged, returned or written to the account's public view
Long-lived token theft	A stolen 30-day user token	Grants listing and re-entry only; every room entry is re-authorised, and removal takes effect immediately

The honest residual risk is that a 30-day token cannot be revoked before it expires, because the project has no token blacklist or refresh-rotation scheme. The exposure is bounded by what a user token can do — list your rooms, and re-enter rooms whose membership the server still confirms — and an administrator removing the member neutralises the second of those immediately. Adding rotation is listed in Section 15.3.

CHAPTER 13

SyncSpace AI — Design and Implementation

SyncSpace AI is a programming assistant reachable from any workspace through the ∗ **AI** button in the editor toolbar. It answers questions, writes code, explains and documents an existing selection, diagnoses compiler and runtime errors, debugs, generates tests, optimises, and translates between languages.

It is documented as its own chapter rather than as a section of Chapter 6 because it is the only subsystem in the project with an external, non-deterministic dependency, and because almost every design decision in it exists to contain a specific failure that a naive integration produced. The module was rebuilt from an earlier version whose defects are described first: the reconstruction is easier to justify when the thing being reconstructed is on the page.

13.1 The Four Defects

13.1.1 A request for Java returned JavaScript

The reported symptom was that “give me the hello world program in java” produced a JavaScript program, with the model itself explaining that it had written JavaScript “as specified in the programming language prompt”.

The cause was not the model. The client page had no language state at all: each of its eight action branches passed the literal string `"javascript"`, and the *convert* action was hardcoded `"javascript"` → `"python"`. The prompt builder emitted every field it held, unconditionally, under generic headers:

```
ACTION:
generate

USER REQUEST:
give me the hello world program in java

PROGRAMMING LANGUAGE:
javascript
```

Two contradictory instructions, no priority between them, and a system prompt (“Follow the requested programming language”) that pointed at the wrong one. The model

obeyed the field the prompt told it was authoritative. Nothing anywhere in the pipeline ever compared the two statements, because nothing in the pipeline *resolved* anything — it passed fields straight through.

13.1.2 Responses took one to two minutes

Two independent causes compounded.

Table 13.1: Latency causes in the previous implementation

Cause	Effect
`generateContent`, the buffered endpoint, rather than `generateContentStream`	Time-to-first-character equalled time-to-*last* token. The backend awaited the whole completion, Express serialised it, and only then did anything appear
No `thinkingConfig`	The configured model defaults to thinking level `medium`. Hidden reasoning tokens are generated, billed and waited for before a single visible character — on "hello world"
`maxOutputTokens` unset	No ceiling on how long an answer could run
Prompts requesting padding	*generate* asked the model to "explain important implementation decisions briefly", which is the three-paragraph essay attached to a four-line program

The last two matter more than they appear. Output tokens are the slowest part of a response, so verbosity is not a style question — it is latency.

13.1.3 Raw Markdown on screen

The message component rendered its content as a bare string into a `<div>` with `white-space: pre-wrap`. No Markdown parser existed anywhere in the frontend, so every answer arrived as literal hashes, asterisks and backtick fences.

13.1.4 Found during the inspection, not reported

- **The endpoint had no authentication.** The route was mounted with no middleware. The frontend dutifully attached a bearer token and the server never read it, so anyone able to reach the port could spend the project's provider quota, 30 000 characters at a

time, without ever joining a workspace. Every other route in the codebase is guarded; this one was not.

- **No rate limiting**, although `/execute` has it.
- **The health check tested the wrong variable.** It reported on `OPENAI_API_KEY` while the service ran on Gemini, so it answered "not configured" on a correctly configured server — and would have answered "configured" on one holding a stale key for a provider no longer in use.

13.2 Module Architecture

Each layer has exactly one job, and the two transport paths share every layer below the route. The buffered endpoint is implemented *on top of* the streaming one, so the two cannot drift apart.

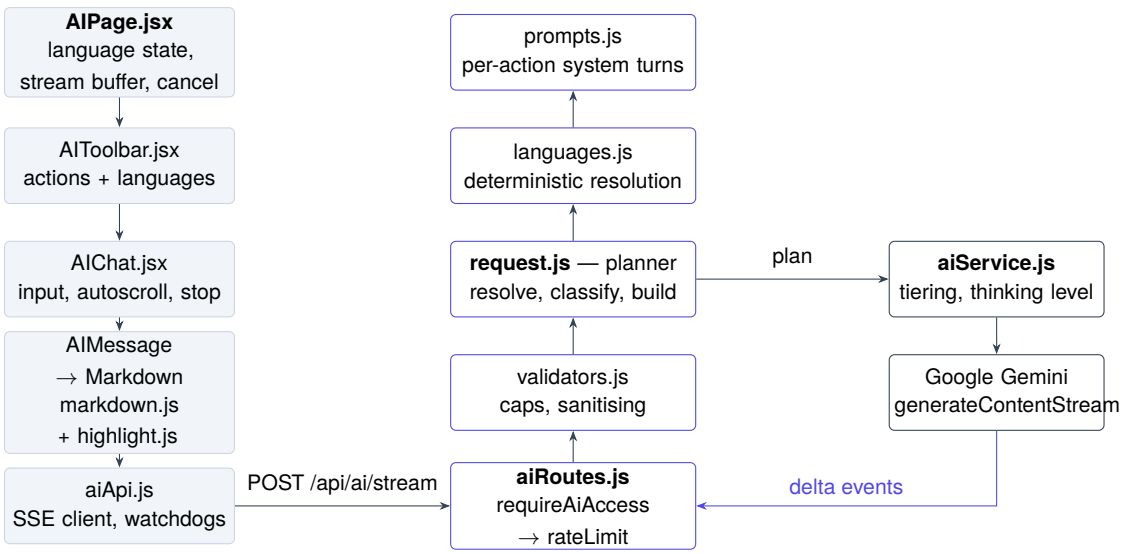


Figure 13.1: SyncSpace AI module architecture. The planner is a pure function, which is what makes every language and model decision testable without a provider

13.3 Validation and Input Hardening

Everything arriving from a browser passes through `validators.js` before it reaches the planner, and nothing downstream re-checks types.

Table 13.2: Per-field input limits

Field	Limit	Rationale
`message`	8 000 chars	Question text
`code`	30 000 chars	Roughly 8 000 input tokens. The model reads all of it before emitting anything, so an unbounded payload is an unbounded wait
`error`	8 000 chars	Stack traces and compiler output
`filename`	200 chars	Editor context only
Combined	45 000 chars	Belt-and-braces against a body slipping past the field caps
Whole body	1 MB	Enforced by Express, unchanged from Section 6.1

The caps are latency controls as much as safety controls. Two further behaviours are less obvious:

Control characters are stripped. Lone carriage returns and the C0 range (except tab and newline) are removed. A stray `\r` inside a pasted code sample can terminate an SSE `data:` line early and desynchronise the client's parser — a transport-level corruption caused by ordinary source code.

An unknown language is not an error. A user asking about COBOL should get an answer, not a validation failure. An unrecognised language simply stops being authoritative, and resolution falls through to the user's own words.

13.4 Deterministic Language Resolution

`services/ai/languages.js` exists to kill the defect in Section 13.1.1. Language is *resolved* before a prompt is built — deterministically, in one place, with an explicit priority order and an explicit conflict rule. No model call is spent deciding what language the user asked for, because a regular expression is both faster and more reliable than a language model at reading the word "java".

13.4.1 The priority order

1. **What the user explicitly wrote** in this request — "…in java".
2. **Client metadata** — the toolbar dropdown, seeded from the editor's current language.
3. **Structural inference** from any attached code.

with one override: when (2) wins but (3) firmly disagrees, (3) takes it. That is the stale-dropdown case — the editor still says JavaScript because nobody touched it, while the user has pasted a Java file. *Pasted code is evidence; a dropdown is a leftover.*

The resolver returns its reasoning as well as its answer, so the route can log why a language was chosen and the test suite can assert on the reason rather than merely on the outcome.

Table 13.3: Language resolution outcomes

Situation	Resolved	Source	Conflict recorded
“hello world in java”, dropdown JavaScript	Java	`user-request`	User request beats editor default
Dropdown Python, no prose hint	Python	`editor-metadata`	None
Dropdown JavaScript, Java file pasted	Java	`supplied-code`	Pasted code overrides a stale default
No hint at all, action *generate*	—	`none`	Answered locally with a clarifying question

13.4.2 Two details that matter more than they look

Aliases are matched longest-first. The alias table is flattened once at module load and sorted by descending alias length, so `javascript` is always scanned before `java` and the substring can never win. This single ordering is the entire defence against the reported bug.

Word boundaries must understand + and #. The regular expression metacharacter `\b` is useless here: in `c++` the boundary sits *between* the `c` and the first `+`, so `/\bc\b/` happily matches the `c` of `c++` and a request for C++ silently becomes C. The patterns use lookarounds that treat language sigils as part of the identifier:

```
1  function aliasPattern(alias)   return new RegExp(`(?<![A-Za-z0-9_+#.])$esc(alias)(?![A-Za-z0-9_+#])`, 'i');
```

13.4.3 Conversions

A conversion needs two languages, and getting the direction backwards is worse than asking. Direction is read from prose — *from*, *to*, *into*, $\rightarrow$ — with each position in the text claimed by at most one alias, so “convert this java to python” yields the ordered pair (Java, Python) rather than an unordered set.

Two languages with no directional word between them is genuinely ambiguous, and the module **asks** rather than guessing. That clarification costs zero provider calls and zero latency, because it never leaves the server.

13.4.4 Where the answer is put

The resolved language is injected into the **system** instruction, which outranks anything in the user turn:

```
OUTPUT LANGUAGE: Java
This is absolute. Every code block must be Java and fenced as ```java.
Never substitute a different language...
```

13.5 The Planner

`services/ai/request.js` turns a validated body into everything the provider call needs, and decides — before any network I/O — whether a model call is warranted at all. Splitting it out of the provider adapter is what makes the module testable: the plan is a plain object, so every language decision, every prompt and every model choice can be asserted without a provider, a key or a millisecond of latency.

13.5.1 Field relevance

Each action declares which fields it may send. Anything else is noise that costs input tokens and gives the model more surface to weigh against the user's actual words.

Table 13.4: Fields each action is permitted to send

Action	Fields used	Needs code?
`chat`	message, code	No
`generate`	message	No
`explain`	code, message	Yes
`error`	error, code, message	No
`debug`	code, error, message	Yes
`tests`	code, message	Yes
`optimize`	code, message	Yes
`convert`	code, message	Yes
`document`	code, message	Yes

13.5.2 Requests answered without a provider call

Three cases are resolved locally and never reach the network: an entirely empty request, a code-bearing action with no code and no instruction, and an ambiguous conversion. Each returns a plain sentence telling the user what is missing. The provider quota is not spent to discover that the user submitted an empty form.

13.5.3 Complexity classification

One function drives *both* the model tier and the thinking level, which makes it the principal lever on latency. Getting it wrong in the cheap direction costs quality on hard problems; getting it wrong in the expensive direction leaves a user watching a spinner while a reasoning model deliberates about `System.out.println`.

Table 13.5: Complexity classification and its consequences

Class	Triggered by	Model tier	Output ceiling
`simple`	*chat* or *generate* under 220 characters — textbook one-liners	fast	1 024 tokens
`standard`	Reasoning actions with a small payload; everything unclassified	per action	3 072 tokens
`deep`	*debug* / *error* / *optimize* over 1 200 characters; *convert* / *explain* / *tests* over 2 500; anything over 3 000	smart	6 144 tokens

Detail explicitly requested (“explain”, “step by step”, “in detail”) raises the ceiling to 4 096

A ceiling is a latency guarantee, not merely a cost control: a Hello World allowed 8 000 tokens can still ramble into 8 000 tokens.

13.6 System Instructions

`prompts.js` assembles one system instruction per request from a shared spine plus an action-specific body. Three properties are enforced deliberately.

They are assembled, not static. The previous prompts were fixed strings, so the resolved language never reached the system turn. Language is now pinned here, where it outranks the user turn.

They do not request unwanted work. The *generate* instruction now ends with an explicit prohibition — no summary, no breakdown of decisions, no list of what the code

does — because the previous wording is what produced the “Key Implementation Decisions” essay attached to a four-line program.

They constrain output shape. Fence tags are pinned to the resolved language, so a Java answer cannot arrive fenced as `javascript` and render under the wrong highlighter.

The assembled instruction is held under roughly 1.2 kB. Long system prompts do not improve adherence past a point, and they cost input tokens on every single request.

13.7 Model Tiering and Thinking Levels

One model for every action was always the wrong shape: “explain this stack trace” and “hello world in java” do not deserve the same reasoning budget.

Table 13.6: Model tiers

Tier	Default model	Thinking	Used for
fast	`gemini-3.5-flash-lite`	`minimal`	chat, generate, tests, document
smart	`gemini-3.6-flash`	`low`	debug, error, optimize, convert, and any payload over 4 kB

The fast tier is not a quality sacrifice for ordinary coding work: per the provider’s own launch figures the lite model outperforms the larger Flash model on SWE-Bench Pro (54.2 % against 49.6 %) while running at roughly 350 output tokens per second. The smart tier uses `low` rather than the model’s `medium` default because this is an interactive assistant rather than a batch agent; genuinely deep requests are lifted back to `medium` by the complexity classifier.

Two operational details:

- `minimal` is not accepted by every model in the family. The selector raises the floor to `low` for models that reject it, rather than letting the request fail.
- **Sampling parameters are deliberately never sent.** `temperature`, `top_p` and `top_k` are deprecated for this model generation and are ignored by the API. Output shape is controlled through the system instruction and the thinking level instead. Sending them would be cargo cult.

A single environment override, `GEMINI_MODEL`, pins one model across both tiers for A/B testing — and, left set by accident, disables tiering entirely so that trivial requests keep paying for the heavier model. The health endpoint reports whether a pin is in force for exactly this reason.

13.8 The Streaming Transport

The transport is Server-Sent Events rather than a WebSocket. This is a one-way, one-shot text stream over the HTTP request that started it: no new transport, no Socket.IO room semantics, no reconnection state, and the existing socket layer of Section 6.4 is untouched.

Response headers are flushed **before** the provider is called. That is the central trick: the browser's time-to-first-byte becomes backend latency — single-digit milliseconds — rather than model latency, so the interface can switch from "thinking" to "streaming" immediately instead of after a minute of silence.

Table 13.7: Server-sent event types

Event	When	Carries
`meta`	Immediately, before the provider call	Resolved language and its source, model, tier, thinking level, prompt size
`delta`	Repeatedly	A text fragment
`done`	On success	Character count, time to first token, total elapsed milliseconds
`error`	On failure	A sanitised code and a human sentence

The `meta` event is a design decision, not diagnostics: it is what allows the interface to display the language the server actually chose, which is the visible proof that "in java" beat the JavaScript default.

13.8.1 Back-pressure and cancellation

Back-pressure is real rather than nominal. If `res.write` returns `false`, the route awaits `drain` before pulling more from the provider, so a slow reader cannot balloon server memory with a queued answer.

Cancellation runs the full length of the chain. Pressing *Stop* aborts the client fetch, which closes the socket, which fires the request's `close` handler, which aborts the provider call server-side. A cancelled request stops costing tokens rather than merely stopping being displayed.

13.8.2 Two independent deadlines

The SDK's own timeout covers the request. Two further timers cover the ways a provider can fail without failing: a hard ceiling of 60 seconds on a whole generation, and a 20-second stall timer, re-armed on every chunk, for a provider that accepts the connection

and then dribbles. When a stall aborts a partly delivered answer, the user is told that the partial text above is all that arrived — the answer is not silently discarded.

13.9 The Client

13.9.1 Language as page state

The old page's defect was structural: it had no language state, and no route by which the editor's real language could reach it. Language is now first-class page state, seeded from whatever the editor was set to when the assistant was opened, editable by the user, and sent as *metadata*. Nothing on the page decides the final language; the server resolves it and reports back, and the page displays what the server chose.

Nine near-identical API wrappers were replaced by one request shape. Nine positional signatures is precisely how `convert(code, "javascript", "python")` came to be hardcoded at the call site.

13.9.2 Buffered rendering

Deltas arrive far faster than React should re-render. Text accumulates in a `ref` and is flushed to component state once per `requestAnimationFrame`:

```
l  const flush = useCallback(() =>   frameRef.current = 0;  const id = activeIdRef.current;  const text = bufferRef.current;  if (!id) return;  setMessag
```

A 600-token answer costs roughly 60 renders instead of roughly 600. The consequence that users notice is that the input box never drops keystrokes while an answer streams in beside it — and the input is never disabled during generation.

13.10 Rendering Markdown Without a Dependency

The renderer is a purpose-built parser, a tokeniser and a React renderer: **zero new dependencies**. The obvious library was evaluated first and lost on three counts.

1. **Weight.** The standard unified/remark/rehype pipeline is a dozen or more transitive packages and roughly 120 kB minified, before a syntax highlighter adds a further 90 kB gzipped. This application already ships Monaco and Konva.

2. **Streaming.** A general parser is built for complete documents. Mid-stream, the text routinely ends inside an unterminated fence, and a strict parser renders that as literal backticks — so every code block flickers as raw text until its closing fence arrives. This parser treats an unterminated fence as an *open code block*. That property, not the saved kilobytes, is the real argument.

3. **Safety.** The parser emits a token tree that the renderer turns into React elements. No HTML string exists anywhere, so `dangerouslySetInnerHTML` is never needed and a raw `<script>` in a model response displays as text.

Monaco was also considered for the code blocks and rejected: it is an *editor*, and mounting one per block in a transcript costs a model, a view and a layout pass each — absurd for a five-line Hello World, and it would stutter while streaming.

Supported syntax: headings, fenced code with language labels and a copy button, ordered, unordered and nested lists, blockquotes, horizontal rules, GFM tables, and inline code, bold, italic, strikethrough and links. Link schemes are restricted to `http`, `https` and `mailto` — a model can emit a `javascript:` URL, and a chat bubble is not the place to find out what it does.

13.11 Security

Table 13.8: AI module security properties

Concern	Implementation
Key handling	The key lives in `backend/.env`, is read only inside `aiService.js`, and never crosses the HTTP boundary. It must never be placed in a `VITE_` variable — anything so prefixed is compiled into the public frontend bundle
Authentication	`requireAiAccess` accepts a workspace *access* token, with membership re-checked live against the store so a removed member loses AI access instantly, or a signed-in *user* token. Lobby tickets are rejected
Rate limiting	20 requests per minute per identity by default, keyed per workspace member or per account
Payload caps	Per field and combined, inside the existing 1 MB body limit
Stream integrity	Control characters and lone carriage returns stripped before the text can reach an SSE frame
Error disclosure	Provider errors are classified by shape, logged in full server-side, and reported to the user as a sentence. Raw provider messages can contain request URLs, model paths and quota identifiers; none of it reaches the browser

Table 13.9: Provider failure classification

Code	Status	Reported to the user as
`bad-key`	502	The service rejected the configured key; check the server
`forbidden`	502	The key may lack access to this model
`bad-model`	502	The configured model does not exist
`rate-limited`	429	The service is rate limiting; wait and retry
`provider-down`	503	Temporarily unavailable
`network`	504	Could not reach the service
`timeout`	—	Took too long, or stopped partway with a partial answer
`cancelled`	499	Request cancelled
`not-configured`	503	No key is set on this server

13.12 Testing

Table 13.10: AI test suites

Suite	Checks	Covers
`backend/test-ai.mjs`	92	Language identity, validation, prompt construction, provider-free answers, model selection, security, streaming, a per-language matrix, an action matrix, failure handling, and buffered-versus-streamed latency
`frontend/test-ai-render.mjs`	62	Block and inline parsing, rendering safety, streaming safety, malformed input, syntax highlighting, the reported bug end to end in the DOM, and performance

The backend suite runs the real route, the real planner, the real prompts and the real SDK against `test-support/mock-gemini-api.mjs`, which speaks the provider's actual wire format — including `:streamGenerateContent?alt=sse` — and models the one property that dominates real latency: a thinking stall sized by the requested level. No key, no network, no cost. **The only fake is the provider itself.**

The frontend suite renders components with `react-dom/server` and, notably, feeds the renderer **every prefix** of a realistic answer, one character at a time, asserting that it

never throws and never displays a raw fence. This is the test that pins the open-fence behaviour described in Section 13.10.

A separate benchmark, `bench-ai.mjs`, measures four configurations against the real provider — from the old buffered, default-thinking setup through to the new streamed, minimal-thinking fast tier — reporting time to first character and total time for each, plus whether each produced Java or JavaScript.

13.13 Known Limits

Stated as plainly as Section 10.9 stated the testing limits:

- **Provider rate limits are a wall no code can climb.** On a 429 the module reports it honestly and stops. It does not silently retry, because a retry loop against a rate limit makes latency worse, not better.
- **Conversation history is not sent.** Each request is independent, so a follow-up such as "now make it recursive" will not see the previous answer. This is a deliberate decision rather than an oversight: history multiplies input tokens on every turn, and this module's first priority was latency.
- **Cold-start TLS.** The first request after the backend starts pays a handshake to the provider. The client is a process-wide singleton so this is paid once rather than per request — but a development reload discards the warm connection, so development feels slightly slower than production for one request after each restart.
- **The assistant cannot see the workspace.** It receives what the user sends it plus an optional filename. It has no access to the whiteboard, the shared document or the chat.
- **Answers are not verified.** The system instruction forbids inventing APIs and asks the model to say when it is unsure, which reduces but does not eliminate confident error. Generated code is not executed before it is displayed, although the user can run it through the existing execution service in one step.

CHAPTER 14

Revised User Interface and Results

14.1 How to Use This Chapter

This chapter documents the screens introduced or rebuilt since the original edition: the redesigned landing page, the account screens, the dashboard, and the SyncSpace AI assistant. It follows the convention of Chapter 11 — each section states what the screen is for, what to look for in it, and reserves a framed region for the capture itself.

> **Filling in the captures**
>
> Each framed region below is a placeholder sized for the capture it names. Replace it with the screenshot described in its caption. Where a section says *two windows*, the capture must be taken with two browser windows visible simultaneously — a single-window capture cannot demonstrate the property the figure exists to show.

Section 11.2 of the original edition documented an earlier landing page with two cards and no navigation. **Section 14.2 below supersedes it.** All other sections of Chapter 11 remain accurate.

14.2 The Redesigned Landing Page

The landing page was rebuilt from a two-card chooser into a full entry experience. The two original doors — create a workspace, join a workspace — are unchanged and still lead to the same routes; what surrounds them is new.

Table 14.1: Landing page structure

Region	Content
Navigation bar	Brand mark, in-page anchors (Features, How it works, Collaboration), and an account area that switches on signed-in state: *Sign in* and *Join Workspace* when anonymous; the account name, *My Workspaces* and *Sign out* when signed in
Hero	Availability badge, headline, one-paragraph description, the two primary actions, and three statistics — live sync, five executable languages, full replay
Features	Six cards: real-time code editor, infinite whiteboard, built-in chat, session replay, secure access, live presence
How it works	Three numbered steps: create, share the code, collaborate
Call to action	The two primary actions repeated after the explanatory content
Footer	Brand, one-line description, technology note

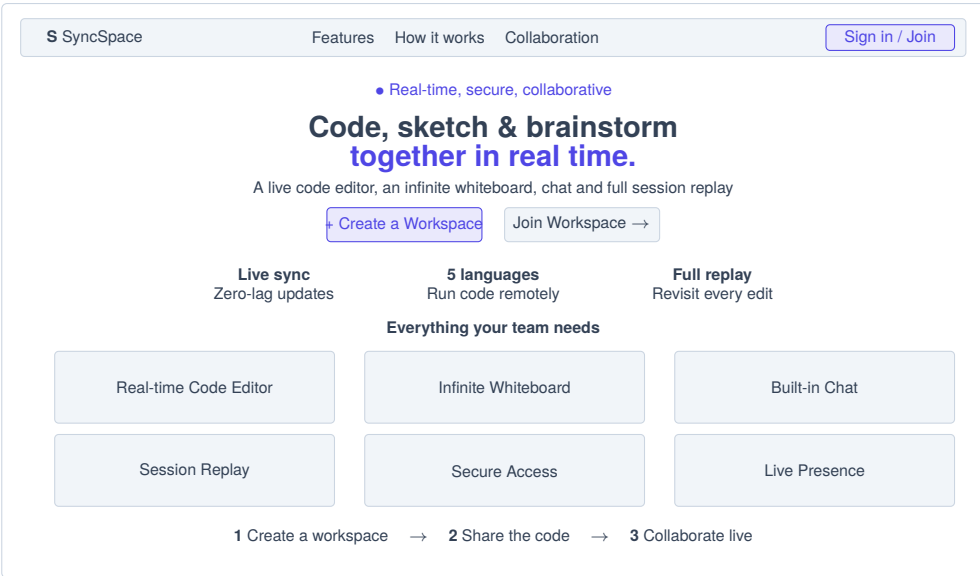


Figure 14.1: Redesigned landing page — annotated wireframe (supersedes Figure 11.1)

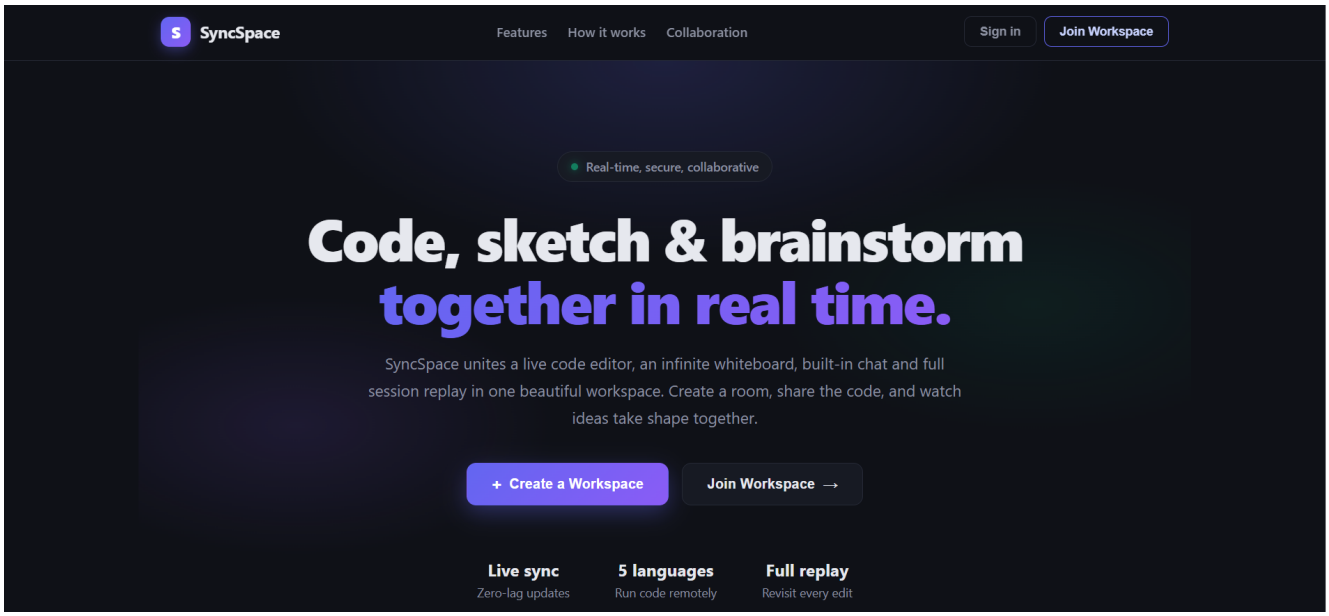


Figure 14.2: Landing page — anonymous visitor

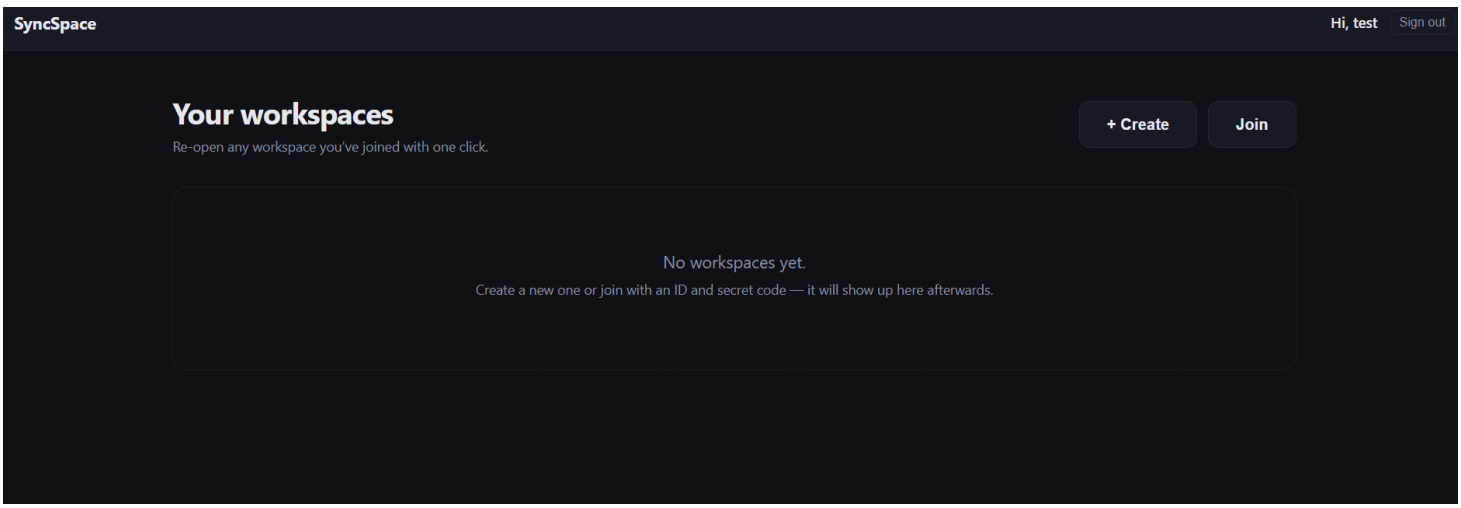


Figure 14.3: Landing page — signed-in state of the navigation bar

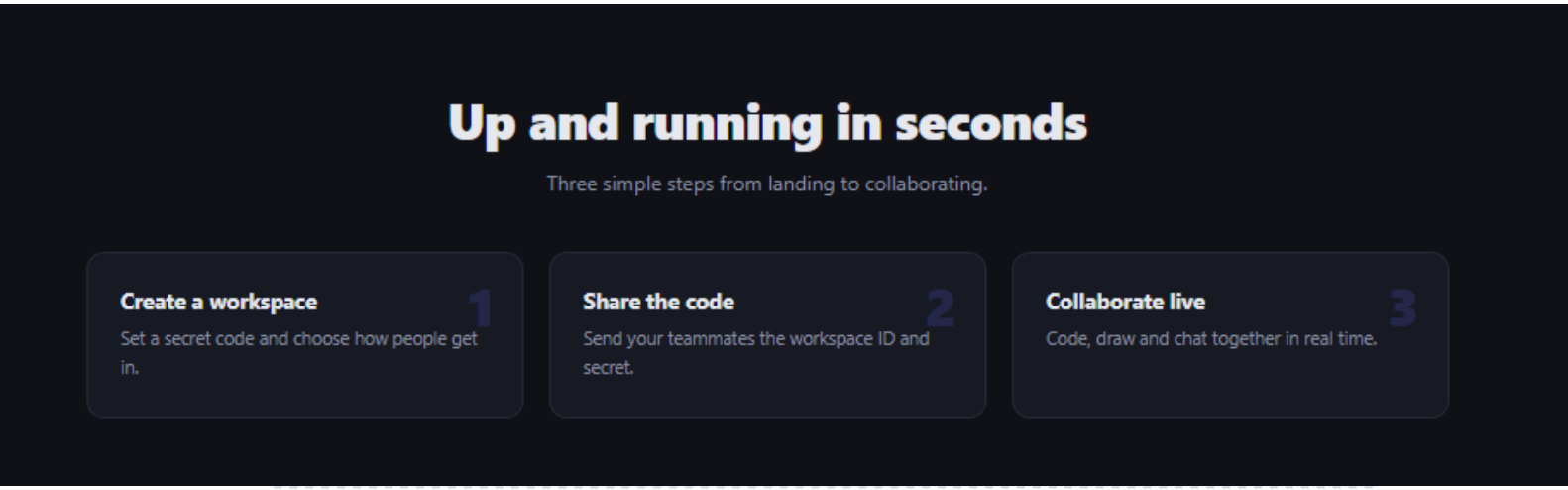


Figure 14.4: Landing page — feature grid and the three-step explanation

14.3 Creating an Account

One component serves both sign-up and sign-in, switchable in place without navigation. Sign-up collects a username, an email address and a password of at least six characters; sign-in collects the last two.

Two things are worth pointing out in the capture. First, the form states plainly that an account is optional — "you can still create or join workspaces without an account" — because the account layer must not read as a registration wall in front of a system that does not require registration. Second, the error region surfaces the server's message verbatim, and that message is identical for an unknown email and a wrong password.

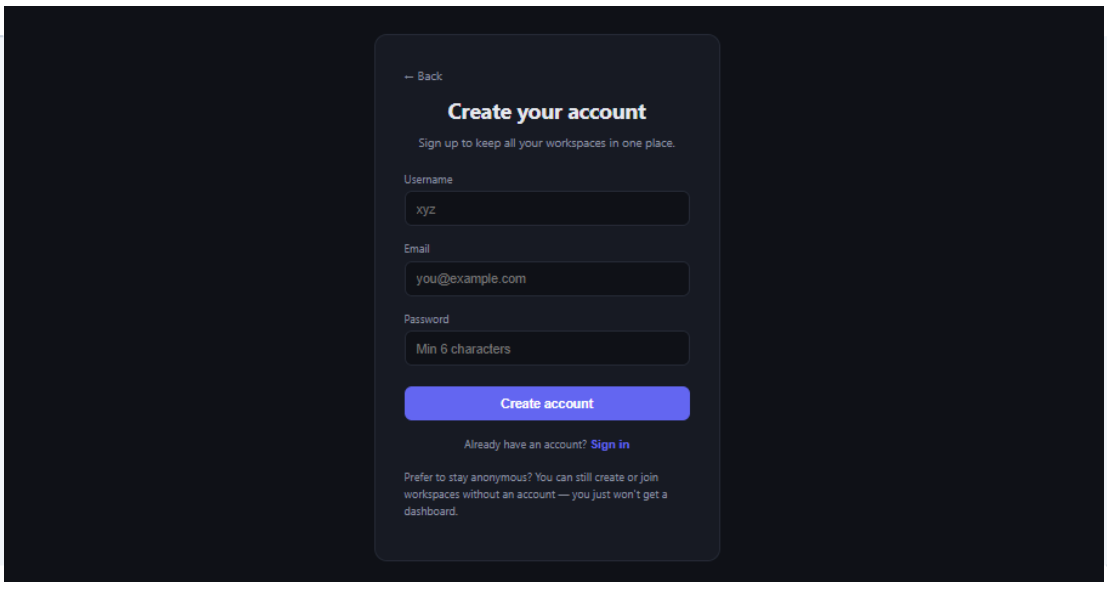


Figure 14.5: Account creation

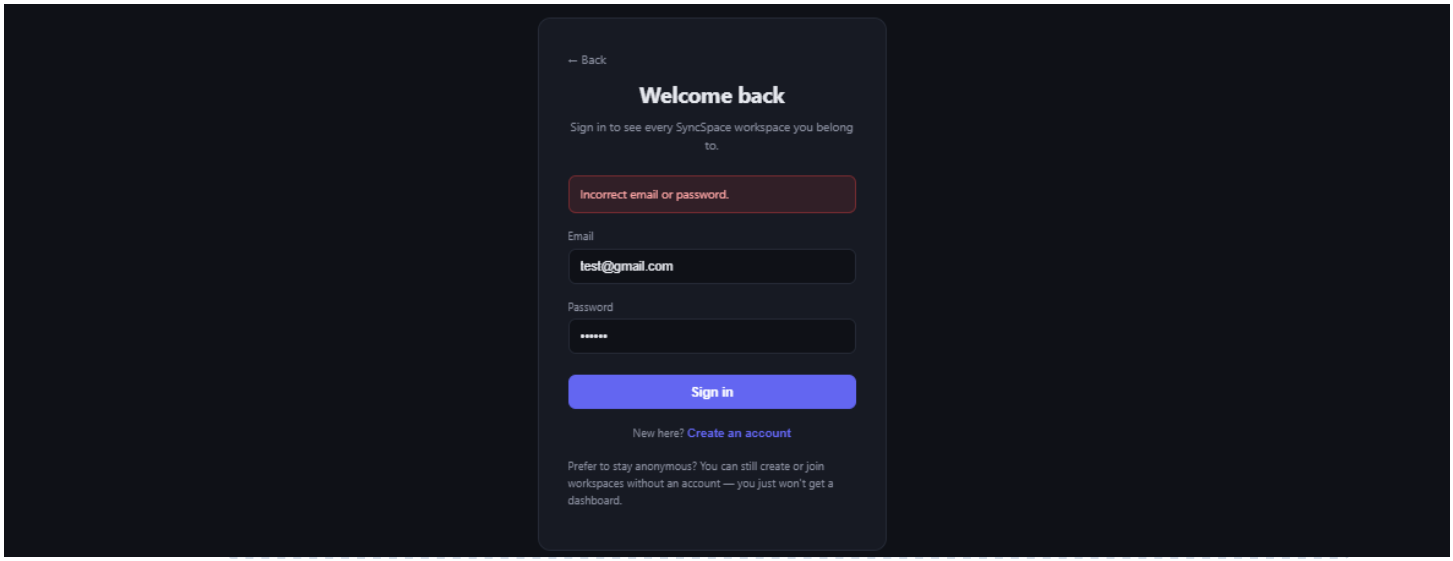


Figure 14.6: Sign-in with a rejected credential — note the deliberately ambiguous message

14.4 The Dashboard

The dashboard is the account layer's reason to exist: every workspace the signed-in account belongs to, listed with its role, its access policy and its member count, each openable in one click.

Table 14.2: Dashboard card elements

Element	What it conveys
Workspace name	The name given at creation
Policy chip	*Approval* or *Password* — the access policy currently in force
Workspace identifier	Shown in monospace, so it can be read out or copied to invite someone
Role tag	*admin* or *member*, read from live membership rather than the cached index
Member count	Current members
Open workspace	Calls `/enter`; the button shows *Opening…* while membership is re-checked

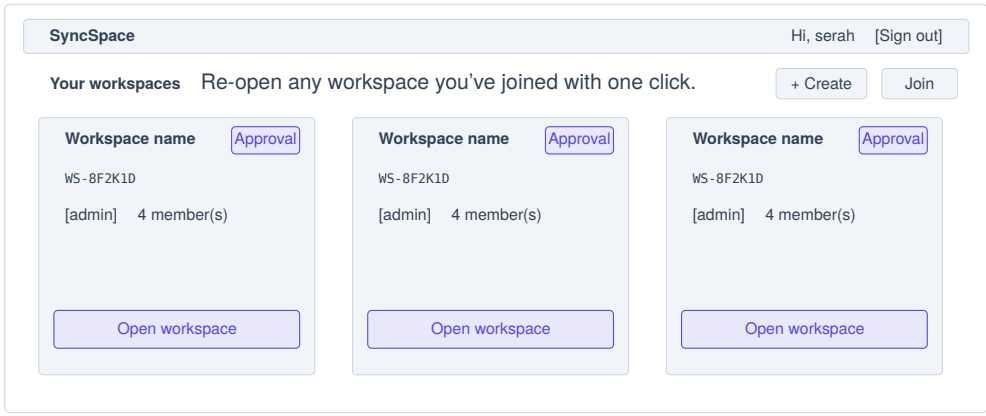


Figure 14.7: Dashboard — annotated wireframe

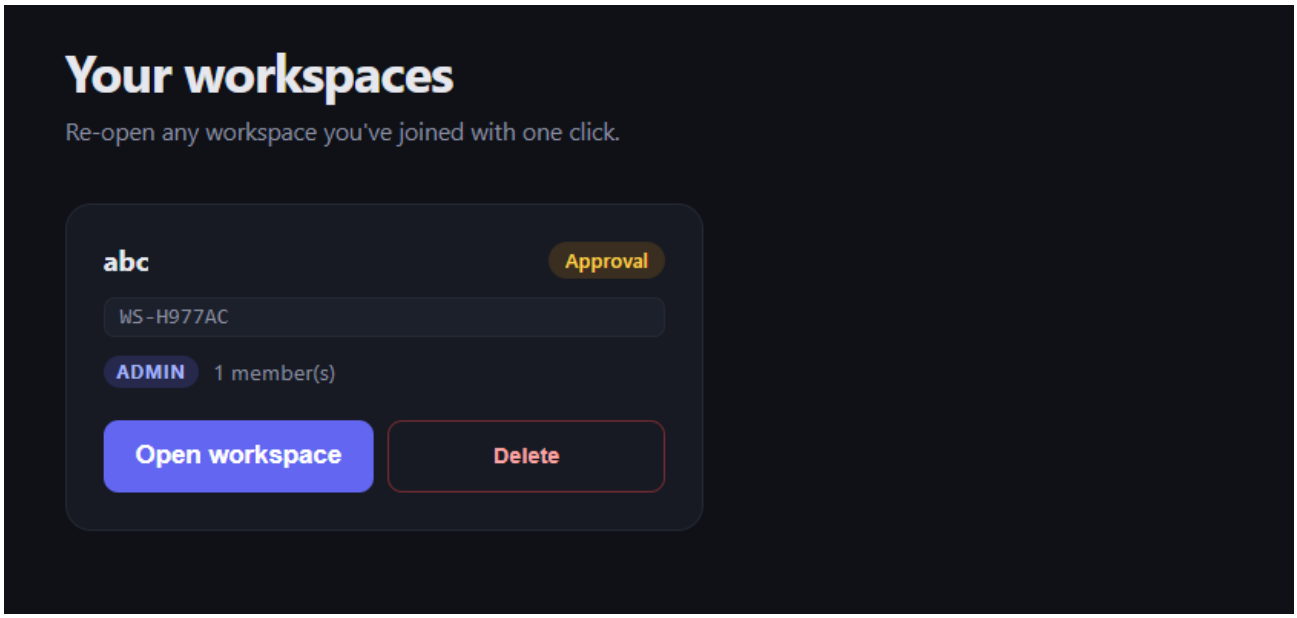


Figure 14.8: The dashboard with several linked workspaces

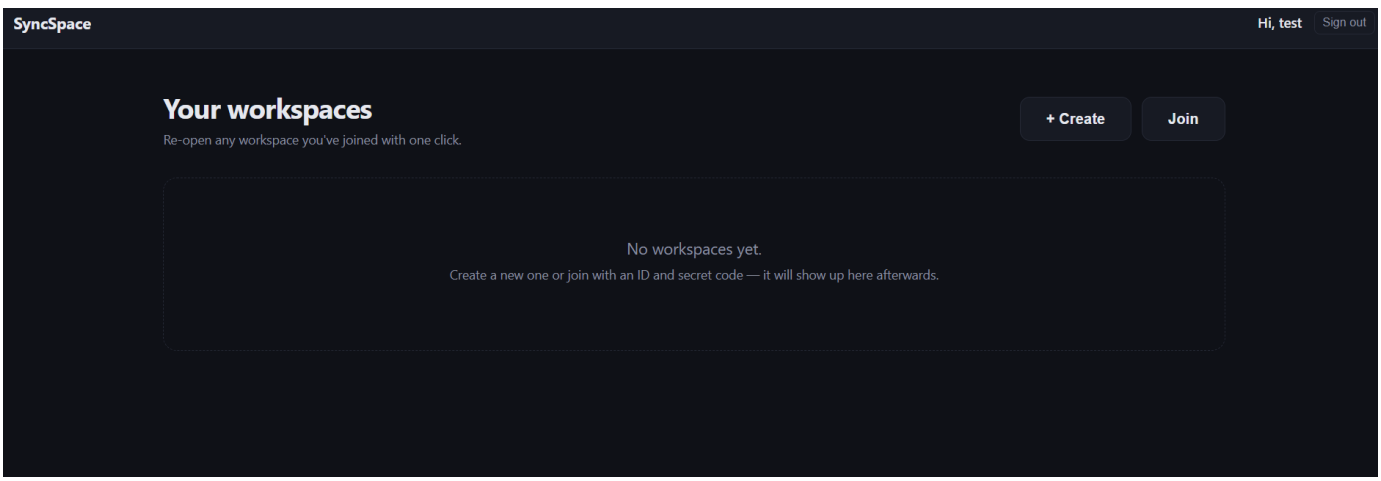


Figure 14.9: Dashboard empty state

14.5 Account-Linked Creation and Re-entry

The workspace creation form is unchanged. What changed is invisible on it: when the creator is signed in, the administrator membership is written with the account's identifier, so the new workspace appears on the dashboard immediately afterwards. The capture below should therefore be taken as a *pair* — create, then return to the dashboard — because the property being demonstrated is the link between the two screens rather than either screen alone.

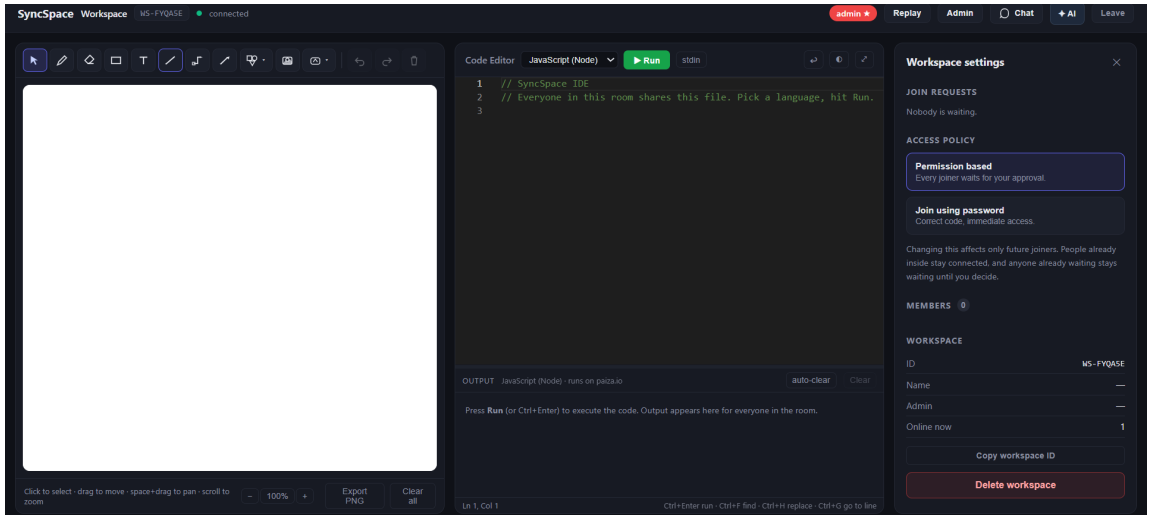
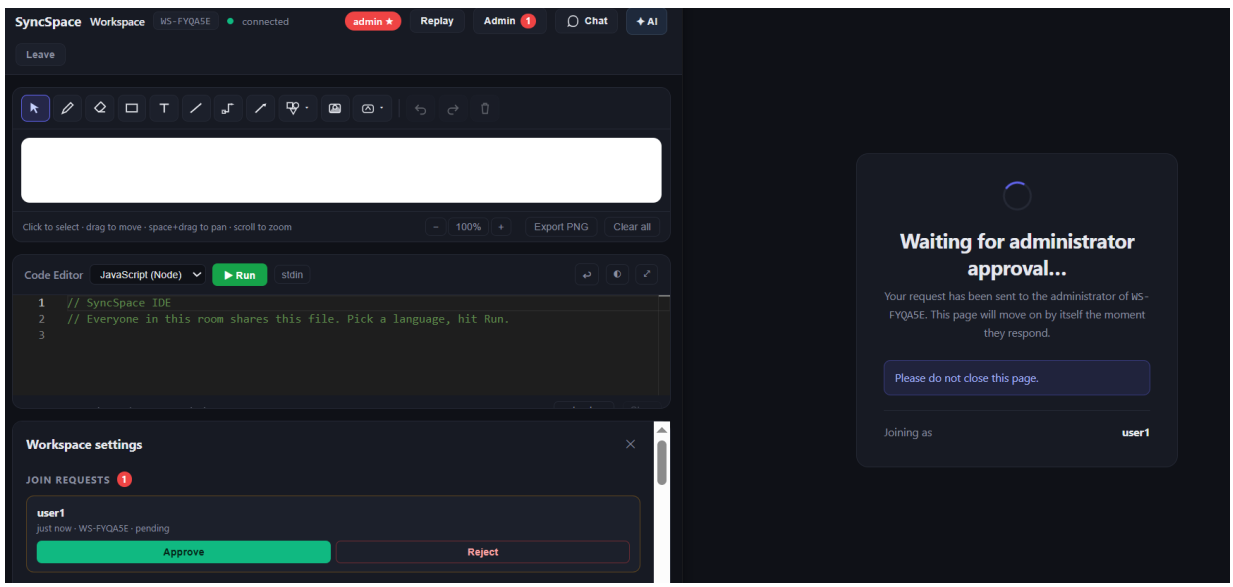


Figure 14.10: A workspace created while signed in appears on the dashboard

Re-entry is the flow that removes the re-approval problem described in Section 12.1. Open a permission-mode workspace from the dashboard: the administrator's join queue must remain empty throughout, because `/enter` re-checks the existing membership instead of re-running the join protocol.



14.6 SyncSpace AI — The Assistant Screen

The assistant occupies its own route beneath the workspace, reached from the editor toolbar and returning to the workspace by the back control. The screen has three regions: the header, the action and language toolbar, and the transcript with its input.

Table 14.3: Assistant interface elements

Element	Location	What it conveys
Action tabs	Toolbar, left	The nine actions: chat, explain, generate, analyze error, debug, tests, optimize, convert, documentation
Language selector	Toolbar, right	Seeded from the editor's language; a default the server may override
Target language	Toolbar, right	Shown only for *convert*
Resolved-language chip	Above each answer	What the server actually chose, and which signal chose it
Stop	Input row	Aborts the fetch, which aborts the provider call server-side
Clear	Header	Empties the transcript; disabled while an answer streams
Workspace identifier	Header, right	The room the assistant was opened from

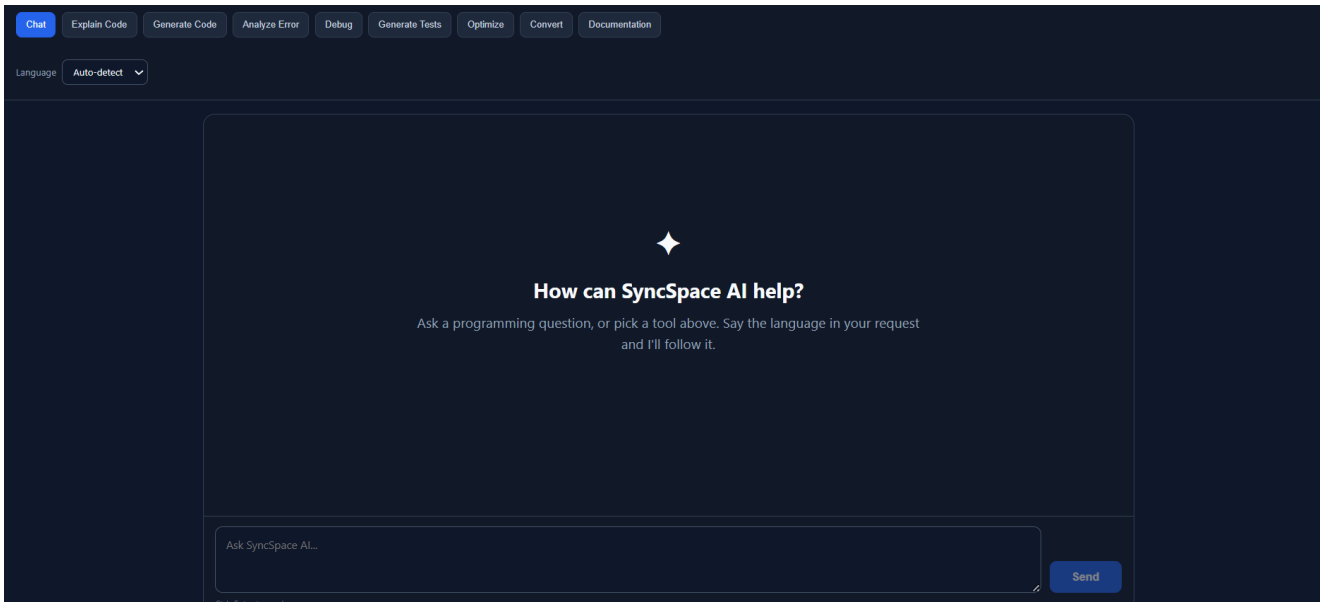


Figure 14.12: SyncSpace AI — initial state

14.6.1 The reported defect, resolved

This is the capture that demonstrates Section 13.1.1 is fixed. Set the language selector to **JavaScript** deliberately, then ask, in the message box, for "the hello world program in java". The answer must be Java, and the metadata chip must attribute the decision to the user's request rather than to the dropdown.

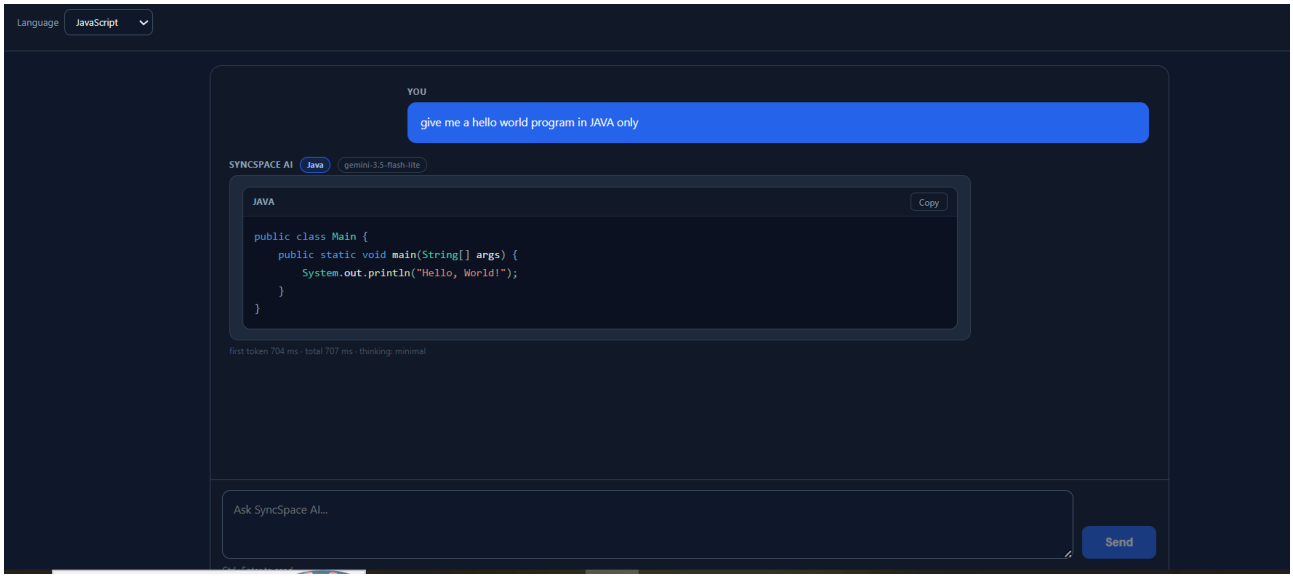


Figure 14.13: Prose outranks the dropdown — a Java request answered in Java

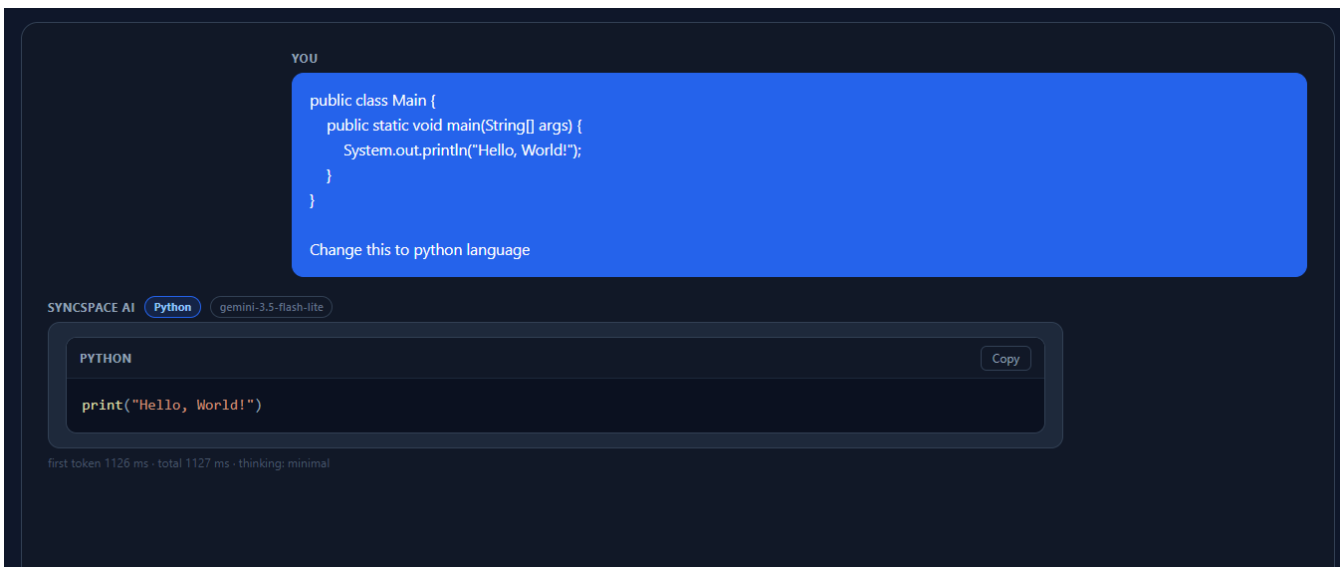


Figure 14.14: Pasted code overrides a stale editor default

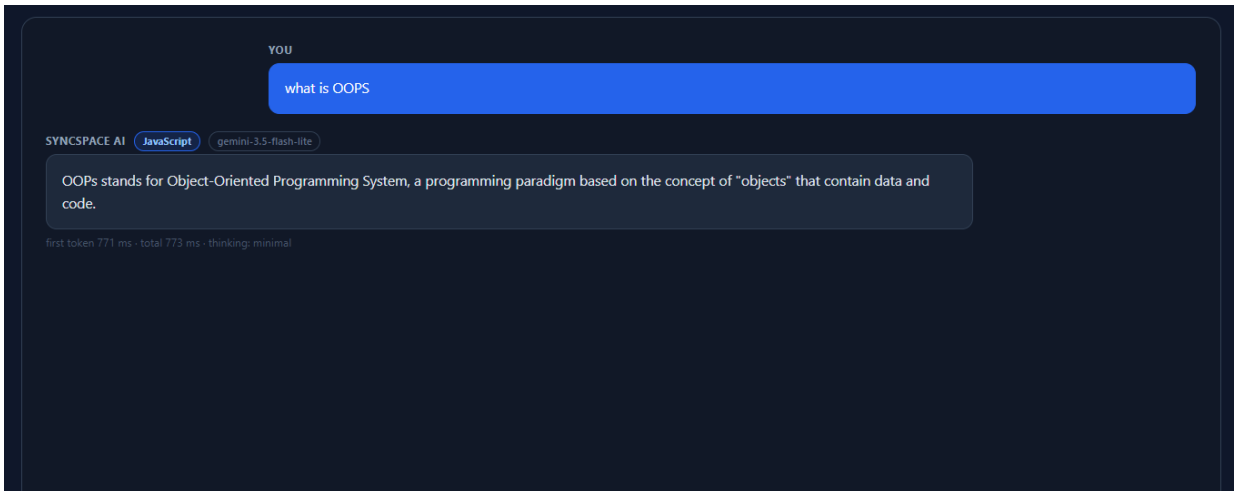


Figure 14.15: A clarification answered locally, at zero provider latency

14.6.2 Streaming and rendering

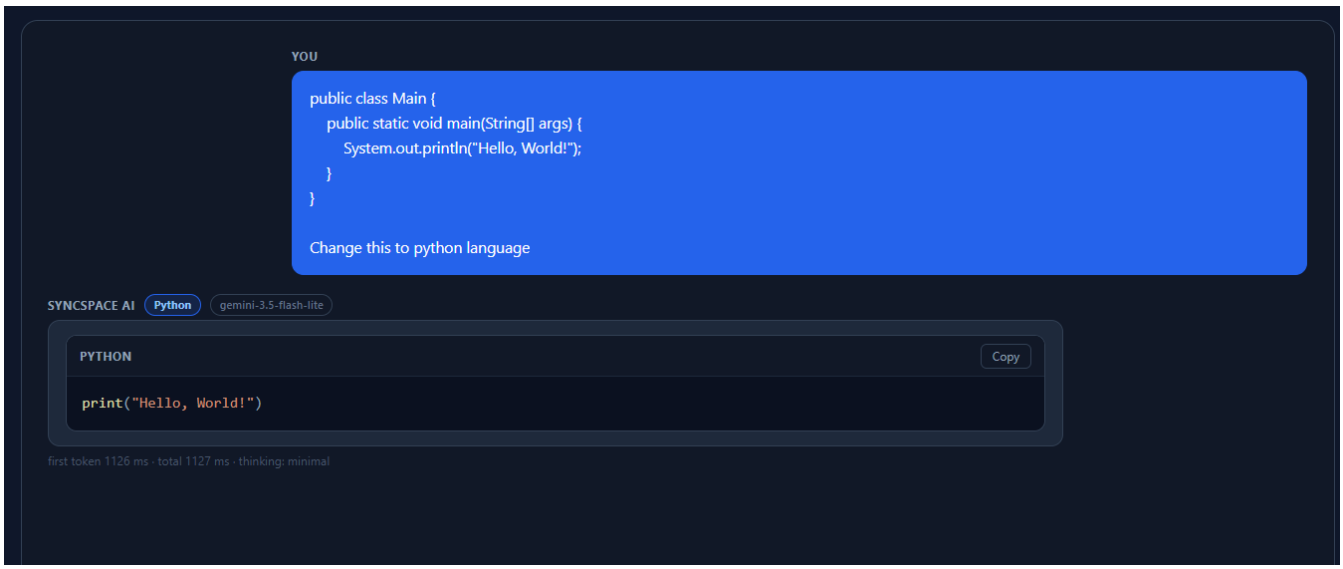


Figure 14.16: Mid-stream rendering — an unterminated fence displayed as an open code block

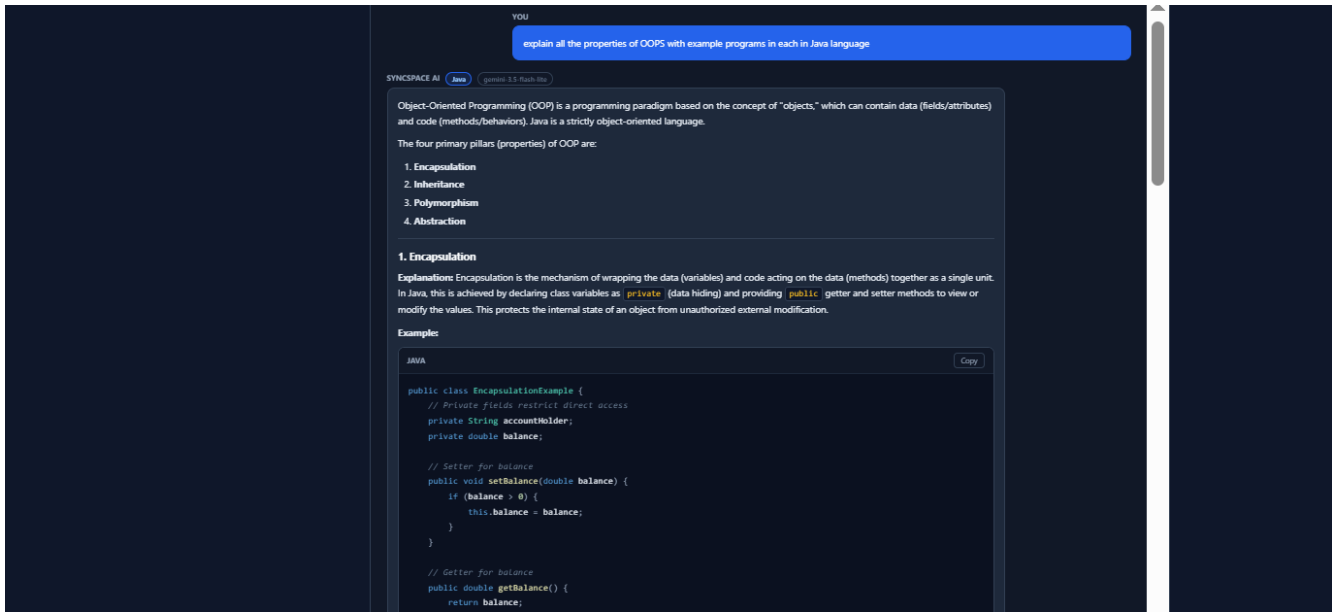


Figure 14.17: Rendered Markdown across the supported constructs

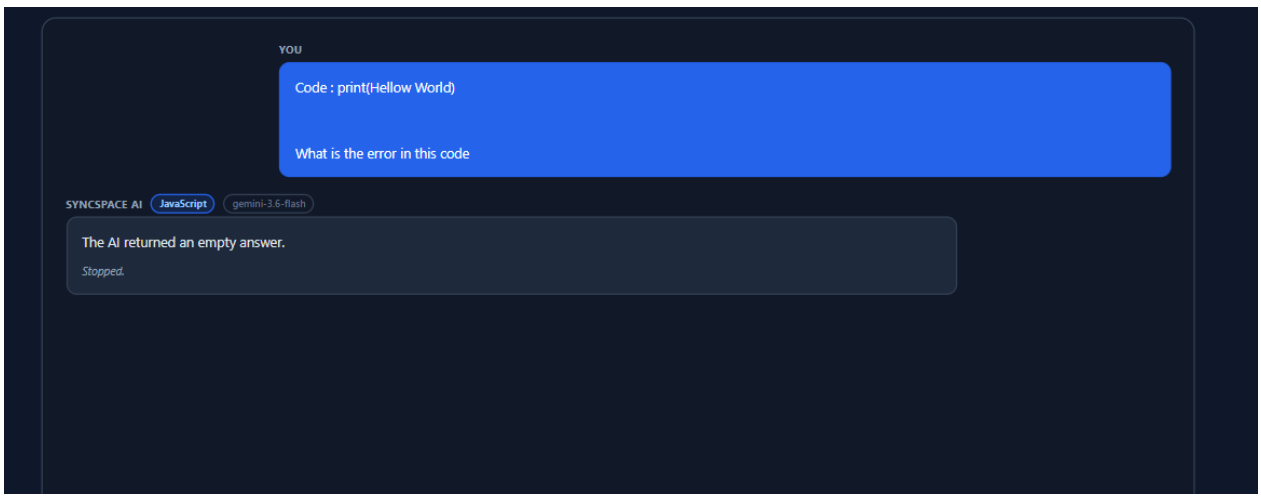


Figure 14.18: A sanitised provider error

14.7 Revised Results Summary

Table 11.2 of the original edition mapped the project's stated objectives to delivered outcomes. Table 14.4 records the outcomes added since.

Table 14.4: Additional delivered outcomes

No.	Objective	Evidence	Status
13	Optional global identity, layered above the workspace-scoped model	`User` model, account service, three token classes; anonymous flows unchanged (Chapter 12)	Delivered
14	Workspace listing and one-click re-entry	Dashboard; `/enter` with server-side membership re-check (Sections 12.6, 14.4)	Delivered
15	Re-entry without re-approval	`requestJoin` short-circuits to `enterWorkspace` for an existing member (Section 12.6)	Delivered
16	An embedded programming assistant	Nine actions over a tiered, streamed provider integration (Chapter 13)	Delivered
17	Correct language identity in assistant requests	Deterministic resolver with an explicit priority order; 92 backend checks (Sections 13.4, 13.13)	Delivered
18	Interactive assistant latency	Streamed transport with headers flushed before the provider call; explicit thinking levels and output ceilings (Sections 13.7, 13.8)	Delivered
19	Safe rendering of untrusted model output	Dependency-free parser emitting a token tree; no HTML string; schemes restricted (Section 13.10)	Delivered
20	Assistant endpoint brought to the project's security baseline	Authentication, live membership re-check, per-identity rate limiting, payload caps, sanitised errors (Section 13.11)	Delivered
21	A landing page that explains the system	Six feature cards, three-step explanation, account-aware navigation (Section 14.2)	Delivered
22	Conversation memory in the assistant	Each request is independent; deliberately deferred (Section 13.14)	Deferred
23	Account token revocation	No blacklist or refresh rotation; bounded by what a user token grants (Section 12.9)	Deferred

Table 14.5: Revised consolidated test results

Suite	Checks	Scope
`frontend/test-rendering.mjs`	156	Canvas, shapes, properties, effects
`frontend/test-brushes.mjs`	43	Brush registry and stroke geometry
`frontend/test-shapes.mjs`	11	Shape registry
`frontend/test-connectors.mjs`	28	Connector routing and attachment
`frontend/test-replay.mjs`	38	Replay reconstruction
`frontend/test-ai-render.mjs`	**62**	**Markdown parsing, streaming safety, highlighting**
`backend/test-workspace.mjs`	15	Workspace lifecycle and permissions
`backend/test-replay.mjs`	27	Replay transport
`backend/test-updatelog.mjs`	16	Update log and coalescing
`backend/stress-replay.mjs`	8	Capacity under load
`backend/test-ai.mjs`	**92**	**Language identity, planning, tiering, streaming, failure handling**
Total	**496**	

The AI suites contribute 154 of the 496 checks. That proportion is deliberate: the assistant is the only subsystem whose correctness depends on an external service, so it is the one place where a fake — a mock speaking the provider's real wire format — carries the weight that a live integration test would otherwise have to.

CHAPTER 15

Conclusion and Future Enhancements

This chapter revises and supersedes Chapter 12 of the original edition. The architectural assessment there remains correct and is restated; what changes is the account of what has been delivered, and the enhancement table, from which two entries have been removed because they are now implemented.

15.1 Advantages of the Proposed System

15.1.1 Architectural advantages

Convergence is structural, not procedural. The system does not resolve conflicts; it makes them unrepresentable. Because the shared document is a CRDT whose merge is commutative, associative and idempotent, no server arbitration exists to be a bottleneck, a single point of failure, or a source of subtle ordering bugs. Every claim about eventual consistency in this report follows from the algebra of the merge rather than from the correctness of any code written for this project.

One document, many features. Chat, presence, the code editor, the whiteboard and session replay are not five subsystems that were separately made collaborative. They are five views over one replicated document and one update log. The clearest illustration remains that the replay feature replays the code editor for free — not because that was implemented, but because there was never a second thing to implement.

Authorisation lives at the transport layer. A participant without an access token does not have their document messages rejected; they have no document message handler registered at all. The waiting room is not a screen that hides the workspace, it is a socket with nothing attached to it. This is why the permission boundary cannot be bypassed by a crafted client.

Provider independence, twice over. The execution service knows nothing about Judge0, Piston or paiza; it knows a provider interface and a canonical result. The AI module now repeats that shape: the planner produces a plain object and the provider adapter consumes it, so the assistant's language and model decisions are testable without a network, a key, or a cent of spend. In both cases the abstraction was justified by a real

substitution — the public Piston API closing, and a mock speaking the AI provider's real wire format.

Identity is layered, not replaced. The account system added in Chapter 12 sits above the workspace-scoped model. It introduced a third token class and one new endpoint, and it changed no existing contract. The system works identically for a user who never creates an account, which is the strongest available evidence that the layering is real rather than nominal.

15.1.2 Operational advantages

- **No infrastructure is required to run it.** MongoDB is optional; without it the system runs in memory. Code execution is remote, so no compilers, SDKs, PATH entries or container engine are needed on the development machine or the server. The AI module needs one API key and degrades to a clear, honest message without it.
- **Failure is data, not an exception.** Execution, replay and the assistant all resolve with a structured result describing what went wrong, which is why the interface can say something specific instead of "something went wrong".
- **The test suite runs without a network.** All 496 checks execute against local processes and local mocks.
- **Latency is a design parameter.** The assistant's thinking level, model tier and output ceiling are chosen per request rather than inherited from a provider default — which is the difference between a one-second answer and a one-minute one for the same question.

15.2 Limitations

Stated as plainly as the original edition stated them, with the two that the account and AI work removed and three that it added.

1. **The server is single-process.** The authoritative late-joiner document and the update sequence counter live in one process's memory, so the system does not scale horizontally without the shared-storage work described below.
2. **History is capped.** At 50 000 entries or 32 MB per room, recording stops and the interface says so. An honest partial history is preferable to an unbounded one, but it is a ceiling.

3. **Cursor awareness is unthrottled.** The 45 ms throttle applied to document writes was never applied to awareness cursor updates.

4. **There is no mobile or touch layout.** The two-pane shell assumes a pointer and a wide viewport.

5. **Account tokens cannot be revoked before expiry.** There is no blacklist and no refresh rotation. The exposure is bounded — a user token lists rooms and re-enters rooms whose membership the server still confirms — but a stolen token remains valid for up to thirty days.

6. **The assistant has no memory.** Each request is independent, so follow-up questions do not see the previous answer. This was a deliberate latency decision, not an oversight.

7. **The assistant cannot see the workspace.** It receives only what the user sends it plus an optional filename. It has no access to the whiteboard, the shared document or the chat, and generated code is not executed before being displayed.

15.3 Future Enhancements

Table 15.1: Proposed future enhancements (revised)

Enhancement	Description	Effort	Priority
Cursor throttling	Apply the existing 45 ms throttle pattern to awareness cursor writes	Low	High
Horizontal scaling	Move the authoritative document and sequence counter into shared storage; introduce a Socket.IO adapter for cross-process rooms	High	High
Accessibility	Keyboard-driven shape creation, screen-reader descriptions, contrast auditing	Medium	High

Continued on next page

Table 15.1 — continued from previous page

Enhancement	Description	Effort	Priority
Token rotation and revocation	Short-lived account tokens with a refresh endpoint and a revocation list, so a stolen user token can be invalidated before its thirty days elapse	Medium	High
Assistant conversation memory	Carry a bounded window of prior turns, with an explicit token budget so the latency guarantees of Section 13.7 survive it	Medium	High
Workspace-aware assistant	Let the assistant read the current editor buffer and selection on request, and offer to run generated code through the existing execution service	Medium	Medium
Container-level execution	Wrap the optional local runner in a container; the interface is unchanged, which is exactly why every process spawn lives in one file	Medium	Medium
Offline-first operation	Add a service worker and offline shell; Yjs already merges offline edits on reconnection, so the CRDT work is done	Medium	Medium
Invitation by account	Now that accounts exist, invite a known user into a workspace directly instead of sharing an identifier and a secret code	Low	Medium
Replay export	Export a reconstructed session as a video or an animated sequence for asynchronous review	Medium	Medium
Responsive and touch layout	Stacked panes and touch-appropriate drawing affordances for tablets	Medium	Medium

Continued on next page

Table 15.1 — continued from previous page

Enhancement	Description	Effort	Priority
Comment threads	Anchored comments as a new record type, inheriting sync and replay automatically	Low	Medium
Vector export	SVG and PDF export alongside PNG, preserving the structured object model	Low	Medium
Live replay streaming	Stream new updates into an open replay panel behind an explicit “follow live” toggle	Low	Low
Additional languages	Extend the registry; each addition is one data entry plus provider identifiers	Low	Low
Templates and libraries	Reusable diagram templates and a shape library, storable as ordinary records	Low	Low
Voice and video	Add WebRTC channels alongside the existing chat	High	Low

Removed from the original table

Global identity layer and, in substance, the assistant that motivated much of the AI work are no longer proposals. They are documented as delivered in Chapters 12 and 13 and evidenced in Table 14.4. Two new entries — token rotation and assistant memory — take their place, both of which exist because implementing the first two revealed them.

15.4 Conclusion

This project set out to build a system in which several people can draw and write code in the same space at the same time, without the losses, locks and merge dialogues that make naive shared editing unusable. That objective was met, and the reason it was met is a single decision taken early: the authoritative state was made a replicated conflict-free data type rather than a centrally arbitrated one. Almost every property the system can claim — convergence without coordination, offline tolerance, late-joiner synchronisation, a replay that reconstructs the whiteboard and the editor together — is downstream of that decision rather than of any code written to enforce it.

The work documented in this revised edition tested that claim in a way the original could not, because it added two subsystems the original architecture was never designed around. Neither required the architecture to bend. The account layer added a third identity without weakening the first two, and left every anonymous flow untouched, because authorisation had always been checked at the point of use rather than inferred from possession of a token. The AI assistant added an external, non-deterministic, rate-limited dependency to a system that previously had none, and it fits behind the same provider-interface shape the execution service had already proven — the planner produces a plain object, the adapter consumes it, and the mock speaking the provider's wire format carries the test weight. An architecture that absorbs two unplanned subsystems without structural change is a better-evidenced architecture than one that has only ever run the code it was drawn for.

The AI work also produced the more uncomfortable lesson. The reported defect — a request for Java answered in JavaScript — was not a model failure and not a hard problem. It was a pipeline that passed fields through without ever reconciling them, so that two contradictory statements reached the model with no priority between them, and the model obeyed the one the prompt declared authoritative. It was correct behaviour given incoherent input. The fix was to put a resolution step where there had been none, and to move the answer into the turn that outranks the other. The one-to-two-minute latency had the same character: nothing was slow, but a buffered endpoint and an inherited default thinking level compounded into a wait that looked like a hang. Both defects were failures to *decide* — to make an explicit choice where the code had left a default in charge. That is a more general lesson than either bug, and it is the one this project would carry forward.

What the system does not do is stated throughout rather than hidden: it runs in one process, it caps its own history and says so, its account tokens outlive the ability to revoke them, and its assistant has no memory of the previous turn. Each of these is a bounded, named limitation with a route to resolution in Table 15.1, which is a different thing from an unknown one. A system that reports its own ceiling honestly is more useful than one that pretends not to have a ceiling, and building it that way was as deliberate as the CRDT.

References

[1] C. A. Ellis and S. J. Gibbs, “Concurrency control in groupware systems,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, Portland, Oregon, 1989, pp. 399–407.

[2] C. Sun and C. Ellis, “Operational transformation in real-time group editors: issues, algorithms, and achievements,” in *Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW)*, Seattle, Washington, 1998, pp. 59–68.

[3] G. Oster, P. Urso, P. Molli and A. Imine, “Data consistency for P2P collaborative editing,” in *Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW)*, Banff, Alberta, 2006, pp. 259–268.

[4] M. Shapiro, N. Preguiça, C. Baquero and M. Zawirski, “Conflict-free replicated data types,” in *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, Grenoble, France, 2011, pp. 386–400.

[5] N. Preguiça, “Conflict-free replicated data types: an overview,” arXiv preprint arXiv:1806.10254, 2018.

[6] P. Nicolaescu, K. Jahns, M. Derntl and R. Klamma, “Near real-time peer-to-peer shared editing on extensible data types,” in *Proceedings of the 19th International Conference on Supporting Group Work (GROUP)*, Sanibel Island, Florida, 2016, pp. 39–49.

[7] C. Gutwin and S. Greenberg, “A descriptive framework of workspace awareness for real-time groupware,” *Computer Supported Cooperative Work (CSCW)*, vol. 11, no. 3–4, pp. 411–446, 2002.

[8] M. Fowler, “Event sourcing,” martinfowler.com, 2005. [Online]. Available: `https://martinfowl`

[9] V. Vernon, *Implementing Domain-Driven Design*. Boston, Massachusetts: Addison-Wesley Professional, 2013.

[10] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, Internet Engineering Task Force, December 2011. [Online]. Available: `https://www.rfc-editor.org/rfc/rfc64`

[11] I. Hickson, “Server-Sent Events,” W3C Recommendation, World Wide Web Consortium, 2015. [Online]. Available: https://www.w3.org/TR/eventsource/

[12] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc

[13] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proceedings of the USENIX Annual Technical Conference*, Monterey, California, 1999, pp. 81–91.

[14] OWASP Foundation, *Authentication Cheat Sheet*. OWASP Cheat Sheet Series, 2024. [Online]. Available: https://cheatsheetseries.owasp.org/

[15] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, New Orleans, Louisiana, 2022.

[16] Y. Zhou, A. Ioan Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan and J. Ba, “Large language models are human-level prompt engineers,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, Kigali, Rwanda, 2023.

[17] OWASP Foundation, *OWASP Top 10 for Large Language Model Applications*, version 1.1, 2024. [Online]. Available: https://owasp.org/www-project-top-10-for-large-lar

[18] Google, *Gemini API: Text Generation and Streaming*. Google AI for Developers documentation, 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs

[19] J. Gruber, “Markdown syntax documentation,” daringfireball.net, 2004. [Online]. Available: https://daringfireball.net/projects/markdown/syntax

[20] R. Fielding and J. Reschke, “Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing,” RFC 7230, Internet Engineering Task Force, June 2014. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7230

CHAPTER A

Appendices

A.1 Appendix A — Declaration of Assumptions and Inferred Content

The body of this report is derived from the project source code, the Axlero project specification, the SyncSpace Progress Tracker dated 27 July, and the four in-repository design documents. This appendix isolates every statement in the report that was *not* directly evidenced by those sources, so that a reader or examiner can distinguish documented fact from reasonable inference.

A.1.1 A.1 — Placeholders Requiring Completion

Values that must be supplied before submission

The following appear on the cover page, certificate and declaration and are collected as editable macros at the top of `main.tex`. They were not present in any provided source and are shown as bracketed placeholders:

- University Seat Number (`\studentUSN`)
- Internal guide name and designation
- External guide name
- Head of Department and Principal names
- Academic year, if different from the value shown
- The institute emblem, for which a dashed placeholder is drawn

The student name, degree programme and institution were taken from the project context and should still be verified.

A.1.2 A.2 — Inferred Technical Content

Explanations inferred rather than documented

1. **Hardware requirements** (Section 3.3) were inferred from the workload profile. No hardware specification appeared in any source document.
2. **The browser compatibility matrix** (Table 3.8) was inferred from the minimum browser versions supporting the APIs used. Only Chrome is evidenced as an

actually exercised platform; the remaining entries are reasoned, not measured. The note about `multiply` blend variation across engines is a known characteristic of the composite operation, not an observed defect.

3. **Non-functional requirements** (Table 3.6) were derived from design decisions visible in the code and its comments. They were not stated as requirements anywhere in the source material.
4. **Use case specifications** (Tables 5.1 and 5.2) were reconstructed from the implemented control flow.
5. **The coalescing effect figures** (Table 8.4) are arithmetic estimates derived from the documented 45 ms drag throttle and 400 ms coalescing window. Only the aggregate figure — a 5,206-update session recording in full where the old flat cap engaged — is directly evidenced.
6. **The line-count estimate** of approximately 12,000 lines in Chapter 12 excludes lock files, dependencies and generated output, and is approximate.
7. **Design pattern attributions** (Table 4.3) name patterns the code structurally exhibits. The source does not use these pattern names itself.
8. **Deployment topology** (Figure 4.4) reflects the deployment targets named in the tracker (Vercel and Render). Only the frontend deployment readiness is evidenced as complete; the backend deployment is marked optional in the tracker and is presented here as an architecture, not as an accomplished deployment.
9. **All screenshots** in Chapter 11 are dashed placeholders accompanied by annotated wireframes. The wireframes were constructed from the component source and describe the intended layout; they are not renderings of the running application and may differ in spacing, colour and exact control placement from the real interface.

A.1.3 A.3 — Divergence Between the Progress Tracker and the Source Code

Where the code has advanced beyond the 27 July tracker

The instruction governing this report was to treat the 27 July Progress Tracker as authoritative where it conflicts with older documents. The submitted source code, however, contains work completed *after* that tracker was written. Where the two disagree, this report documents **the code as submitted**, because a project report must describe the artefact being submitted. Every such divergence is listed below.

Area	27 July Tracker	Submitted source (documented here)
Code execution	Local sandbox forking five compilers on the server, confined with POSIX `ulimit`	Remote providers (Judge0, Piston, paiza.io) behind an adapter chain; the local runner survives only as an explicitly opt-in development escape hatch
Replay history cap	Flat 5,000 entries	50,000 entries *and* 32 MB, whichever is reached first
Update coalescing	Not present	Consecutive same-user updates merged within a 400 ms window via `Y.mergeUpdates`
Test totals	287 checks across eight suites	442 checks across ten suites
Rendering suite	124 checks	156 checks, after a stale-API defect in the suite itself was repaired
Brush suite	29 checks	43 checks, after the eraser densification work
Update-log suite	Not present	`test-updatelog.mjs`, 16 checks, no server required
Chat	Not mentioned	`ChatPanel.jsx` over `Y.Array('chatMessages')` with awareness typing indicators
Gradient and blur fills	Not mentioned	Linear and radial gradients and a blur filter, with three defects in the original implementation repaired
Images and stickers	Not mentioned	Image records sized from natural dimensions, replayed through the existing pipeline

Record normalisation	Not mentioned	`canvas/normalize.js` as an explicit gate between the document and the renderer
Eraser algorithm	Circle stamped along the drag	Swept capsule with per-stroke densification
Socket receive limit	Not mentioned	`maxHttpBufferSize` raised to 8 MB

Two consequences follow. First, statements in the tracker describing local compilation, `ulimit` confinement or a 5,000-entry cap are superseded. Second, the tracker's narrative of milestone sequencing remains accurate and is used as such throughout Chapters 6 and 7.

A.1.4 A.4 — Statements Carried Directly from Source Documents

The following are quoted or closely paraphrased from the in-repository design documents and are therefore evidenced rather than inferred: the Socket.IO ten-attachment ceiling and its consequences; the duplicate sequence-number defect and its resolution; the measured replay figures of approximately 43 ms streaming and 175 ms reconstruction across seventeen chunks; the two sandbox defects found by the execution suite; the gradient, blur, layer-order and image-sizing defects found during the quality pass; the impossibility of installing MongoDB 8.x on the development machine; and the enumerated list of verification activities that were *not* performed.

A.2 Appendix B — Source File Inventory

Table A.2: Backend source files

File	Responsibility
`server.js`	Composition root; middleware, routes, error handlers, socket attachment
`config/db.js`	MongoDB connection with graceful degradation to memory-only mode
`models/Workspace.js`	Workspace schema with embedded members and pending requests

Continued on next page

Table A.2 – continued from previous page

File	Responsibility
`models/DocState.js`	Binary Yjs snapshot, one row per room
`models/UpdateLog.js`	Append-only update history with a compound unique index
`routes/workspaceRoutes.js`	Workspace REST routing with authorisation guards
`routes/executeRoutes.js`	Execution endpoints behind `requireMember`
`controllers/workspaceController.js`	Request validation and HTTP translation
`middleware/authMiddleware.js`	`requireMember` and `requireAdmin`
`services/socketService.js`	Handshake authentication, relay, replay streaming, admin actions
`services/workspaceService.js`	All workspace business rules, called by both transports
`services/workspaceStore.js`	Dual-mode workspace repository
`services/updateLogService.js`	Dual-mode update log with coalescing and budgets
`services/realtime.js`	Emit facade breaking the service/socket import cycle
`services/execution/index.js`	Provider-agnostic orchestrator with queue and fallback
`services/execution/languages.js`	The language registry
`services/execution/result.js`	Canonical result shape, capping, signal wording
`services/execution/http.js`	The only network call: deadlines, retries, redaction
`services/execution/providers/*.js`	One adapter each for Judge0, Piston, paiza.io and local
`utils/token.js`	Access token and lobby ticket signing and verification

Continued on next page

Table A.2 – continued from previous page

File	Responsibility
`utils/validate.js`	Input sanitisation, format validation, rate limiting
`utils/ids.js`	Unambiguous workspace identifiers and UUIDs

Table A.3: Frontend source files

File	Responsibility
`App.jsx`, `main.jsx`	Routing and application mount
`pages/Landing.jsx`	Two entry points: create or join
`pages/CreateWorkspace.jsx`	Creation form including the access policy choice
`pages/JoinWorkspace.jsx`	Join form
`pages/WaitingRoom.jsx`	Lobby socket and live verdict
`pages/Workspace.jsx`	Session shell, top bar, panel orchestration
`hooks/useCollaboration.js`	The Yjs provider over Socket.IO; admin and replay channels
`hooks/useToasts.js`	Transient notifications
`components/Canvas.jsx`	The whiteboard: tools, pointer handling, camera, selection
`components/Toolbar.jsx`	Tool and shape selection, undo and redo
`components/PropertyPanel.jsx`	Contextual property editing
`components/BrushPanel.jsx`	Brush and eraser settings with live preview

Continued on next page

Table A.3 – continued from previous page

File	Responsibility
components/Editor.jsx	Monaco binding, language selection, execution, shared console
components/ReplaySlider.jsx	The replay viewer and scrubber
components/AdminPanel.jsx	Join queue, policy switch, member management
components/ChatPanel.jsx	Workspace chat over the shared document
components/ErrorBoundary.jsx	Application and per-shape failure containment
canvas/shapes.jsx	Shape registry, defaults, geometry generators, icons
canvas/shapeDoc.js	The single write path into the shared document
canvas/normalize.js	Record sanitisation gate
canvas/connectors.js	Connector geometry, routing and snapping
canvas/brushes.js	Brush registry, simplification, smoothing, eraser mathematics
canvas/replay.js	Reconstruction, checkpointed cache, chunk unpacking
canvas/ShapeNode.jsx	Shape rendering with fill, gradient, shadow and blur
canvas/ConnectorNode.jsx	Custom connector scene and hit functions
utils/socket.js	Authenticated socket factory and colour derivation
utils/session.js	Tab-scoped token storage
utils/api.js	REST client with per-request deadlines

A.3 Appendix C — Sample Environment Configuration

```
# --- core ------------------------------------------------------------
PORT=5000
CLIENT_ORIGIN=http://localhost:5173
JWT_SECRET=replace-with-a-long-random-value-in-production
MONGO_URI=                      # empty selects memory-only mode

# --- code execution (see EXECUTION.md) -------------------------------
# Code runs in a REMOTE sandbox. No compilers, SDKs, PATH entries or
# container engine are required on this machine or on the server.
# EXEC_PROVIDERS=judge0,piston,paiza

# Judge0 (recommended)
# JUDGE0_URL=https://judge0-ce.p.sulu.sh
# JUDGE0_KEY=your-key

# Piston -- only if you self-host it (the public API closed Feb 2026)
# PISTON_URL=http://localhost:2000/api/v2

# paiza.io fallback: works with no signup, rate limited, best effort
# PAIZA_ENABLED=false

# --- limits ----------------------------------------------------------
# EXEC_RUN_TIMEOUT_MS=8000
# EXEC_MEMORY_KB=128000
# EXEC_MAX_CONCURRENT=8
# EXEC_RATE_MAX=20
# EXEC_LOG=off

# --- development escape hatch: NOT for a deployed server ---------------
# EXEC_PROVIDERS=local
# EXEC_ALLOW_LOCAL=true
```

Listing A.1: Backend `.env` template

```
VITE_SERVER_URL=http://localhost:5000
```

Listing A.2: Frontend `.env` template

A.4 Appendix D — Demonstration Script

The following sequence exercises every major subsystem in approximately three minutes and is recommended for a viva demonstration.

1. **Isolation.** Open window 1, create workspace “alpha” with the approval policy, and draw a large scribble. Open window 2, create workspace “beta”. It must be completely blank. Draw in beta; window 1 must not change by a single pixel.
2. **The permission boundary.** From window 3, join alpha with the correct secret code. Observe that the correct code is *not* sufficient — the joiner is held in the waiting room. Point out that this socket has no synchronisation handler registered at all.
3. **Approval.** In window 1 the administrator panel has opened automatically with a badge. Approve. Window 3 enters without refreshing, because the access token was pushed down the open lobby socket.
4. **Late-joiner synchronisation.** Window 3 immediately shows window 1’s existing scribble — the full document state is transferred on join.
5. **Concurrent merge.** In window 1 change a rectangle’s fill; in window 3 drag the same rectangle simultaneously. Both changes survive on both replicas.
6. **Connectors.** Press `C` and drag from a rectangle to a circle, showing the green snap ring. Move the circle: the connector follows in both windows, re-routing frame by frame. Double-click the line to insert a bend; switch it to curved in the property panel.
7. **Single-step undo.** Drag a shape a long distance and press Ctrl+Z once. The entire drag reverses as one step despite having emitted dozens of intermediate commits.
8. **The partial eraser.** Draw a long stroke and rub out its middle. Two independent strokes remain. Press Ctrl+Z: the original stroke is restored intact. Press Ctrl+Y: it re-splits.
9. **Shared execution.** Select Python in the editor, type a `print` statement, add stdin, and press Run. The result appears in *both* windows’ consoles, attributed to whoever ran it.
10. **Runtime policy switch.** In the administrator panel switch to password-only. Existing participants stay connected; only future joiners follow the new rule.
11. **Removal.** Remove window 3’s participant. Their socket is disconnected immediately and their token stops working on the next connection attempt.
12. **Session replay.** Open Replay in window 1. Drag the slider to zero — the board is blank. Press play and watch the session rebuild itself with per-update attribution while the code pane fills in alongside. Then point at window 2: it never moved. Replay is a second document.

A.5 Appendix E — Glossary

Term	Definition
Awareness	Ephemeral, non-persisted per-client state: cursor position, selection, in-progress stroke, typing indicator. Deliberately excluded from the document, from persistence and from replay
Capped	The state of a room whose update log has reached its entry or byte ceiling. Recording stops and the interface says so
Checkpoint	An encoded document snapshot laid down periodically by the replay cache so a backward scrub restores from the nearest one instead of rebuilding from zero
Coalescing	Merging consecutive same-user updates inside a short window into a single log entry using `Y.mergeUpdates`
CRDT	Conflict-free Replicated Data Type: a structure whose merge is commutative, associative and idempotent, so replicas converge without coordination
Derived endpoint	A connector endpoint whose position is computed at render time from the referenced shape rather than stored
Lobby ticket	A signed token granting only the waiting room; it cannot reach the document
Live origin	A transaction origin used for high-frequency mid-drag writes, excluded from the undo stack
Normalisation	Coercing every record read from the document into safe, drawable values before it reaches a renderer
Prefix validity	The property that applying the first n updates of a causal log yields a complete, consistent document — and that a suffix does not
Snapshot	The binary encoding of an entire document state, produced by `Y.encodeStateAsUpdate`
Survive-run	A contiguous span of stroke points left intact after an eraser pass, re-emitted as an independent stroke
Workspace	An access-controlled collaborative room; also the unit of isolation, persistence and replay

CHAPTER B

Appendices — Revised Edition

B.1 Appendix F — Declaration for the Revised Edition

Appendices A to E of the original edition remain applicable to Chapters 1–11 and are not restated. This appendix declares the assumptions and the provenance of the material added in Chapters 12–15.

B.1.1 F.1 — Placeholders requiring completion

Every framed region in Chapter 14 is a reserved space, not a result. Sixteen screen captures are outstanding; each is described in the caption of the figure that reserves it. The report makes no claim about a screen until its capture is in place.

B.1.2 F.2 — Content derived directly from the submitted source

The following are documented from the source code and its accompanying module notes rather than inferred: the three token classes and their claims and lifetimes; the `User` schema; the account service rules including the bcrypt cost factor, the twelve-attempts-per-minute login limit and the identical failure message; the middleware behaviour of `requireUser` and `optionalUser`; the account-linking behaviour of workspace creation, joining and approval; the `/enter` endpoint's membership re-check; the AI module's field limits, action-to-field mapping, complexity thresholds, output ceilings, model tiers, thinking levels, SSE event types and error classification; and the check counts of the two AI test suites.

B.1.3 F.3 — Inferred or reasoned content

The following are reasoned rather than measured, and are marked as such where they appear: the threat tables in Sections 12.9 and 13.11, which enumerate plausible attacks against the implemented mitigations rather than the results of a penetration test; the assessment that the account layer's residual token-revocation risk is bounded; and the effort and priority ratings in Table 15.1, which are judgements.

B.1.4 F.4 — Provider figures

The comparative model figures quoted in Section 13.7 (54.2 % against 49.6 % on SWE-Bench Pro) are the provider's published launch figures, carried from the module documentation. They were not independently reproduced, and the report does not rely on them for any claim beyond the tier assignment.

B.1.5 F.5 — Statement on latency

Section 13.1.2 attributes the reported one-to-two-minute latency to a buffered endpoint compounded with an inherited thinking level. This is a diagnosis supported by the benchmark harness `bench-ai.mjs`, which measures four configurations against the live provider. The specific wall-clock figures depend on network conditions and provider load and are therefore not quoted as results in this report.

B.2 Appendix G — Revised Source File Inventory

Files added or substantially changed since the original edition. Table A.2 and Table A.3 of the original edition remain accurate for everything not listed here.

Table B.1: Backend files added or changed

File	Responsibility
`models/User.js`	Account document; denormalised membership index
`services/userStore.js`	Repository for accounts; MongoDB or in-memory
`services/authService.js`	Sign-up, sign-in, account read with live memberships
`routes/authRoutes.js`	`/api/auth/signup`, `/login`, `/me`
`middleware/authMiddleware.js`	*Changed*: adds `requireUser` and `optionalUser`
`utils/token.js`	*Changed*: adds the `user` token class
`services/workspaceService.js`	*Changed*: account-linked create, join, approve; `enterWorkspace`
`services/workspaceStore.js`	*Changed*: `findWorkspacesByUser`

Continued on next page

Continued from previous page

File	Responsibility
`controllers/workspaceControl.js`	*Changed*: threads the optional account id
`routes/workspaceRoutes.js`	*Changed*: `/:id/enter`; optional-user mounting
`routes/aiRoutes.js`	AI endpoints; access control, rate limit, SSE route with back-pressure
`services/ai/aiService.js`	Provider adapter; tiering, thinking level, streaming, error classification
`services/ai/request.js`	The planner: resolve, classify, build; provider-free clarifications
`services/ai/languages.js`	Deterministic language identity, conversion direction, code inference
`services/ai/prompts.js`	Per-action system instructions, assembled per request
`services/ai/validators.js`	Field caps, sanitising, action validation
`test-support/mock-gemini-api.mjs`	Mock provider speaking the real wire format, with a modelled thinking stall
`test-ai.mjs`	92 checks across eleven sections
`bench-ai.mjs`	Live latency benchmark across four configurations
`server.js`	*Changed*: mounts the auth and AI routers

Table B.2: Frontend files added or changed

File	Responsibility
`pages/Landing.jsx`	*Rebuilt*: navigation, hero, features, steps, call to action, footer
`pages/Auth.jsx`	Combined sign-in and sign-up, switchable in place
`pages/Dashboard.jsx`	Workspace listing and one-click re-entry

Continued on next page

Continued from previous page

File	Responsibility
`hooks/useAuth.js`	Account state, storage hydration, token re-validation
`ai/AIPage.jsx`	Assistant page: language state, streaming buffer, cancellation
`ai/AIToolbar.jsx`	Action tabs, language and target-language selectors
`ai/AIChat.jsx`	Input, autoscroll, stop control
`ai/AIMessage.jsx`	One transcript entry with its metadata chip
`ai/Markdown.jsx`	Token tree to React elements; no HTML string
`ai/markdown.js`	Streaming-tolerant Markdown parser
`ai/highlight.js`	Dependency-free tokeniser for code blocks
`ai/aiApi.js`	Single request shape; SSE client with connect and stall watchdogs
`ai/ai.css`	Assistant styling
`App.jsx`	*Changed*: adds `/login`, `/signup`, `/dashboard` and the assistant route
`test-ai-render.mjs`	62 checks including every-prefix streaming safety

B.3 Appendix H — Revised Environment Configuration

The backend template of Appendix C gains the following block. The API key is read only inside `services/ai/aiService.js` and never crosses the HTTP boundary.

```
# --- SyncSpace AI (see AI_MODULE.md) ----------------------------------
# Never place this key in a VITE_ variable: anything so prefixed is
# compiled into the public frontend bundle.
GEMINI_API_KEY=your-key-here

# Model tiers. Simple work uses FAST; work where being wrong costs more
# than being slow uses SMART. Defaults shown.
# AI_MODEL_FAST=gemini-3.5-flash-lite
# AI_MODEL_SMART=gemini-3.6-flash

# Pin ONE model across both tiers, overriding tiering entirely. Useful for
# A/B testing; leave UNSET in normal use.
# GEMINI_MODEL=

# Thinking level: the dominant latency control.
# Allowed: minimal | low | medium | high
# AI_THINKING_FAST=minimal
# AI_THINKING_SMART=low
```

```
19
20 # Timeouts and limits
21 # AI_TIMEOUT_MS=60000          whole-generation ceiling
22 # AI_STALL_TIMEOUT_MS=20000    give up if the provider goes quiet mid-stream
23 # AI_RATE_MAX=20               requests per identity per window
24 # AI_RATE_WINDOW_MS=60000
25
26 # NOTE: temperature / top_p / top_k are deliberately NOT sent. They are
27 # deprecated for this model generation and ignored by the API.
```

B.4 Appendix I — Revised Demonstration Script

The following continues the twelve-step script of Appendix D and exercises the subsystems added in this edition. It takes approximately two further minutes.

13. **Anonymity still works.** Before signing in to anything, create a workspace and draw in it. Point out that nothing asked for an account. This is the control case for everything that follows.

14. **Create an account.** Sign up. Note that the form itself says an account is optional.

15. **Account-linked creation.** Create a second workspace while signed in, then open the dashboard. The new workspace is listed with the *admin* role. The first, anonymous workspace is not — which is the point: linking happens at creation, not retroactively.

16. **Ambiguous failure.** Sign out and attempt to sign in with a wrong password, then with an email that was never registered. The message is identical. Explain that distinguishing them would turn the endpoint into an account-existence oracle.

17. **One-click re-entry.** Sign back in, and open a permission-mode workspace from the dashboard. Show the administrator's window throughout: the join queue never receives a request, because membership was re-checked rather than re-requested.

18. **Authorisation is still server-side.** As the administrator, remove that member. Their dashboard still shows the card — it is a cached index — but clicking *Open workspace* is refused with 403. This is the difference between an index and an authority.

19. **The assistant's language identity.** Open SyncSpace AI. Set the language selector to **JavaScript** deliberately, then ask for the hello world program *in java*. The answer is Java, and the metadata chip attributes the choice to the user's request. This is the reported defect, resolved.

20. **The stale dropdown.** Leave the selector on JavaScript and paste a Java file into *explain*. The chip now reports `supplied-code`: pasted code outranks a leftover default.

21. **Zero-latency clarification.** Ask to convert between two languages without saying which direction. The assistant asks, immediately, with no model name in the metadata — because no provider call was made.

22. **Streaming and cancellation.** Ask for something long. Text appears within a fraction of a second, code blocks render highlighted while still incomplete, and typing in the input box during generation is not blocked. Press *Stop* mid-answer: generation halts server-side, not just on screen.

23. **Honest failure.** Stop the backend and send one request. The interface reports that it cannot reach the AI server — a sentence, with no request URL, model path or quota identifier in it.

B.5 Appendix J — Additional Glossary Terms

Appendix E remains applicable. The following terms are added.

Term	Definition
Account token	A signed token of kind `user`, proving identity but granting no workspace access by itself
Action	One of the assistant's nine request types, each with its own permitted fields and system instruction
Back-pressure	Pausing consumption from the provider while the client's socket buffer is full, so a slow reader cannot grow server memory
Complexity class	The planner's judgement — simple, standard or deep — driving both the model tier and the thinking level
Language resolution	The deterministic decision, taken before any prompt is built, of which programming language a request concerns
Membership index	The denormalised `workspaces` array on an account; used to list rooms, never to authorise entry
Model tier	Fast or smart; which of two configured models handles a request

Term	Definition
Open code block	A fence that has begun but not yet closed, rendered as a code block rather than as literal backticks so streamed answers do not flicker
Planner	The pure function turning a validated request into a plan: resolved language, system instruction, user turn, ceilings and metadata
Server-Sent Events	A one-way text stream over the HTTP response that requested it; the assistant's transport
Stall timer	A deadline re-armed on every received chunk, catching a provider that accepts a connection and then goes quiet
System turn	The instruction channel that outranks the user's message; where the resolved language is pinned
Thinking level	The provider-side reasoning budget spent before the first visible token; the dominant latency control
User token	See *account token*

PROJECT CONTRIBUTION FROM TEAM

1. **Serah (reference link):** https://docs.google.com/spreadsheets/d/1pYPmTNRe8lIumB0kYWy0qnnScmMyn7ReqPaUIPz2Nw8/edit?gid=0#gid=0